
AU CŒUR DU MÉTAL

Commodore 64 & Ultimate 64

synthèse technique des sources primaires

De l'article de Christian Bauer (révision 2024) aux mesures
sur la réimplémentation FPGA de 2026 :
VIC-II · 6510 · SID · CIA · REU · Ultimate 64

Document d'étude à diffusion privée — juillet 2026

Rédaction assistée par IA (Claude) · audits croisés Codex et Claude Opus 5
sous la direction de **Johnny Piette**

Table des matières

Avant-propos	3
Ce qu'est ce document	3
Mode d'emploi des ancrs	3
Convention hexadécimale	3
Chapitre 1 — Le VIC-II : règles complètes (Bauer, révision 2024)	4
1. Contexte système : bus, phases, BA/AEC [Bauer §2.2–2.4]	4
2. Table complète des registres \$d000–\$d02e [Bauer §3.2]	4
3. Palette [Bauer §3.3]	5
4. Dimensions d'affichage, RSEL/CSEL, types de VIC [Bauer §3.4]	6
5. Bad Lines — définition exacte [Bauer §3.5]	7
6. Accès mémoire [Bauer §3.6]	7
7. État display / état idle [Bauer §3.7.1]	8
8. VC, RC, VMLI — les règles [Bauer §3.7.2]	8
9. Les 8 modes graphiques [Bauer §3.7.3]	9
10. Sprites [Bauer §3.8]	10
11. Unité de bordure [Bauer §3.9]	12
12. DEN (Display Enable) [Bauer §3.10]	12
13. Crayon optique [Bauer §3.11]	12
14. Interruptions [Bauer §3.12]	13
15. Rafraîchissement DRAM [Bauer §3.13]	13
16. Effets et applications [Bauer §3.14]	13
17. « Ghost bytes » : \$de00, Color RAM, adresses 0/1 [Bauer §4]	15
Chapitre 2 — Timing fin & vols de cycles (Mäkelä 1994, Ojala 1992, Åkesson 2016)	17
Chapitre 3 — Le 6510 : cycles et interruptions (64doc)	26
Chapitre 4 — SID & CIA (datasheet 6581, modèle Lorenz)	32
1. SID 6581 (Sound Interface Device)	32
2. CIA 6526 — le modèle logiciel de Wolfgang Lorenz	36
Chapitre 5 — Le REU : DMA \$DF00	39
5.1 Vue d'ensemble	39
5.2 Table des registres \$DF00–\$DF0A (verbatim, [REU-reg])	39
5.3 Les quatre commandes (bits 1..0 de \$DF01)	40
5.4 Pièges de programmation recensés dans les docs	41
5.5 Memmap firmware Ultimate-II/U64 en bref [memmap]	41
Chapitre 6 — L'Ultimate 64 : services (REST, streams, turbo, UCI)	42
1. Organisation de la documentation [ultimate64.rst]	42
2. API REST [api_calls.rst]	42
3. Flux de données réseau [data_streams.rst]	44
4. Mode turbo [turbo_mode.rst]	46
5. UCI — Ultimate Command Interface [core_uci_architecture.rst]	47
6. Cible DOS (targets \$01 / \$02) [ultimate_dos_target.rst]	48
7. Cible Control (target \$04), en bref [control_target.rst]	49
Annexe A — Le C64 Ultimate mesuré (firmware Commodore 1.0, juillet 2026)	51
A.1 Le turbo 64 MHz	51
A.2 Accès mémoire DMA via l'API REST	51
A.3 Le fork Commodore face à la documentation Gideon	52
A.4 Flux vidéo UDP	52

A.5 REU : DMA fiable, mise en place coûteuse	52
A.6 Manette au turbo	52
A.7 PETSCII et le piège des simulateurs	53
Annexe B — Étude de cas : servir du FLI 16 couleurs à 64 MHz par polling	54
B.1 Le principe : pas d'IRQ, le gel badline comme synchro	54
B.2 La fenêtre d'écriture de \$D011	54
B.3 La rotation des 8 matrices et son offset machine	55
B.4 Le FLI-bug : 24 pixels irréductibles, masquables	55
B.5 Le décalage +4,5 cellules : le piège du nibble	55
B.6 Sprites actifs : les vols DMA qui font glisser l'écriture	55
B.7 La leçon de calibration : un banc statique ment	56
B.8 Ce qui se transporte, ce qui ne se transporte pas	56
Sources et crédits	57
Références VIC-II et timing	57
SID, CIA, REU	57
Ultimate 64 / 1541 Ultimate-II	57
Brevets (cités par Bauer)	57
Note sur les mesures	58

Avant-propos

Document d'étude à diffusion privée — juillet 2026. Rédaction assistée par IA (Claude), audits croisés (Codex/GPT et Claude Opus 5), sous la direction de Johnny Piette.

Ce que ce document contient d'original : les **annexes A et B** — comportements du **C64 Ultimate** (réimplémentation FPGA, firmware Commodore 1.0) **mesurés en 2026**, et une étude de cas de FLI servi à 64 MHz constant par polling. Ces mesures ne figurent dans aucune source existante. Les chapitres 1 à 6 sont une **condensation-index** des sources classiques, chaque section ancrée vers le texte primaire : le lecteur qui connaît déjà Bauer peut aller directement aux annexes.

Ce volume est né d'une question simple : que reste-t-il des règles de 1982 quand la machine est réimplémentée en FPGA et son processeur cadencé à 64 MHz ? Pour y répondre, il fallait d'abord rassembler ces règles — éparses dans un article devenu canonique, des chronogrammes finlandais de 1994 et trois décennies de savoir de la scène démo — puis les confronter au matériel réel. Les chapitres 1 à 6 sont cette collecte, ancrée ligne à ligne ; les annexes sont la confrontation.

Ce qu'est ce document

Ce livre est une **synthèse fidèle des sources primaires publiques** sur le matériel du Commodore 64 — VIC-II, 6510, SID, CIA, REU — et sur le C64 Ultimate / Ultimate 64 qui le réimplémente en FPGA. Il condense la couche conceptuelle de ces textes **sans perte normative** : les règles numérotées, formules, tables de registres et diagrammes de cycles y sont repris quasi-verbatim. Chaque section est **ancrée** sur sa source pour permettre la vérification : pour toute décision critique au cycle près, vérifier contre le texte primaire ancré avant de coder. Le livre indexe ; les sources prouvent.

Mode d'emploi des ancres

Chaque affirmation normative porte une ancre entre crochets renvoyant à sa source primaire :

- **[Bauer §x.y]** — Christian Bauer, « The MOS 6567/6569 video controller (VIC-II) and its application in the Commodore 64 », révision 2024 (section x.y) ;
- **[pal.timing]** — Marko Mäkelä, chronogrammes mesurés du 6569 (1994) ;
- **[Ojala]** — Pasi Ojala, « Missing Cycles », C=Hacking #3 (1991/1992) ;
- **[MISC]** — Linus Åkesson, « Massively Interleaved Sprite Crunch » (2016) ;
- **[64doc l.x]** — John West & Marko Mäkelä, « Documentation for the NMOS 65xx/85xx Instruction Set », rév. 1.8 (1994), avec le numéro de ligne du texte source ;
- **[SID p.N]** — datasheet MOS 6581, numéro de page des scans ;
- **[Lorenz §...]** — Wolfgang Lorenz, « A Software Model of the CIA6526 » (1997) ;
- **[REU-prog]** / **[REU-reg]** — textes de programmation et table des registres REU ;
- **[memmap]** — memory map du firmware Ultimate ;
- **[api_calls.rst]**, **[data_streams.rst]**, etc. — fichiers de la documentation officielle Ultimate (dépôt GideonZ).

La **bibliographie** en fin de volume donne la référence complète de chaque ancre. Quand le texte signale « ambigu dans la source », l'ambiguïté est dans le document primaire lui-même, pas dans la synthèse.

Convention hexadécimale

Le chapitre 1 conserve les **minuscules de Bauer** (§d011) ; les autres chapitres suivent la casse de leurs sources respectives (majuscules le plus souvent, §D011).

Chapitre 1 — Le VIC-II : règles complètes (Bauer, révision 2024)

La charpente du volume : les règles de fonctionnement du VIC-II, condensées de l'article de référence de Christian Bauer (révision 2024). Dans l'ordre : le contexte système (le bus partagé), les registres, puis les mécanismes — Bad Lines, compteurs internes, modes graphiques, sprites, bordure — et les effets « impossibles » que la scène démo a bâtis sur leurs interstices. Ce chapitre se consulte plus qu'il ne se lit : chaque section porte son ancre vers le texte primaire.

(D'après : Christian Bauer, « The MOS 6567/6569 video controller (VIC-II) and its application in the Commodore 64 », révision 2024-09-29. Toutes les valeurs hex/cycles/registres sont reprises telles quelles de la source.)

1. Contexte système : bus, phases, BA/AEC [Bauer §2.2–2.4]

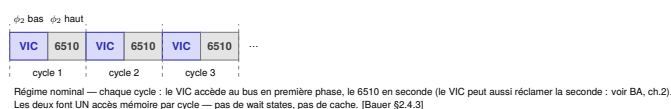


Figure 1.1 — Le partage du bus en régime nominal. Tout le reste du livre décrit les exceptions : les fenêtres où le VIC réclame AUSSI la seconde phase (§5, §10, et le chapitre 2 entier).

- Un cycle d'horloge (ϕ 2, 985,248 kHz PAL / 1022,7 kHz NTSC) = deux phases : **le VIC accède au bus en première phase (ϕ 2 bas), le 6510 en seconde phase (ϕ 2 haut)**. AEC suit ϕ 2 de près. Les deux puces font un accès mémoire à **chaque cycle**, même inutile (pas de wait states, pas de cache ; chaque accès tient en un cycle). Le VIC ne fait que des lectures. [Bauer §2.4.3]
- **BA** : normalement haut. Quand le VIC a besoin de la seconde phase (pointeurs de caractères : 40 cycles supplémentaires ; données sprite : 2 cycles par sprite), BA passe bas **3 cycles avant** la prise de bus (3 = nombre maximal d'écritures successives du 6510 ; RDY est ignoré en écriture). Après ces 3 cycles, AEC reste bas aussi en seconde phase et le VIC sort ses adresses. Un accès lecture du 6510 pendant BA bas l'arrête (« x » devient « X » dans les diagrammes). [Bauer §2.3, Bauer §2.4.3]
- **AEC** : met en tri-state les drivers d'adresse du 6510 ; bas = le VIC tient le bus. Normalement bas en phase 1, haut en phase 2. [Bauer §2.3]
- **Bus vidéo** : 14 bits d'adresse (16 Ko) ; les 2 bits manquants viennent des **bits 0/1 inversés du port A de CIA 2** (choix d'une des 4 banques de 16 Ko). Bus de données 12 bits : 8 bits vers la RAM principale, 4 bits supérieurs vers la Color RAM (1K×4, adressée par les 10 bits bas de l'adresse VIC, donc disponible dans toutes les banques). La **Char ROM** apparaît aux adresses VIC \$1000–\$1fff dans les **banques 0 et 2**. [Bauer §2.3, Bauer §2.4.2]
- Les 47 registres du VIC sont mappés en \$d000 et **répétés tous les 64 octets dans \$d000–\$d3ff** (décodage incomplet). [Bauer §2.4.1]
- Unités d'affichage et unités d'accès mémoire sont **séparées** (tampon de données entre les deux) : on peut les découpler par programmation et afficher des données sans les avoir relues (contenu résiduel du tampon). [Bauer §3.1]

2. Table complète des registres \$d000–\$d02e [Bauer §3.2]

#	Adr.	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	Fonction
0	\$d000	←M0X→								Coordonnée X sprite 0
1	\$d001	←M0Y→								Coordonnée Y sprite 0
2	\$d002	←M1X→								Coordonnée X sprite 1
3	\$d003	←M1Y→								Coordonnée Y sprite 1
4	\$d004	←M2X→								Coordonnée X sprite 2

#	Adr.	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	Fonction
5	\$d005	←M2Y→								Coordonnée Y sprite 2
6	\$d006	←M3X→								Coordonnée X sprite 3
7	\$d007	←M3Y→								Coordonnée Y sprite 3
8	\$d008	←M4X→								Coordonnée X sprite 4
9	\$d009	←M4Y→								Coordonnée Y sprite 4
10	\$d00a	←M5X→								Coordonnée X sprite 5
11	\$d00b	←M5Y→								Coordonnée Y sprite 5
12	\$d00c	←M6X→								Coordonnée X sprite 6
13	\$d00d	←M6Y→								Coordonnée Y sprite 6
14	\$d00e	←M7X→								Coordonnée X sprite 7
15	\$d00f	←M7Y→								Coordonnée Y sprite 7
16	\$d010	M7X8	M6X8	M5X8	M4X8	M3X8	M2X8	M1X8	M0X8	MSB des coordonnées X
17	\$d011	RST8	ECM	BMM	DEN	RSEL	YS2	YS1	YS0	Registre de contrôle 1 (YSCROLL = bits 0–2)
18	\$d012	←RASTER→								Compteur raster
19	\$d013	←LPX→								Crayon optique X
20	\$d014	←LPY→								Crayon optique Y
21	\$d015	M7E	M6E	M5E	M4E	M3E	M2E	M1E	M0E	Sprite activé
22	\$d016	–	–	RES	MCM	CSEL	XS2	XS1	XS0	Registre de contrôle 2 (XSCROLL = bits 0–2)
23	\$d017	M7YE	M6YE	M5YE	M4YE	M3YE	M2YE	M1YE	M0YE	Expansion Y des sprites
24	\$d018	VM13	VM12	VM11	VM10	CB13	CB12	CB11	–	Pointeurs mémoire
25	\$d019	IRQ	–	–	–	ILP	IMMC	IMBC	IRST	Registre d'interruption (latch)
26	\$d01a	–	–	–	–	ELP	EMMC	EMBC	ERST	Interruptions autorisées
27	\$d01b	M7DP	M6DP	M5DP	M4DP	M3DP	M2DP	M1DP	M0DP	Priorité sprite/données
28	\$d01c	M7MC	M6MC	M5MC	M4MC	M3MC	M2MC	M1MC	M0MC	Sprite multicolore
29	\$d01d	M7XE	M6XE	M5XE	M4XE	M3XE	M2XE	M1XE	M0XE	Expansion X des sprites
30	\$d01e	M7M	M6M	M5M	M4M	M3M	M2M	M1M	M0M	Collision sprite–sprite
31	\$d01f	M7D	M6D	M5D	M4D	M3D	M2D	M1D	M0D	Collision sprite–données
32	\$d020	–	–	–	–	←EC→				Couleur de bordure
33	\$d021	–	–	–	–	←B0C→				Couleur de fond 0
34	\$d022	–	–	–	–	←B1C→				Couleur de fond 1
35	\$d023	–	–	–	–	←B2C→				Couleur de fond 2
36	\$d024	–	–	–	–	←B3C→				Couleur de fond 3
37	\$d025	–	–	–	–	←MM0→				Multicolore sprite 0
38	\$d026	–	–	–	–	←MM1→				Multicolore sprite 1
39	\$d027	–	–	–	–	←M0C→				Couleur sprite 0
40	\$d028	–	–	–	–	←M1C→				Couleur sprite 1
41	\$d029	–	–	–	–	←M2C→				Couleur sprite 2
42	\$d02a	–	–	–	–	←M3C→				Couleur sprite 3
43	\$d02b	–	–	–	–	←M4C→				Couleur sprite 4
44	\$d02c	–	–	–	–	←M5C→				Couleur sprite 5
45	\$d02d	–	–	–	–	←M6C→				Couleur sprite 6
46	\$d02e	–	–	–	–	←M7C→				Couleur sprite 7

Notes normatives [Bauer §3.2] :

- Les bits marqués « – » ne sont pas connectés et **se lisent à « 1 »**.
- Registres répétés tous les 64 octets dans \$d000–\$d3ff.
- Les adresses inutilisées **\$d02f–\$d03f** se lisent \$ff ; l'écriture y est ignorée.
- **\$d01e et \$d01f ne peuvent pas être écrits** et sont automatiquement remis à zéro **à la lecture**.
- Le bit **RES** (\$d016 bit 5) n'a aucune fonction connue sur 6567/6569 (sur le 6566, il arrête le VIC).
- **RST8** (\$d011 bit 7) est le bit 8 de \$d012 ; l'ensemble s'appelle « RASTER ». **L'écriture** dans ces bits fixe la ligne de comparaison pour l'interruption raster.

3. Palette [Bauer §3.3]

16 couleurs câblées, codées sur 4 bits :

N°	Couleur	N°	Couleur
0	noir	8	orange
1	blanc	9	brun
2	rouge	10	rouge clair

N°	Couleur	N°	Couleur
3	cyan	11	gris foncé
4	rose (« pink »)	12	gris moyen
5	vert	13	vert clair
6	bleu	14	bleu clair
7	jaune	15	gris clair

Génération analogique (pas de RGB) : mélange sinus/cosinus dérivé de ϕ_{COLOR} , phase+amplitude modulées par couleur. Luminance : les anciennes révisions (6569R1) produisaient **5 niveaux** distincts, les plus récentes **9 niveaux**. Pas d'entrelacement : sortie toujours progressive. [Bauer §2.3, Bauer §3.3]

4. Dimensions d'affichage, RSEL/CSEL, types de VIC [Bauer §3.4]

Convention de repérage : Y = numéro de ligne raster (RASTER, $\$d011/\$d012$) ; X = système de coordonnées **des sprites** ; 8 pixels = 1 cycle d'horloge (une coordonnée X de sprite est donc 8× plus précise qu'un numéro de cycle). Le rendu lu en mémoire est affiché avec un **retard de 12 pixels** [Bauer §3.6.1].

Fenêtre d'affichage (display window) — immobile au centre ; RSEL/CSEL ne déplacent **que** le début/la fin de l'affichage de la bordure ; la position de la fenêtre, sa résolution et la matrice vidéo (toujours 40×25) ne changent pas :

RSEL	Hauteur	Première ligne	Dernière ligne
0	24 lignes texte / 192 pixels	55 ($\$37$)	246 ($\$f6$)
1	25 lignes texte / 200 pixels	51 ($\$33$)	250 ($\$fa$)

CSEL	Largeur	Première coord. X	Dernière coord. X
0	38 caractères / 304 pixels	31 ($\$1f$)	334 ($\$14e$)
1	40 caractères / 320 pixels	24 ($\$18$)	343 ($\$157$)

RSEL=0 : les bordures haute et basse gagnent chacune 4 pixels sur la fenêtre. CSEL=0 : la bordure gauche gagne 7 pixels, la droite 9.

XSCROLL ($\$d016$ bits 0–2) et **YSCROLL** ($\$d011$ bits 0–2) : décalage de la graphique dans la fenêtre de 0 à 7 pixels vers la droite/le bas. Alignement de la graphique avec la fenêtre : **XSCROLL=0 et YSCROLL=3** pour 25 lignes/40 colonnes ; **7 et 7** pour 24 lignes/38 colonnes.

Types de VIC :

Type	Système	Lignes	Lignes visibles	Cycles/ligne	Pixels visibles/ligne
6567R56A	NTSC-M	262	234	64	411
6567R8	NTSC-M	263	235	65	418
6569	PAL-B	312	284	63	403

Type	1re ligne vblank	Dernière ligne vblank	1re coord. X de la ligne	1re coord. X visible	Dernière coord. X visible
6567R56A	13	40	412 ($\$19c$)	488 ($\$1e8$)	388 ($\$184$)
6567R8	13	40	412 ($\$19c$)	489 ($\$1e9$)	396 ($\$18c$)
6569	300	15	404 ($\$194$)	480 ($\$1e0$)	380 ($\$17c$)

Le début de ligne = front descendant d'IRQ raster, qui **ne coïncide pas** avec X=0 mais avec la « 1re coord. X de la ligne ». Les X montent jusqu'à **$\$1ff$ ($\$1f7$ seulement sur 6569)**, puis vient X=0.

5. Bad Lines — définition exacte [Bauer §3.5]

Le VIC a besoin de **40 cycles de bus supplémentaires** pour lire les pointeurs de caractères (les 63–65 premières phases par ligne ne suffisent pas pour lire pointeurs **et** pixels). Il « assomme » donc le processeur pendant **40–43 cycles** dans la première ligne raster de chaque ligne texte. L'accès aux pointeurs a lieu **aussi en mode bitmap** (la matrice vidéo y sert de couleur).

Définition (à prendre littéralement) :

Une condition de Bad Line (Bad Line Condition) existe à n'importe quel cycle d'horloge si, **au front descendant de $\phi 0$ au début du cycle ($\phi 0 = \text{l'horloge système côté VIC — le même signal que } \phi 2 / \text{Phi ; cf. équivalences ch.2})$** , (1) **RASTER $\geq$ \$30 et RASTER $\leq$ \$f7**, et (2) **les trois bits inférieurs de RASTER sont égaux à YSCROLL**, et si (3) **le bit DEN a été à 1 pendant un cycle quelconque de la ligne raster \$30**.

On peut créer/annuler la condition **plusieurs fois dans une même ligne** (\$30–\$f7) en modifiant YSCROLL, et ainsi rendre chaque ligne totalement ou partiellement Bad Line, ou déclencher/supprimer toutes les fonctions qui y sont liées. Si YSCROLL=0, une Bad Line Condition survient en ligne \$30 dès que DEN est mis à 1. Normalement, chaque 8e ligne de la fenêtre (première ligne raster de chaque ligne texte) est une Bad Line ; leur position dépend de YSCROLL.

En clair — trois conditions doivent être vraies en même temps : le compteur de ligne est dans la plage \$30–\$f7, ses 3 bits bas coïncident avec YSCROLL, et DEN a été vu à 1 au moins un cycle pendant la ligne \$30. Comme YSCROLL est sous le contrôle du programme, la coïncidence peut être créée ou détruite à volonté **dans cette plage** — tout le chapitre des effets (FLD, FLI, linecrunch, VSP, §16) tient dans cette phrase.

6. Accès mémoire [Bauer §3.6]

6.1 Coordonnées X [Bauer §3.6.1]

Le VIC n'a pas de registre lisible pour X. La position absolue a été mesurée via **LPX** (IRQ relié à l'entrée LP), les autres X déterminés relativement ; le front descendant de BA en Bad Line mesuré pareil, résultat cohérent. Hypothèse implicite : coordonnées LPX = coordonnées X des sprites (aucune indication du contraire).

6.2 Zones et types d'accès [Bauer §3.6.2]

Zones :

- **Matrice vidéo** : 1000 adresses (40×25, **12 bits** chacune), déplaçable par pas de 1 Ko via VM10–VM13 (\$d018). La Color RAM en fait partie (4 bits supérieurs). Les données lues sont stockées dans le tampon interne « **video matrix/color line** » de 40×12 bits.
- **Générateur de caractères** : 2048 octets, pas de 2 Ko via CB11–CB13 ; **bitmap** : 8000 octets, pas de 8 Ko via CB13 seul.
- **Pointeurs de sprites** : les 8 octets **après la fin de la matrice vidéo** ; chacun sélectionne un bloc de 64 octets parmi 256.
- **Données sprite** : 63 octets par sprite, déplaçables par pas de 64 octets.

Types d'accès :

Type	Cible	Largeur
c	matrice vidéo + Color RAM	12 bits
g	générateur de caractères / bitmap	8 bits
p	pointeurs de sprites	8 bits
s	données de sprites	8 bits
r	rafraîchissement DRAM	5 lectures/ligne
i (idle)	lecture à l'adresse vidéo \$3fff (soit \$3fff/\$7fff/\$bfff/\$ffff selon la banque), résultat jeté	—

6.3 Chronologie d'une ligne raster [Bauer §3.6.3]

(La Figure 2.1, au chapitre 2, dessine cette chronologie cycle par cycle.) La séquence d'accès est **câblée, identique pour toute ligne et tout mode**. La ligne commence au cycle 1 (= front descendant de l'IRQ raster = moment où RASTER est incrémenté). **Exception ligne 0** : IRQ et incrément/reset de RASTER y ont lieu **un cycle plus tard** ; par convention le début de la ligne 0 est défini un cycle avant l'IRQ. 63 cycles (6569), 64 (6567R56A), 65 (6567R8).

Disposition (6569, phases 1 sauf mention ; d'après les diagrammes de la source) :

Cycles (6569)	Accès VIC
1, 3, 5, 7, 9	p-accès sprites 3, 4, 5, 6, 7 (les s-accès d'un sprite occupent les 3 demi-cycles suivant immédiatement son p-accès)
11–15	5 accès r (refresh)
15–54 (phase 2)	40 c-accès (si Bad Line ; BA bas cycles 12–54)
16–55	40 g-accès
56, 57	idle
58, 60, 62	p-accès sprites 0, 1, 2

Sur les diagrammes (raccourcis) des modèles NTSC : p3–p7 aux mêmes cycles 1–9 et refresh 11–15 ; p0/p1/p2 = cycles **59/61/63** (6567R56A) et **60/62/64** (6567R8). Pour les sprites 0–2, les s-accès tombent donc toujours **dans la ligne précédant** leur affichage. Pour les s-accès, BA passe bas 3 cycles avant l'accès, comme pour les Bad Lines. Cycle 1 phase 1 = X \$194 (6569) ; 8 pixels par cycle, 4 par phase. La bordure est générée env. **8 pixels plus tard** que la projection « Graph. » des diagrammes.

Méthode pratique de calage donnée par la source : modifier un octet de mémoire graphique avec le 6510 et observer à partir de quel caractère le changement est visible — l'écriture a eu lieu dans la phase d'horloge immédiatement précédente.

7. État display / état idle [Bauer §3.7.1]

- **État display** : c- et g-accès ont lieu, adresses et interprétation selon le mode.
- **État idle** : seuls des g-accès, toujours à **\$3fff** (**\$39ff** si ECM=1) ; la graphique est affichée comme en display, mais les données de matrice vidéo sont traitées comme des bits « 0 ».
- **Transition idle→display** : **dès qu'il y a une Bad Line Condition.**
- **Transition display→idle** : **au cycle 58 d'une ligne si RC=7 et pas de Bad Line Condition.**

Si \$d011 n'est pas modifié en cours de trame : display dans la fenêtre, idle dehors. (Avec YSCROLL≠3 en fenêtre 25 lignes et un octet ≠0 en \$3fff, on voit les bandes noires du séquenceur en idle en haut/bas de la fenêtre.)

8. VC, RC, VMLI — les règles [Bauer §3.7.2]

Registres :

- **VC** (video counter) : compteur 10 bits, chargeable depuis VCBASE.
- **VCBASE** : registre 10 bits avec entrée de reset, chargeable depuis VC.
- **RC** (row counter) : compteur 3 bits avec reset.
- **VMLI** (video matrix line index) : en réalité un **registre à décalage de 40 bits** qui promène un unique bit « 1 » adressant une cellule de la video matrix/color line ; en display un nouveau « 1 » est injecté à l'entrée, en idle le registre s'est vidé (raison probable du noir de l'idle : plus d'information couleur). Des manipulations (Bauer §3.14.6) peuvent y mettre plusieurs bits → comportement très instable, dépendant de la révision et même de la température ; sans usage pratique connu, la source le traite ensuite comme un simple **compteur 6 bits avec reset**.

Règles (quasi-verbatim) :

1. Une fois quelque part **hors de la plage de lignes raster \$30–\$f7** (hors plage des Bad Lines), **VCBASE est remis à zéro**. Vraisemblablement en ligne raster 0 ; l'instant exact ne peut être déterminé et est sans importance.

2. Dans la **première phase du cycle 14** de chaque ligne, **VC est chargé depuis VCBASE** (VCBASE→VC) et **VMLI est mis à zéro**. S'il y a une **Bad Line Condition** dans cette phase, **RC est aussi remis à zéro**.
3. S'il y a une **Bad Line Condition** dans les **cycles 12–54**, **BA passe bas** et les c-accès démarrent. Une fois démarrés, un c-accès a lieu dans la **seconde phase de chaque cycle de la plage 15–54**. La donnée lue est rangée dans la video matrix/color line à la position **VMLI**, et relue en interne à la position **VMLI** à chaque g-accès en état display.
4. **VC et VMLI sont incrémentés après chaque g-accès en état display**.
5. Dans la **première phase du cycle 58**, le VIC vérifie si **RC=7**. Si oui, la logique vidéo passe en **idle** et **VCBASE est chargé depuis VC** (VC→VCBASE). Si la logique vidéo est en display (toujours le cas s'il y a une **Bad Line Condition**), **RC est incrémenté**.

Ces règles font normalement compter à VC les 1000 adresses de la matrice par trame et à RC les 8 lignes de pixels de chaque ligne texte ; via YSCROLL, le processeur garde un contrôle partiel.

En clair — VC est le doigt qui suit la lecture dans la matrice écran, VCBASE le marque-page auquel ce doigt revient à chaque ligne raster, RC le compteur des 8 lignes de pixels d'une rangée de caractères. Tant qu'aucune **Bad Line** ne survient, le VIC relit la même rangée (VC repart de VCBASE, règle 2) ; c'est la fin de rangée qui « tourne la page » (règle 5 : VCBASE avance). Retenez : c'est la **Bad Line** qui **amorce** le passage d'une rangée à la suivante — sans elle, VC retombe sur son marque-page à chaque ligne — d'où le pouvoir des effets qui la manipulent (§16).

9. Les 8 modes graphiques [Bauer §3.7.3]

Sélection par **ECM/BMM/MCM** (\$d011/\$d016). Trois des huit combinaisons sont « invalides » et sortent du noir. Le cœur du séquenceur graphique est un **registre à décalage 8 bits**, décalé d'un bit par pixel, rechargé après chaque g-accès ; **XSCROLL retarde le rechargement de 0–7 pixels**. Si **XSCROLL est augmenté** en milieu de ligne, le séquenceur produit des pixels de couleur de fond supplémentaires ; s'il est **diminué**, l'affichage des 8 pixels suivants démarre en avance et tronque le caractère en cours. Le séquenceur sort la graphique dans la colonne d'affichage tant que la bascule de bordure verticale est à 0 ; sinon (et hors colonne), c'est la **dernière couleur de fond courante** qui est affichée. Si **ECM=1**, le générateur d'adresses force les **bits d'adresse 9 et 10 à 0** sans autre changement (idle → \$39ff).

c-accès (identique pour tous les modes) : adresse = VM13 VM12 VM11 VM10 | VC9...VC0 (14 bits). En idle : pas de c-accès, données = 12 bits à 0.

Mode (ECM/BMM/MCM)	Adresse du g-accès	Interprétation
Texte standard (0/0/0) [Bauer §3.7.3.1]	CB13 CB12 CB11 D7...D0 RC2 RC1 RC0	8 px 1 bit : « 0 » = fond 0 (\$d021) ; « 1 » = couleur bits 8–11 du c-data. 256 caractères.
Texte multicolore (0/0/1) [Bauer §3.7.3.2]	idem texte std	Bit 11 du c-data = drapeau MC. MC=0 : comme texte std, couleurs 0–7 seulement (bits 8–10). MC=1 : 4 px 2 bits — « 00 » fond 0 (\$d021), « 01 » fond 1 (\$d022), « 10 » fond 2 (\$d023), « 11 » couleur bits 8–10. « 01 » compte comme fond pour priorité/collisions.
Bitmap standard (0/1/0) [Bauer §3.7.3.3]	CB13 VC9...VC0 RC2 RC1 RC0	8 px 1 bit : « 0 » = couleur bits 0–3 du c-data, « 1 » = bits 4–7. 320×200, bloc 8×8 formé de 8 octets successifs.
Bitmap multicolore (0/1/1) [Bauer §3.7.3.4]	idem bitmap std	4 px 2 bits : « 00 » fond 0 (\$d021) ; « 01 » bits 4–7 ; « 10 » bits 0–3 ; « 11 » bits 8–11 du c-data. 160×200. « 01 » = fond.
Texte ECM (1/0/0) [Bauer §3.7.3.5]	CB13 CB12 CB11 0 0 D5...D0 RC2 RC1 RC0	Comme texte std mais fond choisi par bits 6/7 du c-data : 00→\$d021, 01→\$d022, 10→\$d023, 11→\$d024 ; « 1 » = bits 8–11. Jeu réduit à 64 caractères .
Texte invalide (1/0/1) [Bauer §3.7.3.6]	CB13 CB12 CB11 0 0 D5...D0 RC	Écran noir . Le séquenceur génère quand même des données valides (structure ~texte MC, 64 caractères) lisibles via collisions de sprites (avant/arrière-plan seulement, pas la couleur).

Mode (ECM/BMM/MCM)	Adresse du g-accès	Interprétation
Bitmap invalide 1 (1/1/0) [Bauer §3.7.3.7]	CB13 VC9 VC8 0 0 VC5...VC0 RC	Noir. Structure ~bitmap std, bits 9/10 d'adresse à 0 → quatre « sections » répétées quatre fois. Lisible par collisions.
Bitmap invalide 2 (1/1/1) [Bauer §3.7.3.8]	idem bitmap invalide 1	Noir. Structure ~bitmap MC ; « 01 » = fond. Lisible par collisions.

En clair — huit combinaisons, mais la hiérarchie est simple : trois modes texte et deux modes bitmap utiles ; les trois combinaisons « invalides » sortent toutes du **noir** (leurs données restent lisibles par collisions de sprites) ; et côté adresses, ECM ne fait qu'une chose : forcer les bits 9 et 10 à zéro.

État idle [Bauer §3.7.3.9] : g-accès à \$3fff (\$39ff si ECM), l'octet est répété, matrice = « 0 ». Affichage : modes texte std/MC/ECM → « 0 » = fond 0 (\$d021), « 1 » = **noir** ; bitmap std/texte invalide/bitmap invalide 1 → noir (0=fond, 1=avant-plan) ; bitmap MC → « 00 » = fond 0 (\$d021), « 01 » noir (fond), « 10 »/« 11 » noir (avant-plan) ; bitmap invalide 2 → tout noir (« 00 »/« 01 » fond, « 10 »/« 11 » avant-plan).

10. Sprites [Bauer §3.8]

24×21 pixels, 8 sprites indépendants (« MOBs »). Y sur 8 bits, X sur 9 bits (MSB dans \$d010). Activation MxE (\$d015), expansion ×2 en X (\$d01d) et/ou Y (\$d017) — résolution inchangée —, multicolore MxMC (\$d01c) → 12×21 pixels doublés, priorité MxDP (\$d01b), couleurs \$d027–\$d02e.

10.1 Accès mémoire et affichage [Bauer §3.8.1]

- 63 octets linéaires, 3 octets par ligne de sprite. **p-accès à chaque ligne raster** depuis les 8 derniers octets de la matrice vidéo → 8 bits hauts de l'adresse des s-accès ; 6 bits bas = compteur **MC** (on peut changer le pointeur en cours d'affichage).
- s-accès : dans les **3 demi-cycles suivant directement le p-accès du sprite**, avec BA/AEC comme pour les Bad Lines (BA bas 3 cycles avant).
- **Expansion X** : le séquenceur sort les pixels à demi-fréquence. **Expansion Y** : le générateur d'adresses relit **les mêmes adresses deux lignes de suite**.
- Par sprite : registre à décalage **24 bits**, **MC** (compteur 6 bits chargeable depuis MCBASE), **MCBASE** (registre 6 bits chargeable depuis MC ou remis à zéro), et une bascule « **advance line** » (avance de ligne) pour l'expansion Y.

Règles d'affichage (quasi-verbatim ; numéros de cycles valables pour le 6569 uniquement). (La révision 2024 numérote 1–7 plus 7a ; il n'y a pas de règle « 8 » distincte dans cette version. ▲ Lecteurs de la version 1996 : l'ancienne formulation « MCBASE += 2 au cycle 15, += 1 au cycle 16 » a été **remplacée** dans la révision 2024 par la sémantique de copie MC→MCBASE présentée ici — correction issue des travaux de Kahlin, Nuotio, Lankila et Matthies, crédités par Bauer.)

1. La bascule d'avance de ligne est **maintenue à 1 tant que le bit MxYE (\$d017) est à 0**.
2. Dans les **premières phases des cycles 55 et 56**, le VIC vérifie pour chaque sprite si **MxE (\$d015) est à 1** et si la **coordonnée Y du sprite (registres impairs \$d001–\$d00f) égale les 8 bits bas de RASTER**. Si oui et que le **DMA du sprite est encore éteint** : le DMA est allumé, **MCBASE est mis à zéro**, et la bascule d'avance de ligne est mise à 1.
3. Si le bit **MxYE est à 1 dans la seconde phase du cycle 56** et que le DMA du sprite est allumé, la bascule d'avance de ligne est **inversée**.
4. Dans la **première phase du cycle 58**, le **MC de chaque sprite est chargé depuis son MCBASE** (MCBASE→MC) et le VIC vérifie si le DMA est allumé **et** si la coordonnée Y égale les 8 bits bas de RASTER. Si oui, **l'affichage du sprite est allumé**. Sinon, si le DMA est éteint, l'affichage est éteint aussi.
5. Si le DMA d'un sprite est allumé, **trois s-accès** sont effectués dans les cycles attribués au sprite. **Les p-accès ont toujours lieu, même sprite éteint**. 1er accès → 8 bits hauts du registre à décalage, 2e → 8 bits du milieu, 3e → 8 bits bas. **MC est incrémenté après chaque s-accès**.
6. Si l'affichage du sprite est allumé, le registre à décalage est décalé **à gauche d'un bit par pixel dès que la coordonnée X courante du faisceau égale la coordonnée X du sprite** (registres pairs \$d000–

\$d00e), et les bits sortants sont affichés. Si **MxXE** (\$d01d) est à 1, le décalage n'a lieu qu'**un pixel sur deux** (sprite deux fois plus large). En multicolore, deux bits adjacents forment un pixel.

7. Dans la **première phase du cycle 16**, le VIC vérifie pour chaque sprite si sa bascule d'avance de ligne est à 1. Si oui, **MCBASE est chargé depuis MC** (MC→MCBASE), avançant l'affichage à la ligne suivante. Ensuite, le VIC vérifie si **MCBASE = 63** et, le cas échéant, **éteint le DMA** du sprite. 7a. Cas particulier : si le CPU a **remis MxYE à 0 au cycle 15** et que la bascule d'avance de ligne du sprite **n'était pas** à 1, alors la bascule est mise à 1 et au cycle suivant (16) le VIC charge MCBASE non pas depuis MC mais selon : $MCBASE = (101010_2 \& (MCBASE \& MC)) \mid (010101_2 \& (MCBASE \mid MC))$ (bits impairs = ET de MCBASE et MC, bits pairs = OU ; formule reproduite de la source — équivalente à la forme 0x15/0x2a d'Åkesson au ch.2 (§3.2) : %010101 = \$15, %101010 = \$2a).

Conséquences normatives :

- Le test de la règle 2 étant fait en fin de ligne, la valeur écrite dans le registre Y doit être **la position Y désirée — 1** ; l'affichage ne commence qu'à la ligne suivante, après lecture des premières données (sauf sprite placé à droite de la coordonnée X \$164 — cycle 58, règle 4).
- **Multiplexage vertical possible** (re-déclenchement en changeant Y après la fin d'affichage — fonctionnalité officielle) ; **impossible horizontalement** : après 24 pixels le registre est vide, max 8 sprites par ligne raster.
- Sur 6569, les X \$1f8–\$1ff **rendent le sprite invisible** (jamais atteints par le compteur X) ; un pixel à gauche de X=0 → X=\$1f7.

Adresses : p-accès = VM13...VM10 | 1 1 1 1 1 1 | n° de sprite (3 bits) → donnée MP7...MP0 ; s-accès = MP7...MP0 | MC5...MC0. Données : MxMC=0 → 8 px 1 bit (« 0 » transparent, « 1 » couleur \$d027–\$d02e) ; MxMC=1 → 4 px 2 bits (« 00 » transparent, « 01 » multicolore 0 \$d025, « 10 » couleur sprite, « 11 » multicolore 1 \$d026).

En clair — le DMA d'un sprite s'allume quand le sprite est **activé** (MxE) et que sa coordonnée Y coïncide, s'il était éteint (règle 2, testée en fin de ligne — d'où le « Y désiré — 1 », sauf sprite placé à droite de \$164) ; il lit 3 octets par ligne (règle 5), avance par le couple MC/MCBASE (règles 4 et 7) et s'éteint quand MCBASE atteint 63. L'expansion Y ne fait que retarder cette avance une ligne sur deux (règles 1–3). Toutes les acrobaties — stretching, crunch (§16.7) — consistent à mentir à la bascule d'avance **au bon cycle**.

10.2 Priorités et collisions [Bauer §3.8.2]

- Hiérarchie fixe entre sprites : **sprite 0 = priorité la plus haute, sprite 7 la plus basse**.
- Partition avant/arrière-plan de la graphique décidée par **MCM (\$d016) seul**, indépendamment de l'état du séquenceur et des bits BMM/ECM — valable aussi pour la graphique d'état idle :

	MCM=0	MCM=1
Bits/pixel	1	2
Pixels/octet	8	4
Fond	« 0 »	« 00 », « 01 »
Avant-plan	« 1 »	« 10 », « 11 »

- **MxDP=0** : priorité croissante = fond < avant-plan < sprite x < bordure. **MxDP=1** : fond < sprite x < avant-plan < bordure. (La table de [2], la spec Commodore, est fausse.) La **bordure a toujours la priorité maximale**.
- Avec MxDP=1, les pixels d'avant-plan recouvrant un pixel de sprite non transparent « héritent » de la priorité du sprite (comme « incrustés » dans l'image du sprite) : ils restent visibles même par-dessus des sprites de numéro supérieur configurés MxDP=0. Dans les modes invalides, l'avant-plan (noir) redevient ainsi visible par-dessus les sprites.
- **Collision sprite–sprite** : détectée dès que **deux séquenceurs de sprites ou plus** sortent un pixel non transparent en même temps (**y compris hors zone visible**) → bits MxM dans \$d01e + interruption si autorisée. Bits conservés jusqu'à **lecture du registre, qui les efface**.
- **Collision sprite–graphique** : un séquenceur de sprite sort un pixel non transparent pendant que le séquenceur graphique sort un **pixel d'avant-plan** → bits MxD dans \$d01f (même effacement à la lecture).
- Si la **bascule de bordure verticale** est à 1, la sortie du séquenceur graphique est coupée → **aucune collision sprite–graphique**.

11. Unité de bordure [Bauer §3.9]

Deux bascules : **principale** (affiche \$d020 si à 1, priorité d'affichage maximale, recouvre graphique et sprites) et **verticale** (si à 1 : la principale ne peut pas être remise à 0, et la sortie du séquenceur graphique est coupée — il affiche la couleur de fond). 2×2 comparateurs par bascule, valeurs câblées selon CSEL/RSEL ; la comparaison n'est vraie que si la valeur est atteinte exactement (pas de test d'intervalle).

Comparaison X	CSEL=0	CSEL=1
Gauche	31 (\$1f)	24 (\$18)
Droite	335 (\$14f)	344 (\$158)

Comparaison Y	RSEL=0	RSEL=1
Haut	55 (\$37)	51 (\$33)
Bas	247 (\$f7)	251 (\$fb)

Règles :

1. Quand X atteint la valeur de comparaison **droite**, la bascule principale est **mise à 1**.
2. Quand Y atteint la valeur **basse** au **cycle 63**, la bascule verticale est **mise à 1**.
3. Quand Y atteint la valeur **haute** au **cycle 63** et que **DEN** (\$d011) est à 1, la bascule verticale est **remise à 0**.
4. Quand X atteint la valeur **gauche** et Y la valeur **basse**, la bascule verticale est **mise à 1**.
5. Quand X atteint la valeur **gauche**, Y la valeur **haute** et que **DEN** est à 1, la bascule verticale est **remise à 0**.
6. Quand X atteint la valeur **gauche** et que la bascule verticale **n'est pas** à 1, la bascule principale est **remise à 0**.

Y est donc testé une ou deux fois par ligne : au cycle 63 et quand X atteint la comparaison gauche.

En clair — la bordure n'est pas une zone, c'est **deux interrupteurs** commandés par des comparateurs de position exacte (« la valeur est atteinte », jamais « on est au-delà »). Si plus aucune comparaison pertinente n'est vraie — parce qu'on a déplacé la valeur comparée (RSEL/CSEL) après son passage — l'interrupteur ne bascule plus : c'est tout le secret de l'ouverture des bordures (§16.1).

12. DEN (Display Enable) [Bauer §3.10]

Bit 4 de \$d011, normalement à 1. Deux effets :

- Une **Bad Line Condition** ne peut survenir que si DEN a été à 1 au moins un cycle quelque part dans la ligne raster \$30.
- DEN à 0 → l'entrée de reset de la bascule de bordure verticale est désactivée : la bordure haut/bas ne s'éteint jamais.

DEN=0 empêche donc normalement toute Bad Line (donc tout c/g-accès) et l'écran entier affiche la couleur de bordure.

13. Crayon optique [Bauer §3.11]

Sur front descendant de LP : **L PX (\$d013) = 8 bits hauts (sur 9) de X** (résolution horizontale 2 pixels), **L PY (\$d014) = 8 bits bas (sur 9) de Y**. Un seul front reconnu par trame ; les suivants sont ignorés jusqu'au début de la trame suivante. LP partage une ligne avec la matrice clavier → déclenchable par logiciel via **bit**

4 du port B de CIA A (\$dc01/\$dc03), ce qui permet de lire la position X du faisceau (synchronisation cycle-exacte possible). Référence = fin du cycle du déclenchement ; ex. : déclenchement au cycle 20 → LPX=\$1e (coordonnée sprite \$03c). Peut aussi déclencher une interruption (une fois par trame).

14. Interruptions [Bauer §3.12]

IRQ du VIC → IRQ du 6510 (masquable par le flag I). Quatre sources ; chaque source a un bit dans le **latch \$d019** et un bit d'autorisation dans **\$d01a**. Événement → bit du latch mis à 1. **Le VIC n'efface jamais le latch lui-même : le processeur doit écrire un « 1 » sur le bit pour l'effacer**, avant de rendre la main (l'entrée IRQ du 6510 est sensible à l'état → sinon re-déclenchement immédiat). Si (latch ∧ enable) ≠ 0, le VIC tient IRQ bas. **Bit 7 de \$d019 = état inversé de la sortie IRQ.**

Bit	Nom	Condition
0	RST	Atteinte de la ligne raster de comparaison (écrite via \$d012 + bit 7 de \$d011, mémorisée en interne). Test au cycle 1 de chaque ligne (cycle de rafraîchissement).
1	MBC	Collision d'au moins un sprite avec la graphique (pixel de sprite non transparent simultané à un pixel d'avant-plan).
2	MMC	Collision de deux sprites ou plus (deux séquenceurs sortent un pixel non transparent en même temps).
3	LP	Front descendant sur l'entrée LP.

MBC/MMC : **seule la première collision déclenche** (si \$d01f resp. \$d01e valait zéro avant — MBC surveille \$d01f [sprite-graphique], MMC \$d01e [sprite-sprite]) ; pour ré-armer, lire (donc effacer) le registre de collision.

15. Rafraîchissement DRAM [Bauer §3.13]

5 lectures de refresh par ligne raster. Compteur REF 8 bits : mis à **\$ff en ligne raster 0**, décrémenté après chaque accès, boucle à \$ff après passage sous zéro. Adresse = 1 1 1 1 1 1 | REF7...REF0 (\$fxx). Ligne 0 : \$3fff, \$3ffe, \$3ffd, \$3ffc, \$3ffb ; ligne 1 : \$3ffa...\$3ff6 ; etc.

16. Effets et applications [Bauer §3.14]

16.1 Hyperscreen (ouverture de bordure) [Bauer §3.14.1]

La bordure est **commutée à des coordonnées précises**, pas affichée sur un intervalle : si la comparaison n'est jamais vraie, la bordure ne s'allume jamais. Bordure haut/bas :

1. Dans la partie haute de l'écran, passer **RSEL=1**.
2. Attendre RASTER dans **248–250** (la bascule verticale est encore à 0 : la comparaison RSEL=1 est à la ligne 251).
3. Mettre **RSEL=0** : le comparateur bascule sur 247, déjà passée → le VIC « oublie » d'allumer la bordure verticale.
4. Après la ligne 251, remettre RSEL=1 et reprendre en 2.

Passer RSEL 0→1 dans les lignes **52–54** → la bordure ne s'éteint jamais et couvre tout l'écran (comme DEN=0, mais les Bad Lines continuent). Bordure gauche/droite : passage **CSEL 1→0 exactement au cycle 56** pour ouvrir ; **CSEL 0→1 au cycle 17** empêche l'extinction. Pour ouvrir gauche/droite dans la zone de bordure haut/bas, il faut soit commencer avant que la bascule verticale soit à 1, soit ouvrir aussi le haut/bas (règle 6 : la principale ne se remet à 0 que si la verticale est à 0). Dans la zone ouverte : sprites, et graphique d'état idle (aucune Bad Line hors Y \$30–\$f7) ; première méthode → seule la couleur de fond y est visible, seconde → la graphique idle.

16.2 FLD (Flexible Line Distance) [Bauer §3.14.2]

La Bad Line est le « signal de départ » de chaque ligne texte. En modifiant YSCROLL on **supprime/retarde** la condition à volonté → on choisit les lignes raster où démarrent les lignes texte (ex. trois Bad Lines en

\$50, \$78, \$a0 → trois lignes texte, séquenceur idle entre elles). Retarder seulement la première Bad Line = **scrolling vertical vers le bas** de tout l'écran sans déplacer un octet.

16.3 FLI (Flexible Line Interpretation) [Bauer §3.14.3]

Créer artificiellement une **Bad Line supplémentaire avant la fin de la ligne texte en cours** : en rendant chaque ligne raster Bad Line (via YSCROLL), le VIC relit la matrice vidéo à **chaque ligne** → nouvelle info couleur par ligne ; en bitmap multicolore, chacun des 4×8 pixels d'un bloc peut avoir sa couleur.

- **Problème 1** : une nouvelle Bad Line avant la fin de la ligne texte → **VCBASE n'est pas incrémenté** (Bauer §3.7.2) : le VIC relit **les mêmes adresses de matrice** qu'à la ligne précédente. Trop lent à changer par CPU → on commute la base de la matrice avec **VM10–VM13 (\$d018)**. En pratique : **8 matrices vidéo** en mémoire, utilisées cycliquement (en base 0 : ligne raster $n \rightarrow$ rangée $\lfloor n/8 \rfloor$ de la matrice ($n \bmod 8$)). **La Color RAM, elle, ne peut pas être commutée** → choix des couleurs pas entièrement libre.
- **Problème 2** (le « **FLI-bug** »), **cause exacte** : l'accès à \$d011 créant la Bad Line **ne doit pas survenir avant le cycle 14** (sinon RC serait remis à 0 à chaque ligne, cassant l'affichage bitmap). Mais alors **les trois premiers c-accès de chaque ligne lisent des données invalides** : le premier c-accès au cycle 15 exigerait BA bas dès le cycle 12 pour qu'AEC puisse rester bas au cycle 15 (**AEC ne reste bas que trois cycles après le front descendant de BA** — **c'est câblé, sans contournement**). La Bad Line étant créée au cycle 14, BA est bas au cycle 15 mais **AEC encore haut** → les portes de bus internes **D0–D7 du VIC sont fermées et il lit \$ff** au lieu des données de matrice (les portes D8–D11 sont ouvertes, détail en Bauer §3.14.6). Ces données parasites forment des **bandes de 24 pixels de large au bord gauche** de l'écran.
- Variantes : **AFLI** (bitmap standard, dégradés simulés par pixels adjacents de couleurs proches), **IFLI** (alternance de deux trames, entrelacement simulé).

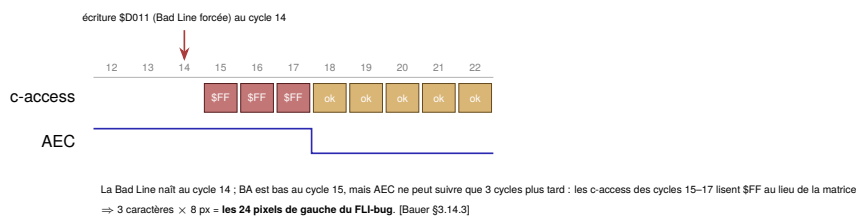


Figure 1.2 — Le FLI-bug. C'est une contrainte de câblage, pas de programmation : aucune donnée en RAM ne peut corriger ce que le VIC n'a pas relu (parades : annexe B, §B.4).

16.4 Linecrunch [Bauer §3.14.4]

Avorter une Bad Line commencée en annulant la condition avant le cycle 14. Conséquences :

- Le séquenceur est en display → la graphique s'affiche.
- **RC n'est pas remis à 0** (si on avorte la toute première ligne de la trame, RC vaut encore 7, hérité de la dernière ligne de la trame précédente).
- Au cycle 58, RC vaut toujours 7 → passage en idle et **VCBASE chargé depuis VC** ; or VC a été incrémenté après chaque g-accès de la ligne → **VCBASE a effectivement augmenté de 40**. RC ne déborde pas, il reste à 7.

La ligne texte est réduite à sa dernière ligne raster (« crunch »). Répété à chaque ligne : RC reste à 7, **pas de c-accès**, VCBASE +40 par ligne → dépasse la limite des 1000 octets et le VIC affiche les **24 derniers octets normalement invisibles de la matrice (dont les pointeurs de sprites)** ; **VCBASE boucle à 0 en atteignant 1024**. Usage : **scrolling vers le haut** de grandes distances sans déplacer la mémoire ; les lignes « crunchées » s'empilent en haut de l'écran (on peut les masquer avec un mode invalide).

16.5 Lignes de texte doublées [Bauer §3.14.5]

Normalement une ligne texte finit quand RC=7 (idle au cycle 58). Si on **crée une Bad Line Condition entre les cycles 54 et 57 de la dernière ligne raster**, le séquenceur reste en display et **RC est incrémenté encore une fois (débordement à 0)** : le VIC réaffiche la ligne texte précédente à la ligne suivante — aucune nouvelle donnée de matrice n'ayant été lue, elle est simplement **affichée deux fois**.

16.6 DMA delay / VSP [Bauer §3.14.6]

Créer une **Bad Line Condition** dans les cycles 15–53 d'une ligne où le séquenceur est en idle (p. ex. rendre YSCROLL égal aux 3 bits bas de RASTER via \$d011). Le VIC met **BA bas dès le cycle suivant**, passe en display et lit la matrice. Mais **AEC suit encore 0 pendant trois cycles** (câblé). Pendant ces trois premiers cycles : les lignes D0–D7 du VIC sont en tri-state → le VIC lit **\$ff comme pointeurs de caractères** ; les lignes hautes (la source écrit ici « D8–D13 », mais « D8–D11 » en Bauer §3.14.3 — incohérence dans la source ; le bus vidéo étant **12 bits** [Bauer §2.3], D8–D11 est nécessairement la bonne lecture) n'ont pas de drivers tri-state et restent en entrée, mais la Color RAM n'est pas sélectionnée (AEC haut → le 6510 est maître du bus) ; un **commutateur analogique 4 bits, U16**, relie alors les bits de données **D0–D3 du processeur** aux bits de données de la Color RAM. Résultat : pour la couleur, le VIC lit **les 4 bits bas de l'opcode suivant l'accès à \$d011**. Ensuite seulement, lectures normales. Les c/g-accès continuent jusqu'au cycle 54 ; ayant commencé en cours de ligne, **moins de 40 accès ont eu lieu → VC n'est plus multiple de 40** en fin de ligne, et ce **désalignement persiste sur toutes les lignes suivantes** (fonctionnement de VC, Bauer §3.7.2) : **tout l'écran apparaît décalé à droite d'autant de caractères que l'accès \$d011 a eu lieu de cycles après le cycle 14**. Marche aussi en bitmap (VC sert aussi aux accès bitmap). Combiné à FLD + Linecrunch : scrolling de tout l'écran dans toutes les directions à coût CPU minime. Variante avec DEN : YSCROLL=0 (la ligne \$30 devient Bad Line dès que DEN passe à 1) et basculer **DEN 0→1 au milieu de la ligne \$30**.

En clair — on réveille le VIC **en cours de ligne** : il commence sa lecture au milieu, son compteur VC ne fait pas ses 40 pas, et tout l'écran reste décalé d'autant — un scrolling horizontal quasi gratuit, choisi au cycle près. Mal subi, c'est **vraisemblablement** la même classe de mécanisme que le « peigne » de l'annexe B (§B.6) : une écriture de \$d011 qui glisse parce qu'un vol DMA a retardé la boucle (hypothèse forte).

16.7 Sprite stretching / sprite crunch [Bauer §3.14.7]

Stretching (répéter une ligne de sprite 3 fois ou plus, facteur d'agrandissement Y arbitraire) — mécanisme complet (cycles 6569) : cycle 55 d'une ligne où le sprite 0 (M0YE=1) matche en Y → DMA on, MCBASE=0, bascule d'avance mise à 1, puis **inversée au cycle 56** (M0YE=1). BA bas pour les s-accès des secondes phases des cycles 58 et 59 ; au cycle 58 phase 1, MC←MCBASE (=0), p-accès puis 3 s-accès, MC=3. À la ligne suivante, **au cycle 16 la bascule d'avance n'est pas à 1 → MCBASE reste à 0**. On efface alors M0YE (ce qui met la bascule à 1) puis on le remet aussitôt à 1 : M0YE étant à 1, la bascule est **réinvertie au cycle 56, donc remise à 0** — exactement l'état du cycle 56 de la ligne précédente. Le VIC « croit » être encore dans la première ligne raster d'un sprite étendu et, MC étant (rechargé) à zéro, relit la première ligne de sprite **deux fois de plus — trois fois au total**. **Sprite crunch** : même procédé, mais effacer M0YE **dans la seconde phase du cycle 15** (et non après le cycle 16) → **MCBASE est mis à 1** (cas 7a). La ligne suivante est lue avec **MC=1..3**, un octet trop haut ; le désalignement persiste sur tout l'affichage. La condition d'arrêt du DMA au cycle 16 (**la source écrit ici « MC=63 » ; la règle 7 parle de MCBASE=63**) n'est pas remplie → le sprite est **affiché deux fois de suite** ; le DMA ne s'éteint qu'à la fin du second affichage, quand la valeur atteint 63.

En clair — le crunch surprend le circuit d'avance du sprite en flagrant délit d'indécision (le cycle 15) : le compteur retombe sur une valeur bâtarde — bits pairs en OU, bits impairs en ET (règle 7a) — et le sprite « saute » dans ses propres données selon un graphe de transitions fixe. Le chapitre 2 (§3) cartographie les boucles utilisables de ce graphe.

17. « Ghost bytes » : \$de00, Color RAM, adresses 0/1 [Bauer §4]

- \$de00–\$dfff (zones I/O 1/2 ouvertes) et les nibbles hauts de \$d800–\$dbff se lisent « aléatoires », mais **sur certains C64** (pas tous, pas toujours fiable) la valeur lue = **le dernier octet lu par le VIC en première phase du cycle**. Applications : mesurer le timing du VIC entièrement en logiciel ; faire exécuter au 6510 du code dans la zone \$de00 ou dans la Color RAM si le VIC lit des opcodes valides.
- **Écriture en RAM 0/1** : lors d'une écriture CPU vers 0/1 (port I/O interne), les drivers de données du 6510 restent en tri-state mais **R/W est bas** → c'est **l'octet lu par le VIC dans la première phase** qui est écrit en RAM. Pour écrire une valeur choisie : écrire n'importe quoi en 0/1 en s'assurant que le VIC vient de lire la valeur voulue au cycle précédent.

- **Lecture de 0/1** : via la zone \$de00, ou par **collisions de sprites** (bitmap basée à l'adresse 0, sprite d'un seul pixel promené sur les bits des deux premiers octets ; collision/pas collision = bit 1/0).
-

Chapitre 2 — Timing fin & vols de cycles (Mäkelä 1994, Ojala 1992, Åkesson 2016)

Où l'on chronomètre ce que le chapitre 1 a décrit. Trois textes se répondent : les chronogrammes mesurés de Mäkelä (l'autorité), l'article fondateur d'Ojala sur les cycles volés, et l'exploitation extrême du DMA sprite par Åkesson. Le fil rouge : sur ce bus partagé, dès que le VIC doit dépasser sa phase réservée (pointeurs de caractères des Bad Lines, données de sprites), il se paie en cycles CPU. La figure ci-dessous situe tous les acteurs d'une ligne raster — elle sert de carte au chapitre entier.

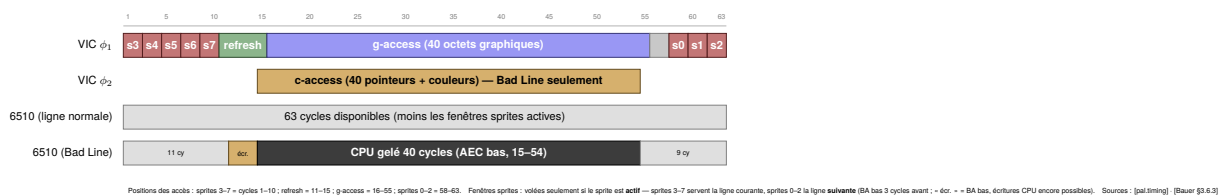


Figure 2.1 — Anatomie d'une ligne raster PAL (63 cycles) : fenêtres sprites, refresh, accès graphiques, et ce qu'il en coûte au 6510 selon que la ligne est normale ou Bad Line.

Ce chapitre rassemble trois sources primaires sur le vol de cycles du VIC-II : l'article fondateur de Pasi Ojala (1992), les chronogrammes mesurés de Marko Mäkelä (1994) — qui corrigent explicitement Ojala, qu'il qualifie d'« inexact » mais de bonne introduction — et l'exploitation extrême du DMA sprite par Linus Åkesson (2016). Statut des affirmations : Mäkelä = **mesuré** (sur vrai matériel), Ojala = **documenté** (avec approximations connues), Åkesson = **mesuré** (démon fonctionnelle) sauf mention contraire.

1. [pal.timing] Chronogrammes du 6569 — Marko Mäkelä, 3 juin 1994

Titre original : *The memory accesses of the MOS 6569 VIC-II and MOS 8566 VIC-IIe Video Interface Controller.*

1.1 Méthode et matériel

Les résultats reposent sur : des mesures de taille d'écran chez Peter Andersson (Suède), les diagrammes « inexacts » d'Ojala, et surtout le programme d'Andreas Boose qui détermine **quelles adresses le VIC lit**, en lisant l'espace d'adressage non connecté **\$DE00-\$DFFF** : le bus de données y reflète la donnée lue par le VIC au **cycle Phi-1 précédent** (découvert par Mäkelä chez Boose, fin mars 1994). Attention : ce truc n'est **pas fiable sur tous les C64/C128** (certains bits restent collés à 0 ou 1 ; seuls 2 des 6 machines testées étaient « \$DE00-compatibles » — question de blindage RF).

Matériel mesuré : deux C64, un C128D, un C128DCR ; puces : 6569R1 céramique, 6569R3 céramique, 6569R3 plastique (+ 8566 des C128, révisions inconnues). **Les accès mémoire sont identiques sur toutes ces puces, mais pas le timing complet** : il existe des différences au moins dans le timing sprite du 6569R1.

1.2 Les six chronogrammes (verbatim)

Bad scan line, no sprites:

```
1      2      3      4      5      6
12345678901234567890123456789012345678901234567890123
[ { } ]
3-4-5-6-7-rrrrrggggggggggggggggggggggggggggggggggggggggg -0-1-2- Phi-1 6569R3
cccccccccccccccccccccccccccccccccccccccccccccccccccccc Phi-2 6569R3
xxxxxxxxxxxX=====xxxxxxxxxxx Phi-2 6510
```

Normal scan line, no sprites:

```
1      2      3      4      5      6
```

```

12345678901234567890123456789012345678901234567890123
      [      {                                }      ]
3-4-5-6-7-rrrrrrggggggggggggggggggggggggggggggggggggggg--0-1-2- Phi-1 6569R3
                                         Phi-2 6569R3
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx Phi-2 6510

```

Overscan raster line (vertical border) or blanked screen, no sprites:

```

      1          2          3          4          5          6
12345678901234567890123456789012345678901234567890123
      [      {                                }      ]
3-4-5-6-7-rrrrr+++++-----0-1-2- Phi-1 6569R3
                                         Phi-2 6569R3
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx Phi-2 6510

```

Bad scan line, at least the sprites 3--7 active on the current scan line and the sprites 0--2 on the following scan line:

```

      1          2          3          4          5          6
12345678901234567890123456789012345678901234567890123
      [      {                                }      ]
3s4s5s6s7srrrrrgggggggggggggggggggggggggggggggggggggg--0s1s2s Phi-1 6569R3
ssssssssss ccccccccccccccccccccccccccccccccccccccc ssssss Phi-2 6569R3
=====xXX=====**===== Phi-2 6510

```

Normal scan line, no sprites on the current scan line but at least the sprites 1 and 2 active on the following scan line:

```

      1          2          3          4          5          6
12345678901234567890123456789012345678901234567890123
      [      {                                }      ]
3-4-5-6-7-rrrrrgggggggggggggggggggggggggggggggggggggg--0-1s2s Phi-1 6569R3
                                         ssss Phi-2 6569R3
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxXX== Phi-2 6510

```

Two successive overscan raster lines (vertical border), sprites 1, 3 and 7 active on the latter:

```

      1          2          3          4          5          6
12345678901234567890123456789012345678901234567890123
      [      {                                }      ]
3-4-5-6-7-rrrrr+++++-----0-1s2- Phi-1 6569R3
                                         ss Phi-2 6569R3
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxXX==* Phi-2 6510

3s4-5-6-7srrrrr+++++-----0-1-2- Phi-1 6569R3
ss      ss Phi-2 6569R3
==xxxXX==xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx Phi-2 6510

```

1.3 Légende complète (verbatim)

Note: The left edge of the diagrams is the start of the raster line, as indicated by the Raster Counter register (\$D012/\$D011).

Legend: [left edge of the overscan screen (sprite co-ordinate \$1E0 at the beginning of this cycle)
{ left edge of the text screen (at the beginning of this cycle)
} right edge of the text screen (at the end of this cycle)
] right edge of the overscan screen (a bit after the end of this cycle)

- idle bus cycle (reads from the last byte of the video bank, e.g. (video bank base address) + \$3FFF)
- + idle bus cycle (just like '-', but reads from (video bank base address) + \$39FF if ECM (\$D011 bit 6) is selected)
- 0 pointer fetch for sprite 0
- 1 pointer fetch for sprite 1
- 2 pointer fetch for sprite 2
- 3 pointer fetch for sprite 3
- 4 pointer fetch for sprite 4

5 pointer fetch for sprite 5
 6 pointer fetch for sprite 6
 7 pointer fetch for sprite 7
 r memory refresh cycle
 g graphics fetch
 c character pointer and/or color data fetch

x processor executes instructions (BA high, AEC high)
 X bus request pending, bus still available (BA low, AEC high);
 processor may execute write cycles, stops on the next read cycle.
 * bus request pending, bus still available (BA low, AEC high);
 processor is blocked because it would like to read something.
 = bus unavailable (BA low, AEC low);
 processor is blocked because it would like to read something.

En clair — comment lire un chronogramme : une **colonne** = un **cycle** (63 par ligne PAL) ; la ligne Phi-1 6569 montre ce que le VIC lit pendant SA phase, la ligne Phi-2 6569 ce qu'il prend EN PLUS, et la ligne Phi-2 6510 est **votre budget** — comptez les x. Attention à l'origine des diagrammes : chez Mäkelä, le bord gauche est le début de la ligne raster ; chez Ojala (§2), c'est la coordonnée sprite X 0.

Glose des quatre états du bus 6510 — c'est le cœur du chapitre :

Symbole	BA	AEC	État du 6510
x	haut	haut	exécution normale
X	bas	haut	requête bus en attente ; le CPU peut encore exécuter des cycles d'écriture , il s'arrête au prochain cycle de lecture
*	bas	haut	bus encore disponible, mais CPU bloqué car il voudrait lire
=	bas	bas	bus retiré ; CPU bloqué

Horloges — équivalences de notation : *Phi-1* = première phase du cycle ($\phi 2$ bas) = phase VIC ; *Phi-2* = seconde phase ($\phi 2$ haut) = phase CPU. Les symboles $\phi 0/\phi 2$ (Bauer), **Phi** (Mäkelä) et **ø2** (datasheet SID, ch.4) désignent la même horloge système.

À noter : l'origine des diagrammes de Mäkelä est le **début de la ligne raster** (\$D012/\$D011), contrairement à Ojala dont le bord gauche est la coordonnée sprite X=0 (voir §2).

1.4 Notes clés de Mäkelä

- **Écran overscan** (mesuré) : **411 × 284 points** (ses notes disent 411,5 en horizontal — incertitude reconnue par l'auteur : « peut-être le point le plus à droite était à demi-largeur », ambigu dans la source). Tout à gauche (coord. sprite \$1D8) : barre verticale de 8 pixels couleur 8 (brun foncé), puis barre blanche de 2 pixels (\$1E0-\$1E1). Zone contrôlable par programme : coordonnées **\$1E2 à \$17C** inclus. Lignes raster **\$0-\$137** inclus, visibles **\$10-\$12B**. Écran texte par défaut : coordonnées horizontales **\$18-\$157**, lignes **\$33-\$F9**. (NB : les **403** « pixels visibles » de Bauer [ch.1 §4] mesurent la fenêtre affichée hors blanking — définition différente de la zone overscan de 411 points : les deux chiffres sont exacts.)
- **Latence couleur** : les couleurs changent **8,5 pixels après le début du cycle d'écriture** dans un registre VIC-II (John West dit « un cycle après l'écriture » pour les registres \$20-\$2F — les deux formulations coexistent dans la source). Puces plus récentes que 6569/6567 : point gris clair parasite quand on change une couleur en cours d'utilisation (test : F0RA=1T01E37:POKE53281,0:NEXT).
- **Coordonnées Y des sprites = ligne raster** — 1 : le VIC préfetche les données sprite de la ligne *suivante* juste après l'affichage des caractères de la ligne courante, et la ligne raster ne s'incrémente qu'au moment du fetch de données du quatrième sprite (formulation de la source : « the data for the fourth sprite » — sprite 3 en numérotation 0, ambigu dans la source).
- **Badline** : les pointeurs de caractères sont lus **un cycle avant** les données image de la première scanline d'une ligne texte ; de même les données graphiques sont lues un cycle avant affichage (buffering).
- **Sprites bufferisés** : sprite 0 collé au bord droit de l'overscan → bord droit distordu (le VIC est en train de fetcher ses données à cet endroit). Les données sprite sont fetchées **même hors de l'écran overscan** et pendant le retour vertical, si la coordonnée X sprite (*sic* — la source écrit « horizontal », mais Bauer règle 2 [ch.1 §10.1] et la conséquence « Y 0-55 → double fetch » de la phrase suivante imposent la coordonnée Y) correspond aux 8 bits bas du compteur raster ; conséquence : avec Y sprite entre **0 et 55 inclus**, les données sont **fetchées deux fois par frame** (sprite visible en haut ET en bas si bordures ouvertes ; avec expansion Y : haut du sprite en bas d'écran, bas du sprite en haut, sprite entier près du haut).

- **Bordure verticale ouverte** (basculer 24/25 lignes pendant la 25e ligne texte) : les cycles - affichent le dernier octet de la banque vidéo (\$3FFF par défaut), graphisme toujours noir (« ou c'est ce qu'on nous a dit » — l'auteur demande si quelqu'un sait le colorer).
- **Modes illégaux** : ECM+BMM → écran noir, adresses bitmap ANDées avec \$B9FF ; les cycles + lisent \$39FF en bordure aussi dans ce mode hybride. ECM+MCM : écran noir mais adressage singulier — A13–A11 = base générateur de caractères, A10–A9 = 0, **A8–A5 = bits de base de la matrice texte**, A4–A3 apparemment toujours 0, A2–A0 = scanline du caractère (1–7 hors badline ; 0 sur badline, non mesurable sans matériel supplémentaire). Probablement un mode de test d'usine. **Tous les modes illégaux ont des badlines** — aucun intérêt pratique sauf obfuscation.
- **Refresh mémoire** : 5 cycles r par ligne. Première adresse rafraîchie sur une ligne raster donnée : (**base banque vidéo**) OR \$3F00 OR (\$FF AND (–1–5×ligne)), compteur **décroissant**. Exemple (banque 0) : lignes \$36 et \$136 → lectures \$3FF1, \$3FF0, \$3FEF, \$3FEE, \$3FED.
- **Horloges** : système = 17734472/18 Hz en PAL, 14318181/14 Hz en NTSC ; dot clock = 8×. PAL : 63 cycles/ligne, 312 lignes. NTSC : le 6567R8 testé avait 64 cycles/262 lignes (frame rate 60,9928 Hz), les 6567 habituels ont 65 cycles/263 lignes. (⚠ *Divergence avec la table de Bauer [ch.1 §4], qui attribue 64 cycles/262 lignes au 6567R56A et 65/263 au 6567R8 — témoignages contradictoires entre les deux sources, non résolus ici.*)

2. [Ojala] « Missing Cycles » — Pasi Ojala, C=Hacking #3 (écrit 15-mai-91, traduit 30-mai-92)

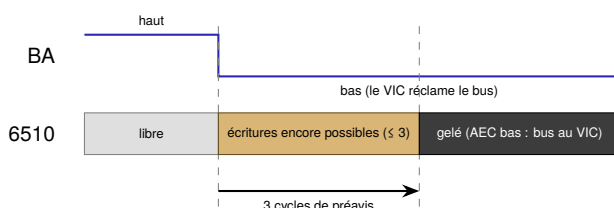
Tous les timings sont PAL ; le principe s'applique au NTSC.

2.1 Le mécanisme du vol de cycles

Base : **63 cycles CPU par ligne**, sauf la « bad line » qui n'en laisse que 23. Le VIC fait plus qu'afficher : sur chaque scanline il **rafraîchit 5 rangées mémoire** et **fetche 40 octets de données graphiques**, le tout pendant les cycles Phi-1 que le CPU n'utilise pas. Ces cycles ne suffisent plus quand le VIC doit aussi lire les codes caractère et couleur de la rangée suivante : le bus étant unique, l'accès CPU est refusé. Le bus VIC de **12 bits** permet heureusement de lire caractère (8 bits) et couleur (4 bits) **en même temps**.

Sprites : sur chaque ligne, les pointeurs image des sprites sont fetchés en Phi-1. Si le sprite est affiché sur cette ligne, les **3 octets** de données image sont fetchés juste après — **2 de ces 3 fetches ont lieu en Phi-2**, donc le CPU les perd. En moyenne : **2 cycles perdus par sprite affiché** (plus l'overhead BA, ci-dessous).

2.2 Le signal BA — 3 cycles d'avance



BA tombe **3 cycles avant** la prise du bus : le maximum d'écritures consécutives du 6510 est 3, et RDY est ignoré en écriture.
Le CPU finit ses écritures puis s'arrête à sa première lecture. [Bauer §2.4.3] · [Ojala] · [64doc]

Figure 2.2 — La prise de bus vue du CPU. C'est cette fenêtre d'écritures qui rend possible la synchronisation par sprite (§2.4) — et c'est elle que le displayer de l'annexe B exploite.

Quand le VIC veut le bus, **BA (Bus Available) devient inactif 3 cycles avant** que le bus doive être libéré. Pendant ces 3 cycles, le CPU doit finir ses accès mémoire ou les différer. Il peut terminer l'instruction en cours dans les cycles restants (pendant BA bas le bus lui reste disponible tant qu'AEC est haut : ses **cycles d'écriture** passent encore ; il s'arrête au premier cycle de lecture), mais **il ne peut pas démarrer une nouvelle instruction tant qu'il n'a pas le bus**. C'est pourquoi des cycles « disparaissent » au-delà de ceux volés directement pour les sprites.

Compte théorique, 8 sprites sur la même ligne : 16 cycles de fetch (2×8) + **3 cycles d'overhead BA = 19 cycles** volés. En pratique souvent moins, car le CPU consomme une partie de la fenêtre BA en cycles d'écriture ; le texte dit « habituellement les 8 sprites prennent 17 cycles, un sprite en prend 3 » et ailleurs « 16 à 19 cycles

2.4 Les règles pratiques d'Ojala

1. **Budgets par type de ligne** (table ci-dessus) : 63 / 46–49 (8 sprites) / 48–51 (4 sprites) / 23 (badline) / 4–7 cycles (badline + 8 sprites — le pire cas absolu).
2. **Dans les routines raster critiques, toujours activer les sprites dans l'ordre** : désactiver un sprite « au milieu » (fenêtres BA jointives) ne rend aucun cycle.
3. **Synchronisation par sprite** : normalement, à l'entrée d'une IRQ raster, le CPU finit une instruction de longueur indéterminée (2 à 7 cycles de gigue). Avec un sprite bien placé, un simple **DEC ou INC au bon endroit** synchronise : si le CPU est en avance, il attend le bus ; sinon il continue son instruction. Il faut une instruction dont un cycle **peut s'exécuter pendant BA bas** — p. ex. **INC \$3FFF** : son 5^e cycle est une **écriture** (ch.3 §3.3), que le CPU exécute encore bus-réclamé (état X de la glose ci-dessus) — c'est précisément pourquoi la synchronisation fonctionne. (Ojala n'a testé que DEC/INC ; d'autres « devraient marcher aussi ».)
4. **8 pixels par cycle** : le faisceau parcourt 8 pixels par cycle d'horloge, d'où la précision requise pour un effet raster horizontal.

2.5 Le programme de démonstration (résumé)

Barres de couleur : le fond (\$D020) est changé **12 fois par ligne** sur 104 lignes (LDY #103 ; « réduire pour NTSC, 55 ? »). Sync : IRQ à la ligne RASTER=\$FA, **sprite 0 placé 20 lignes plus haut** (\$D001 = RASTER–20) pour que son DMA serve de point d'ancrage, puis INC DUMMY / DEC DUMMY (DUMMY=\$CFFF, instructions à 6 cycles) réalisent la synchronisation ; boucle calibrée à 63 cycles exactement (LDA #0 jette les 2 cycles restants ; ajouter des NOP pour NTSC, 65 cycles/ligne). La source complète (assembleur, chargeur BASIC avec checksums, binaires uuencodés PAL et NTSC) figure dans le texte original [Ojala].

3. [MISC] Massively Interleaved Sprite Crunch — Linus Åkesson, 19 déc. 2016

Effet introduit dans la démo **Lunatico** (partie greetings). Le MISC combine **trois techniques nouvelles** exploitant des bizarreries connues du VIC.

3.1 DMA sprite normal : MCBASE et MC

Les pixels d'un sprite occupent **63 octets** (21 rangées × 3 octets), alignés sur 64 octets ; les bits hauts de l'adresse = pointeur sprite, les bits bas = **offset 6 bits** tenu dans le registre interne **MCBASE** (un par sprite). Fonctionnement normal : à l'activation, MCBASE = 0 ; à chaque ligne avec DMA actif, MCBASE est copié dans **MC**, MC est incrémenté pendant le fetch des 3 octets (donc MC = MCBASE + 3 mod 64 après la rangée) ; puis, **selon le latch d'expansion Y**, MCBASE est soit laissé tel quel, soit mis à MC. Répétition jusqu'à **MCBASE = 63**. En mode Y-expand, le latch bascule à chaque ligne → chaque rangée est fetchée deux fois. Contrainte fondamentale : un sprite activé reste actif jusqu'à émission de ses 21 lignes — **on ne peut pas l'avorter** à la ligne 10.

3.2 Le bug « sprite crunch »

Découvert par **Pernod** dans *Rutig Banan* (1989), exploré et nommé par **Crossbow** dans *Krestage* (1997). Déclenchement : **désactiver Y-expand au 15^e cycle d'horloge d'une rasterline**. Le latch bascule pendant que le VIC est en train de décider s'il assigne MCBASE depuis MC ou depuis lui-même ; le matériel, confus, produit :

$$\text{MCBASE} = ((\text{MC} \mid \text{MCBASE}) \& 0x15) \mid ((\text{MC} \& \text{MCBASE}) \& 0x2a)$$

c'est-à-dire **OR bit à bit sur les bits pairs, AND sur les bits impairs** (ce n'est PAS équivalent à un OR global, précise Åkesson en commentaire). Explication probable (**hypothèse** de l'auteur, pas mesurée) : un bit sur deux est inversé dans la représentation interne — technique NMOS classique pour accélérer l'incrément de MC sur trois demi-cycles successifs — et l'opération réelle serait un AND global sur cette représentation. On peut donc **sauter dans les données du sprite**, mais uniquement selon un graphe prédéfini de transitions. Crossbow avait trouvé le plus court chemin début→fin : **17 nœuds**, d'où le « sprite minimum 17 lignes » — vrai seulement si on part du début. La séquence courte **15–18–1b–1e** (boucle de 4 lignes depuis l'offset \$15) a servi dans *Edge of Disgrace* (2008) et *Shards of Fancy* (2013).

3.3 Innovation 1 : boucles depuis l'offset \$35 — hauteurs possibles

En étudiant le graphe, Åkesson découvre qu'en partant de l'offset **\$35**, on dispose de **huit boucles** de longueurs **1, 13, 14, 17, 18, 19, 20 et 21** lignes. En organisant les pixels avec le coin haut-gauche de l'objet à l'offset \$35, on crée des **colonnes de sprites de hauteur variable**. Lunatico utilise quatre hauteurs (14, 17, 19, 21), avec ces *crunch schedules* (séquences d'offsets MCBASE, verbatim) :

```
35-38-3b-3e-15-18-1b-1e-21-25-28-2b-2e-31
```

```
35-38-3b-3e-01-05-08-0b-0f-17-1a-1d-20-23-27-2a-2d
```

```
35-38-3b-3e-01-04-07-0a-0d-15-18-1b-1e-21-25-28-2b-2e-31
```

```
35-38-3b-3e-01-05-08-0b-0e-11-14-17-1a-1d-20-23-26-29-2c-2f-32
```

Exemple pour la hauteur 17 : partir de \$35, avancer normalement (+3) et **déclencher le glitch aux offsets \$01, \$0b, \$0f, \$23 et \$2d** — retour à \$35 en exactement 17 pas.

3.4 Innovation 2 : la boucle pleine hauteur (21 lignes)

La dernière séquence ci-dessus boucle sur **21 lignes** — la hauteur normale d'un sprite — grâce à un crunch au bon moment (saut \$01→\$05 ; possible aussi à \$03 ou à \$39). Intérêt général : pour un « **tapis de sprites** » pleine largeur (grille de sprites bord à bord), il faut normalement **au moins 34 cycles par rangée de sprites** rien que pour réécrire les registres Y ; avec la boucle 21 lignes, **12 cycles** suffisent. Inconvénients : timing précis obligatoire, et **ne fonctionne pas sur les badlines**.

3.5 Innovation 3 : le schedule encodé dans les pixels, lu par collision

Problème : avec les 8 sprites actifs, il ne reste que **44 cycles par rasterline** pour calculer et écrire les 8 bits du registre Y-expand au cycle près. Solution : **encoder le crunch schedule dans les pixels du sprite lui-même** et le lire via la **détection de collision sprite/fond** (\$D01F — latches mis à 1 à la collision, effacés à la lecture, réutilisables plusieurs fois par frame). Chaque colonne est couverte d'une fine bande noire de pixels *foreground* cachée dans le bitmap (fond noir → invisible), sur les **pixels les plus à droite** de chaque sprite : quand ce pixel est allumé, il déclenche une collision. Lire \$D01F en fin de ligne donne donc le pixel droit de la rangée courante des 8 sprites, **déjà emballé en un octet**, copié tel quel dans \$D017 (Y-expand). Les mises à jour des pointeurs sprite après chaque lettre aiguillent ensuite le parcours du graphe vers le schedule de la lettre suivante : la gestion de chaque crunch individuel se réduit à la gestion d'**une mise à jour de pointeur par lettre**.

3.6 Contraintes de timing

- **Y-expand doit être activé avant d'être coupé au cycle 15** ; si on ne veut pas de crunch, il ne faut pas l'activer du tout (sinon une ligne s'affiche deux fois). L'information dynamique dans les pixels indique donc « faut-il armer Y-expand », et au cycle 15 on coupe pour tous les sprites.
- **Délai de 3 rasterlines** : données fetchées ligne 0 → affichées ligne 1 → échantillonnées (collision) tôt ligne 2 → crunch ligne 3. Le schedule doit être encodé **3 lignes avant** dans les pixels.
- Tous les schedules commencent par **35-38-3b-3e**, ce qui rend les coutures sûres ; impossible de mettre à jour 8 pointeurs sur une ligne → **sprites 0-4 sur lignes impaires, 5-7 sur lignes paires**, d'où la **rangée VIERGE (pixels vides) obligatoire à l'offset \$35** (un pointeur en retard d'une ligne ne fetch que du vide — « blank » = transparent, PAS blanc).
- Le DMA sprite prend les **cycles 55 à 10** (le cycle 55 reste utilisable en écriture). Code d'une paire de lignes (verbatim, déroulé ~100 fois) :

```
;          Invariants: x = y = 0      Cycle

template
    nop                ; 54 11

    sty    $d017        ; 12, crunch
    lda    $d01f        ; 16
    sta    $d017        ; 20

tem_s5
    lda    #$80         ; 24
    lda    vm+$3fd      ; 26

tem_s6
```

```

lda    #$80      ; 30
lda    vm+$3fe    ; 32
tem_s7

lda    #$80      ; 36
lda    vm+$3ff    ; 38

nop     ; 42
nop     ; 44
nop     ; 46
nop     ; 48
tem_rr

lda    #$3e      ; 50
sta    $d011     ; 52, repeat row

shy     $d017,x   ; 11, crunch
lda     $d01f     ; 16
sta     $d017     ; 20
tem_s0

lda     #$80      ; 24
lda     vm+$3f8    ; 26
tem_s1

lda     #$80      ; 30
lda     vm+$3f9    ; 32
tem_s2

lda     #$80      ; 36
lda     vm+$3fa    ; 38
tem_s3

lda     #$80      ; 42
lda     vm+$3fb    ; 44
tem_s4

lda     #$80      ; 48
lda     vm+$3fc    ; 50

```

Détails : le crunch est déclenché par l'écriture dans \$d017 au **cycle 15** (dernier cycle du sty, 4 cycles ; ou du shy, illégal 5 cycles — nécessaire quand on revient du DMA au cycle 11, faute d'instruction à 1 cycle). L'écriture \$d011 au cycle 55 une ligne sur deux **repousse les badlines**, qui ruineraient le timing. #\$80 est une valeur factice : pendant le vblank, les pointeurs sprite sont écrits dans ces opérandes et les lda transformés en sta (puis remis en lda inoffensifs).

- **Initialisation** : les offsets de départ (\$35 n'est pas atteignable en séquence normale, tous les offsets normaux étant divisibles par 3) sont obtenus en démarrant les 8 sprites à la même coordonnée Y dans la bordure haute et en les crunchant tous à **\$2d** (→ \$35) ; ensuite, pendant N lignes (selon le scroll de chaque colonne), la valeur de \$D01F est **forcée à 1**, ce qui crunch de \$35 vers \$35 (boucle de longueur 1) et « étire » le sprite. Masques pris dans une table de **22 octets** calculée pendant le vblank.
- **Animation** : seules 2 ou 3 colonnes bougent par frame (2 colonnes, appel du player musique, puis lecture de la position raster pour décider d'une 3e) — visuellement l'effet passe pour du 50 fps.

3.7 Les trois techniques du MISC en une phrase chacune

1. **Boucles à hauteur variable depuis \$35** : huit longueurs de boucle (1, 13, 14, 17, 18, 19, 20, 21) dans le graphe du crunch → colonnes de sprites de hauteur choisie.
2. **Boucle pleine hauteur 21 lignes** : un crunch au bon offset (\$01→\$05, ou à \$03/\$39) reboucle un sprite entier → tapis de sprites pour 12 cycles/rangée au lieu de 34.
3. **Schedule auto-porté** : les crunchs sont encodés dans le pixel droit de chaque rangée du sprite et récupérés en un octet via \$D01F (collision sprite/fond), copié dans \$D017 — avec un délai obligatoire de 3 rasterlines.

Croisement des trois sources

Les trois textes se recoupent exactement sur les invariants : **BA tombe 3 cycles avant la prise de bus** (Ojala §2.2 = états x/* de Mäkelä, où seuls les cycles d'écriture passent encore) ; **2 cycles Phi-2 volés par sprite + fenêtres jointives** (Ojala) = paires s en Phi-2 des chronogrammes de Mäkelä ; **DMA sprite en début/fin de ligne** (fenêtre 0s1s2s...7s de Mäkelä) = « cycles 55 à 10 » d'Åkesson, dont le template exploite précisément la règle d'Ojala (l'écriture au cycle 55 passe pendant BA bas). Divergences à retenir : les chiffres de cycles volés d'Ojala (16–19 dans le texte, 14–17 implicites dans sa table) sont approximatifs de son propre aveu, et

Mäkelä — la référence mesurée — utilise une **origine de diagramme différente** (début de ligne raster vs coordonnée sprite X 0). Autre réconciliation : les **44 cycles** restants d'Åkesson (8 sprites) vs les **46–49** d'Ojala — le DMA sprite occupe les cycles 55 à 10 (19 cycles) → $63 - 19 = 44$; Ojala crédite en plus les écritures que le CPU place encore dans les fenêtres BA. En pratique : ce chapitre explique pourquoi une badline gèle le CPU même à horloge accélérée (états = : le bus est au VIC, pas au 6510) et pourquoi tout sprite ajouté à l'écran se paie en fenêtres BA, pas seulement en cycles de fetch.

Sources : [pal.timing] Mäkelä 1994 · [Ojala] Ojala 1992 · [MISC] Åkesson 2016 (références complètes en bibliographie).

Chapitre 3 — Le 6510 : cycles et interruptions (64doc)

Le versant processeur : ce que coûte chaque instruction, cycle par cycle, et ce qui se passe exactement quand une interruption frappe. C'est ici que vivent les pièges qui font ou défont une routine raster — double écriture des RMW, lectures parasites, latences d'IRQ.

Source unique : [64doc] — « Documentation for the NMOS 65xx/85xx Instruction Set », John West & Marko Mäkelä, rév. 1.8 du 3 juin 1994 (fichier du projet X64). Tout ce chapitre est **documenté** (mesuré par les auteurs sur 6510/8502 NMOS réels) ; les points que la source elle-même qualifie d'incertains sont marqués.

1. Principes de base du séquençement

[64doc §6510 Instruction Timing, 1.864–878]

- Le 65xx NMOS effectue **au moins deux lectures par instruction** : l'opcode, puis l'octet suivant (même pour les instructions à 1 octet — cet octet est lu et jeté).
- **Pipeline** : si le dernier cycle d'une instruction n'écrit pas en mémoire, le processeur y superpose le fetch de l'opcode suivant. EOR # $\text{\$FF}$ prend « vraiment » 3 cycles, mais le 3^e est recouvert → 2 cycles effectifs. Les diagrammes ci-dessous incluent parfois ce fetch : pour le temps réel, soustraire le dernier cycle quand la source le précise (cas des branchements).
- **RMW = double écriture** : les instructions lecture-modification-écriture écrivent d'abord la valeur *non modifiée*, puis la valeur modifiée (« INC effectively does LDX loc;STX loc;INX;STX loc ») [1.812–813].
- **-RDY est ignoré pendant les écritures** : d'où l'attente de 3 cycles avant un DMA — le maximum d'écritures consécutives est 3, atteint pendant les séquences d'interruption (sauf -RESET) [1.815–818].
- **Lectures parasites (dummy reads)** : tout franchissement de page en mode indexé provoque d'abord une lecture à une adresse fausse (trop basse d'une page) [1.808–810] ; l'octet qui suit un branchement est toujours lu [1.802–806]. Conséquence pratique : ces accès parasites touchent réellement le bus (voir §4.4, acquittement d'interruptions par effets de bord).
- **Pas de gestion du franchissement de page en indirect** : pointeur $\text{\$xxFF}$ → l'octet haut est lu à $\text{\$xx00}$ (JMP ($\text{\$01FF}$) lit PCH à $\text{\$0100}$; LDA ($\text{\$FF}$), Y lit la base à $\text{\$FF}$ et $\text{\$00}$). Idem en zéropage indexé : LDA ($\text{\$FF}, X$) avec X=1 lit l'adresse effective à $\text{\$00}/\text{\$01}$ [1.792–800].

2. Cycles par mode d'adressage — récapitulatif

Synthèse des diagrammes [64doc 1.934–1374]. +1* = un cycle de plus si franchissement de page.

Mode	Lecture	RMW	Écriture
Implicite / Accumulateur	2	—	—
Immédiat	2	—	—
Zéropage	3	5	3
Zéropage,X (ou ,Y)	4	6	4
Absolu	4	6	4
Absolu,X / Absolu,Y	4 +1*	7 (fixe)	5 (fixe)
(indirect,X)	6	8	6
(indirect),Y	5 +1*	8 (fixe)	6 (fixe)

Instructions particulières :

Instruction	Cycles	Instruction	Cycles
JMP absolu	3	PHA/PHP	3
JMP (indirect)	5	PLA/PLP	4
JSR	6	RTS	6
BRK	7	RTI	6

Instruction	Cycles	Instruction	Cycles
Branchement non pris	2	Branchement pris	3 (+1 si page franchie)
IRQ / NMI	7	RESET	~6 (« probably », ambigu dans la source) [1.929–930]

Règles à retenir : **écriture indexée et RMW indexé paient toujours le cycle de correction de page**, pris ou pas (le CPU ne peut pas annuler une écriture à une adresse fausse, il lit donc d'abord) ; seule la *lecture* indexée économise ce cycle quand la page n'est pas franchie.

3. Détail cycle par cycle — les cas piégeux (verbatim)

3.1 Lecture absolue indexée, franchissement de page [64doc I.1155–1176]

```
#  address R/W description
-----
1  PC      R  fetch opcode, increment PC
2  PC      R  fetch low byte of address, increment PC
3  PC      R  fetch high byte of address,
               add index register to low address byte,
               increment PC
4  address+I* R  read from effective address,
               fix the high byte of effective address
5+ address+I R  re-read from effective address

Notes: I denotes either index register (X or Y).
      * The high byte of the effective address may be invalid
        at this time, i.e. it may be smaller by $100.
      + This cycle will be executed only if the effective address
        was invalid during cycle #4, i.e. page boundary was crossed.
```

3.2 Écriture absolue indexée : lecture parasite obligatoire (5 cycles fixes) [64doc I.1199–1217]

```
#  address R/W description
-----
1  PC      R  fetch opcode, increment PC
2  PC      R  fetch low byte of address, increment PC
3  PC      R  fetch high byte of address,
               add index register to low address byte,
               increment PC
4  address+I* R  read from effective address,
               fix the high byte of effective address
5  address+I W  write to effective address

Notes: * ... it may be smaller by $100. Because
        the processor cannot undo a write to an invalid
        address, it always reads from the address first.
```

3.3 RMW absolu : la double écriture (6 cycles) [64doc I.1046–1057]

```
#  address R/W description
-----
1  PC      R  fetch opcode, increment PC
2  PC      R  fetch low byte of address, increment PC
3  PC      R  fetch high byte of address, increment PC
4  address R  read from effective address
5  address W  write the value back to effective address,
               and do the operation on it
6  address W  write the new value to effective address
```

En RMW absolu, X (7 cycles), s'ajoutent la lecture parasite (cycle 4, adresse potentiellement fausse) puis la relecture, avant les deux écritures [1.1178–1196]. En RMW (ind), Y : 8 cycles fixes [1.1320–1334].

En clair — une instruction lecture-modification-écriture écrit **deux fois** : d'abord la valeur inchangée, puis le résultat. Sur de la RAM c'est invisible ; sur un registre d'E/S, la première écriture est un événement réel — c'est le fondement de l'acquiescement en un seul opcode (LSR \$D019, §4.4).

3.4 Branchements (relatif) [64doc I.1220–1245]

#	address	R/W	description

1	PC	R	fetch opcode, increment PC
2	PC	R	fetch operand, increment PC
3	PC	R	Fetch opcode of next instruction, If branch is taken, add operand to PCL. Otherwise increment PC.
4+	PC*	R	Fetch opcode of next instruction. Fix PCH. If it did not change, increment PC.
5!	PC	R	Fetch opcode of next instruction, increment PC.

Notes: The opcode fetch of the next instruction is included to this diagram for illustration purposes. When determining real execution times, remember to subtract the last cycle.

* The high byte of Program Counter (PCH) may be invalid at this time, i.e. it may be smaller or bigger by \$100.
+ If branch is taken, this cycle will be executed.
! If branch occurs to different page, this cycle will be executed.

Donc : 2 cycles non pris, 3 pris même page, 4 pris avec page franchie — avec, en cas de franchissement, un fetch parasite dans l'ancienne page (adresse $\pm \$100$) [1.802–806].

3.5 (indirect),Y lecture [64doc I.1297–1317]

#	address	R/W	description

1	PC	R	fetch opcode, increment PC
2	PC	R	fetch pointer address, increment PC
3	pointer	R	fetch effective address low
4	pointer+1	R	fetch effective address high, add Y to low byte of effective address
5	address+Y*	R	read from effective address, fix high byte of effective address
6+	address+Y	R	read from effective address

(Le pointeur est toujours lu en page zéro, sans gestion du wrap \$FF→\$00.)

3.6 JMP (indirect) : le bug \$xFF [64doc I.1363–1374]

#	address	R/W	description

1	PC	R	fetch opcode, increment PC
2	PC	R	fetch pointer address low, increment PC
3	PC	R	fetch pointer address high, increment PC
4	pointer	R	fetch low address to latch
5	pointer+1*	R	fetch PCH, copy latch to PCL

Note: * The PCH will always be fetched from the same page than PCL, i.e. page boundary crossing is not handled.

3.7 BRK (7 cycles) [64doc I.936–947]

#	address	R/W	description

1	PC	R	fetch opcode, increment PC
2	PC	R	read next instruction byte (and throw it away), increment PC
3	\$0100,S	W	push PCH on stack, decrement S
4	\$0100,S	W	push PCL on stack, decrement S
5	\$0100,S	W	push P on stack (with B flag set), decrement S
6	\$FFFE	R	fetch PCL
7	\$FFFF	R	fetch PCH

Autres accès pile : JSR = fetch opcode + fetch adr basse + 1 cycle interne pile + push PCH + push PCL + fetch adr haute (6 cy) [1.993–1003] ; RTS ajoute un incrément final de PC (6 cy) [1.962–971] ; RTI = 6 cy [1.950–959].

4. Interruptions [64doc §Interrupts, I.883–931]

4.1 Reconnaissance et latence

- **NMI et IRQ prennent 7 cycles**, séquence analogue à BRK (mais B poussé à 0). IRQ n'est exécutée que si le flag I est à 0 ; IRQ et BRK positionnent I, **NMI ne touche pas I**.
- **Règle de latence** : le processeur termine l'instruction en cours avant la séquence d'interruption. Pour être traitée *avant* l'instruction suivante, « the interrupt must occur before the last cycle of the current instruction » [I.890–893]. Latence donc : 7 cycles + le reliquat de l'instruction en cours ; si l'IRQ tombe SUR le dernier cycle, elle rate la frontière et attend l'instruction SUIVANTE entière (reliquat au pire 8 cycles, RMW (ind),Y).
- **Le cas branch** : Δ le raffinement bien connu « une branche prise retarde d'un cycle la prise en compte de l'IRQ » **n'est pas couvert par 64doc** — la source ne donne que la règle générale ci-dessus (ambigu dans la source pour ce cas précis). Ce que la source documente sur les branchements, c'est leurs fetchs parasites (§3.4) et leur usage pour acquitter des interruptions (§4.4).

4.2 BRK détourné (hijack) et priorités

- **Exception unique à la règle** : si une interruption matérielle (NMI ou IRQ) survient **avant le « cycle d'empilement de P » de BRK** (la prose de 64doc l'appelle « fourth (flags saving) cycle » [I.895], mais sa table le place au cycle 5 [I.938] — conflit de numérotation interne à la source), BRK est « sauté » : le processeur saute au vecteur matériel, la séquence fait toujours 7 cycles — mais **le P empilé porte B=1** [I.895–903]. BRK+IRQ simultanés : bénin (l'IRQ re-survient après le RTI si sa source n'est pas acquittée). **BRK+NMI : fatal** si le handler NMI ne teste pas le flag B (et ne retranche pas 2 à l'adresse de retour) — le BRK est perdu [I.903–910].
- Officiellement BRK ne saute qu'au vecteur IRQ \$FFFE ; en réalité un NMI pendant BRK détourne vers \$FFFA avec B=1 empilé [I.229–234].
- **NMI et IRQ qui se chevauchent** : « le processeur sautera *très probablement* au vecteur NMI, puis au vecteur IRQ après la première instruction du handler NMI. Ceci n'a pas encore été mesuré » — il se pourrait aussi qu'un NMI soit perdu si une IRQ arrive dans les 4 cycles suivants (**ambigu dans la source**, hypothèse explicite des auteurs) [I.912–921].
- **RESET** : n'empile pas PC, dure « probablement » 6 cycles après relâchement du signal ; comme NMI, préserve tous les registres sauf PC [I.929–931] — d'où l'obligation d'initialiser D et I dans le handler RESET [I.822–823].

4.3 Pipeline et fin de séquence

Après la séquence d'interruption, le fetch de la première instruction du handler est recouvert par le dernier cycle — même pipeline que les instructions normales [I.923–927].

4.4 Acquittement par effets de bord (« Real Programmers ») [64doc I.1378–1610]

Applications directes des lectures/écritures parasites — utile en démo/raster :

- **RMW sur registre d'I/O** : LSR \$D019 acquitte l'IRQ VIC (cycle 5 = réécriture de la valeur lue) *et* récupère le flag raster dans C — un seul opcode pour tester et acquitter [I.1382–1401] :

Operational diagram of LSR \$D019:

#	data	address	R/W	
1	4E	PC	R	fetch opcode
2	19	PC+1	R	fetch address low
3	D0	PC+2	R	fetch address high
4	xx	\$D019	R	read memory
5	xx	\$D019	W	write the value back, rotate right
6	xx/2	\$D019	W	write the new value back

- **Indexé avec page franchie** : LDX #\$10 : LDA \$DCFD, X lit parasitement \$DC0D (cycle 4) puis \$DD0D (cycle 5) → acquitte **les deux CIA** en une instruction [I.1405–1420]. Variante écriture : STA \$DDFD, X lit \$DD0D puis écrit \$DE0D.
- **Branchements** : un BPL placé dans les registres CIA acquitte via ses fetchs parasites (cycles 3–4 lisent \$xx0D) [I.1437–1538].
- **RTI logé en \$DD0C** : le fetch systématique de l'octet suivant (\$DD0D = ICR) acquitte le CIA 2 pendant le RTI lui-même — « the fastest possible interrupt handler in the 6500 family » [I.1540–1597].

5. Opcodes non documentés [64doc I.61–119, 308–582, 750–755]

Matrice complète en [I.61–100]. Légende source : * = non documenté, ** = opération inhabituelle, t = bloque la machine, *t = bloque très rarement.

Opcode(s)	Comportement (une ligne)
SLO, RLA, SRE (\$x3/\$x7/\$xF/\$xB...)	RMW combinés (classés avec ASL/ROL/LSR dans toutes les tables de timing RMW [I.1046+]) ; la source ne détaille pas leur opération, contrairement à RRA/ISB/DCP.
RRA	ROR mémoire + ADC, hérite du mode décimal de l'ADC officiel [I.750–753].
ISB	INC mémoire + SBC, hérite du mode décimal ; flags insensibles à D comme SBC [I.750–755].
DCP	DEC mémoire + CMP ; flags insensibles au flag D [I.754–755].
SAX (\$87/\$97/\$8F/\$83)	Range (A & X) — le AND est fait par les drivers open-collector du bus, pas par l'ALU [I.349–353].
LAX	Charge le même octet simultanément dans A et X [I.355–356].
NOP* (variantes multi-modes)	Comme un load : lit l'opérande (dummy reads compris) mais ne stocke rien et ne touche aucun flag [I.116–119].
SBC \$EB	Identique au SBC officiel (« 'NOP' seems to completely disappear ») [I.496–498].
ANC (\$0B/\$2B)	Immédiat AND A, puis l'opération accumulateur de sa colonne (ASL/ROL) — principe de la ligne \$0B [I.500–507].
ASR (\$4B)	A = (A AND #imm) puis LSR A [I.500–507].
ARR (\$6B)	AND #imm + ROR A, mais flags hérités d'ADC : C = bit 6 du résultat, V = bit 6 XOR bit 5 ; mode décimal spécial avec fixup BCD des deux nybbles (code C complet [I.394–424]) [I.359–392].
SBX (\$CB)	X ← (A & X) — #imm ; flags façon CMP : D et C ignorés en entrée, C positionné par le résultat, V intact, pas de mode décimal [I.427–437].
ANE (\$8B)	A = (A #\$EE) & X & #imm — la constante \$EE varie (\$8C/\$CC/\$0C/\$8E... selon machine, DMA VIC, mode décimal) : instable, ne pas utiliser [I.314–323, 509–544, 567].
LXA (\$AB)	A = X = (A & #imm) ou variante ANE selon le processeur ; résultats aléatoires observés : instable, ne pas utiliser [I.325–328, 547–567].
SHA (\$93/\$9F)	Range (A & X & (ADDR_HI + 1)) ; si page franchie, cette valeur écrase aussi l'octet haut de l'adresse effective [I.330–336, 569–581].
SHX (\$9E)	Range (X & (ADDR_HI + 1)), même corruption d'adresse si page franchie [I.331].
SHY (\$9C)	Range (Y & (ADDR_HI + 1)), idem [I.332].
SHS (\$9B)	SHA + TXS avec X remplacé par (A & X) : S ← A & X, et range (A & X & (ADDR_HI + 1)) [I.333].
LAS/LAE (\$BB)	Marqué ** dans la matrice et listé parmi les lectures absolue, Y [I.196, 1156] ; comportement non décrit (ambigu dans la source).
*t / t (\$02, \$12, \$22... ; \$x2)	t bloque la machine ; *t bloque très rarement [I.109–110].

Timing : les non documentés suivent exactement les diagrammes de leur classe (LAX = lecture ; SLO/SRE/RLA/RRA/ISB/DCP = RMW ; SAX/SHA/SHX/SHY = écriture) — ils figurent nommément dans les tables du §3.

6. Le port du 6510 : adresses \$00/\$01 [64doc §Memory Management, I.1625–1656]

- \$00 = **DDR** (Data Direction Register), \$01 = **PR** (Peripheral Register). Bit à 1 dans le DDR → ligne en sortie pilotée par le bit PR ; bit à 0 → le bit PR reflète l'état de la ligne. 6 lignes (P0–P5) sur 6510 ; le 8502 (C128) en a 7 (P6 = touche Caps Lock).

Direction	Line	Function
-----	-----	-----
out	P5	Cassette motor control. (0 = motor spins)
in	P4	Cassette sense. (0 = PLAY button depressed)
out	P3	Cassette write data.
out	P2	CHAREN
out	P1	HIRAM
out	P0	LORAM

- **Défauts** : DDR = \$2F, PR = \$37 [I.1648–1650]. Une ligne de gestion mémoire passée en *entrée* lit « 1 » via les pull-ups externes ; -RESET remet toutes les lignes en entrée → les trois lignes LORAM/HIRAM/CHAREN montent à 1 → **les ROM sont toujours réactivées au reset** [I.1651–1656].

- LORAM/HIRAM/CHAREN + -EXROM/-GAME (port cartouche) sélectionnent la configuration mémoire (table complète des 9 configurations en [1.1684–1719]). CHAREN choisit entre I/O et ROM caractères en \$D000–\$DFFF. Toute écriture vers une zone ROM traverse vers la RAM sous-jacente (sauf configuration Ultimix) [1.1761–1778].
- **Écrire les octets RAM 0 et 1** (sous le port) : lors d'un STA vers 0/1, le CPU pose l'adresse et R/-W mais **ne pilote pas le bus de données** — c'est la dernière valeur laissée sur le bus par le VIC qui est écrite en RAM. Astuce documentée : faire lire \$3FFF au VIC (bordure haut/bas) au moment du store [1.1862–1881] ; peu fiable sur toutes les machines [1.1905]. Lecture des octets 0/1 : via le VIC (bitmap/sprite en \$0000 + registres de collision) ou en lisant l'espace d'adressage ouvert \$DE00–\$DFFF juste après un accès VIC [1.1846–1903].

7. Divers normatifs utiles

- PHP (et BRK) empile **toujours B=1** [1.785].
- Comparaisons (CMP) : C supposé mis, insensibles au flag D — mais SBX prouve que CMP *force* temporairement D=0 en interne [1.734–737].
- Différences NMOS vs CMOS (65C00) à connaître pour ne pas se faire piéger par une doc CMOS : dummy read (adresse invalide vs dernier octet d'instruction), opcodes invalides (NOP sur CMOS), JMP (\$xxFF) corrigé sur CMOS (+1 cycle), RMW = 2 lectures/1 écriture sur CMOS, D initialisé au reset sur CMOS, flags décimaux valides (+1 cycle), BRK non détourné par interruption sur CMOS [64doc §Different CPU types, 1.826–858].

En pratique : la double écriture RMW (§3.3) et les lectures parasites indexées (§3.1–3.2) sont ce qui rend LSR \$D019 et LDA \$DCFD, X légitimes pour l'acquittement d'IRQ raster ; et la règle « -RDY ignoré pendant les écritures » (§1) est la raison des 3 cycles de garde avant tout DMA — pertinent pour le VIC-II (badlines) comme pour la REU.

Chapitre 4 — SID & CIA (datasheet 6581, modèle Lorenz)

Les deux compagnons du VIC : le SID (29 registres — et un générateur aléatoire gratuit sur l'oscillateur 3) et le CIA (timers au comportement subtil, et un registre d'interruption qui s'efface à la lecture). Le texte SID provient d'un OCR de scans : les réserves sont balisées au fil du texte.

Sources : datasheet MOS 6581 (scans zimmers.net 6581.zip, 12 pages, OCR 2 colonnes — **entrelacement de colonnes par endroits, valeurs douteuses signalées**) et « A Software Model of the CIA6526 » de Wolfgang Lorenz, v2.15, mai 1997 (Emulator Developers Kit, accompagne CIA6526.cpp/.h + TESTSUIT.ZIP). Les ancres renvoient à la page du datasheet ([SID p.N]) ou à la section/programme de test du document Lorenz ([Lorenz §...]).

1. SID 6581 (Sound Interface Device)

1.1 Vue d'ensemble

Synthétiseur 3 voix mono-puce, compatible famille 65XX. Chaque voix = oscillateur/générateur de forme d'onde + générateur d'enveloppe + modulateur d'amplitude ; un filtre programmable commun permet la synthèse soustractive. Le CPU peut lire la sortie de l'oscillateur 3 et de l'enveloppe 3 comme sources de modulation (vibrato, balayages) ou de nombres aléatoires. [SID p.2]

Caractéristiques annoncées [SID p.1] : 3 oscillateurs (plage 0–4 kHz) ; 4 formes d'onde par oscillateur (triangle, dent de scie, pulse variable, bruit) ; 3 modulateurs d'amplitude (dynamique 48 dB) ; 3 enveloppes exponentielles (Attack 2 ms–8 s, Decay 6 ms–24 s, Sustain 0–pic, Release 6 ms–24 s) ; synchronisation d'oscillateurs ; ring modulation ; filtre programmable (coupure 30 Hz–12 kHz, pente 12 dB/octave, sorties passe-bas/passe-bande/passe-haut/notch, résonance variable) ; volume maître ; 2 interfaces A/N pour potentiomètres ; générateur de nombres aléatoires ; entrée audio externe.

1.2 Les 29 registres (\$D400–\$D41C)

29 registres 8 bits, **écriture seule** sauf les 4 derniers (**lecture seule**) [SID p.3, Table 1]. Le datasheet donne les offsets \$00–\$1C ; sur C64 la base est \$D400. Les colonnes de bits de la Table 1 sont mutilées par l'OCR ; la décomposition ci-dessous est reconstruite depuis les descriptions de registres [SID p.4–6], cohérentes entre elles.

Adresse	Reg	Nom	D7 ... D0	Type
\$D400	00	Freq Lo V1	F7–F0	W
\$D401	01	Freq Hi V1	F15–F8	W
\$D402	02	PW Lo V1	PW7–PW0	W
\$D403	03	PW Hi V1	——, PW11–PW8 (b4–7 inutilisés)	W
\$D404	04	Control V1	NOISE·PULSE·SAW·TRI·TEST·RING·SYNC·GATE	W
\$D405	05	Attack/Decay V1	ATK3–0 · DCY3–0	W
\$D406	06	Sustain/Release V1	STN3–0 · RLS3–0	W
\$D407–\$D40D	07–0D	Voix 2	idem V1	W
\$D40E–\$D414	0E–14	Voix 3	idem V1	W
\$D415	15	FC Lo	——, FC2–FC0 (b3–7 inutilisés)	W
\$D416	16	FC Hi	FC10–FC3	W
\$D417	17	RES/Filt	RES3–0 · FILTEX·FILT3·FILT2·FILT1	W
\$D418	18	Mode/Vol	3OFF·HP·BP·LP · VOL3–0	W
\$D419	19	POTX	PX7–PX0	R
\$D41A	1A	POTY	PY7–PY0	R
\$D41B	1B	OSC3/Random	O7–O0	R
\$D41C	1C	ENV3	E7–E0	R

Les 3 emplacements restants (\$1D–\$1F) ne sont pas utilisés : écriture ignorée, lecture renvoie des données invalides [SID p.7].

1.3 Fréquence et largeur d'impulsion

- **Fréquence** (16 bits, linéaire) : $F_{out} = (F_n \times F_{clk} / 16777216) \text{ Hz}$; à $\varnothing 2 = 1,0 \text{ MHz}$: $F_{out} = F_n \times 0,0596 \text{ Hz}$. Résolution suffisante pour tout tempérament et pour un portamento sans marches audibles. [SID p.4]
- **Pulse width** (12 bits, b4–7 de PW Hi inutilisés) : $PW_{out} = (PW_n / 40,95) \%$. La forme pulse doit être sélectionnée pour que PW ait un effet. $PW_n = 0$ ou 4095 (\$FFF) → sortie continue (DC) ; $PW_n = 2048$ (\$800) → signal carré. Balayage sans marches audibles. [SID p.4]

1.4 Registre de contrôle : les 4 formes d'onde et les bits d'effet

Registre 04 (et 0B, 12) [SID p.4] :

Bit	Nom	Effet
0	GATE	1 = déclenche Attack/Decay/Sustain ; 0 = lance Release. Le GATE contrôle seul l'amplitude finale : il est inutile de désélectionner.
1	SYNC	Synchronisation « hard sync » de l'oscillateur sur l'autre voix : $V1 \leftarrow \text{Osc}3$, $V2 \leftarrow \text{Osc}1$, $V3 \leftarrow \text{Osc}2$. L'oscillateur source doit avoir un effet.
2	RING MOD	Remplace la sortie triangle par la combinaison ring-modulée : $V1 = \text{Osc } 1 \times 3$, $V2 = \text{Osc } 2 \times 1$, $V3 = \text{Osc } 3 \times 2$. Audible seulement si l'effet est sélectionné.
3	TEST	1 = remet à zéro et verrouille l'oscillateur ; sortie noise réinitialisée, pulse maintenu à un niveau DC. Sert au test, ou à synchroniser.
4	TRIANGLE	Pauvre en harmoniques, doux, « flûte ». [SID p.4]
5	SAWTOOTH	Riche en harmoniques paires et impaires, brillant, « cuivré ». [SID p.4]
6	PULSE	Contenu harmonique réglé par PW ; du carré creux au pulse nasal ; balayage temps réel = effet de phasing. [SID p.4]
7	NOISE	Signal aléatoire changeant à la fréquence de l'oscillateur : du grondement au souffle blanc (explosions, vent, caisses claires, cymbales).

Les formes d'onde ne sont PAS additives : en sélectionner plusieurs produit un ET logique. Si une autre forme est active en même temps que NOISE, la sortie bruit peut se **verrouiller** (« lock up ») et reste muette jusqu'à un reset par le bit TEST ou par RES (broche 5) bas. [SID p.5]

1.5 Enveloppe ADSR

- Attack (b4–7 de \$05) : temps de montée 0→pic ; Decay (b0–3) : temps pic→niveau sustain ; Sustain (b4–7 de \$06) : 16 niveaux **linéaires**, 0 = silence, 15 (\$F) = pic, 8 = mi-pic ; Release (b0–3) : chute sustain→0, **taux identiques au Decay**. [SID p.5]
- Les taux sont donnés pour $\varnothing 2 = 1,0 \text{ MHz}$; pour une autre horloge, multiplier par $1 \text{ MHz} / \varnothing 2$. [SID p.5]

Table 2 — taux d'enveloppe [SID p.5]. Note OCR : dans les lignes 9–15, l'OCR confond « S » et « 8 » ; les valeurs en secondes ci-dessous sont reconstruites par recoupement interne (page 1 : Attack max 8 s, Decay/Release max 24 s ; exemples p.11 : Release 9 = 750 ms, Release 3 = 72 ms, Attack 10 = 500 ms) — cohérent, mais à vérifier sur les scans.

Valeur	Attack (temps/cycle)	Decay/Release (temps/cycle)
0 (\$0)	2 ms	6 ms
1 (\$1)	8 ms	24 ms
2 (\$2)	16 ms	48 ms
3 (\$3)	24 ms	72 ms
4 (\$4)	38 ms	114 ms
5 (\$5)	56 ms	168 ms
6 (\$6)	68 ms	204 ms
7 (\$7)	80 ms	240 ms
8 (\$8)	100 ms	300 ms
9 (\$9)	250 ms	750 ms
10 (\$A)	500 ms	1,5 s (OCR incertain : « 500 nS 158 »)
11 (\$B)	800 ms	2,4 s (OCR « 248 »)
12 (\$C)	1 s	3 s
13 (\$D)	3 s (OCR « 38 »)	9 s
14 (\$E)	5 s (OCR « 58 »)	15 s
15 (\$F)	8 s	24 s (OCR « 2458 »)

Enveloppes types (annexe B du datasheet SID, valeurs du texte) [SID p.11] : violon/soutenu A=10 (\$A, 500 ms), D=8 (300 ms), S=10 (\$A), R=9 (750 ms) ; percussion/cymbale A=0 (2 ms), D=9 (750 ms), S=0, R=9

(750 ms) — décroît dès le pic quel que soit GATE ; piano/clavecin A=0, D=9 (750 ms), S=0, R=0 (6 ms) ; orgue A=0 (2 ms), D=0 (6 ms), S=15 (\$F), R=0 (6 ms) ; « backwards » A=10 (\$A, 500 ms), D=0 (6 ms), S=15 (\$F), R=3 (72 ms).

1.6 Filtre

- **FC Lo/FC Hi (\$15/\$16)** : nombre 11 bits linéaire (b3–7 de FC Lo inutilisés) = fréquence de coupure/centre. Plage annoncée : « environ 30 Hz à 10 kHz » avec les condensateurs recommandés de 2200 pF [SID p.5] — mais la page 1 et la description des broches disent « ~30 Hz–12 kHz » [SID p.1, p.7] : divergence telle qu'imprimée (l'OCR p.5 lit « 10KHz »).
- **RES/Filt (\$17)** : b4–7 = résonance, 16 réglages linéaires de 0 (aucune) à 15 (\$F, maximum) — accentue les composantes à la coupure, son plus « pointu ». b0–3 = routage vers le filtre : FILT1 (b0) voix 1, FILT2 (b1) voix 2, FILT3 (b2) voix 3, FILTEX (b3) entrée EXT IN (broche 26). Bit à 0 = la source va directement à la sortie audio, non filtrée. [SID p.6]
- **Mode/Vol (\$18)** : LP (b4) passe-bas 12 dB/oct (sons pleins) ; BP (b5) passe-bande, atténuation 6 dB/oct de part et d'autre (sons fins, ouverts) ; HP (b6) passe-haut 12 dB/oct (sons métalliques) ; 3 OFF (b7) = coupe la voix 3 du chemin audio direct — avec FILT3 = 0, la voix 3 sert de modulateur sans sortie parasite. **Les modes de filtre SONT additifs** : LP+HP = notch (réjecteur de bande). Il faut au moins un mode sélectionné ET au moins une source routée pour un effet audible. b0–3 = VOL, 16 pas linéaires, 0 = muet, 15 (\$F) = max ; un volume $\neq 0$ est obligatoire pour tout son ; utilisable en trémolo dynamique. [SID p.6]
- **Condensateurs** : C1 entre broches 1–2 (CAP1A/B), C2 entre 3–4 (CAP2A/B), valeurs identiques ; 2200 pF pour la plage audio ; polystyrène de préférence, appairés en multi-SID. Coupure maximale : $FC_{max} = 2,6 \cdot 10^{-5} / C$; la plage s'étend ~9 octaves sous ce maximum. Des C plus gros privilégient les basses (applications à coût réduit). [SID p.7]

1.7 Registres en lecture (\$19–\$1C)

- **POTX \$19 / POTY \$1A** : position du potentiomètre (0 à résistance min → 255/\$FF à résistance max), toujours valide, rafraîchie toutes les 512 périodes $\phi 2$. [SID p.6]
- **OSC3/RANDOM \$1B** : les 8 bits de poids fort de l'oscillateur 3, **jamais affecté par l'enveloppe**. Dent de scie → rampe 0→255 ; triangle → 0→255→0 ; pulse → saute entre 0 et 255 ; **noise** → **suite de nombres aléatoires** (générateur pour les jeux). Usage principal : générateur de modulation ajouté par logiciel aux registres de fréquence, de coupure ou de PW — sirène (scie→fréquence), sample & hold (noise→coupure filtre), vibrato (Osc3 $\approx$ 7 Hz, triangle mis à l'échelle→fréquence). Pendant ce temps, couper la voix 3 (3 OFF = 1 ; l'OCR imprime « 8 OFF = 4 »). [SID p.6–7]
- **ENV3 \$1C** : sortie du générateur d'enveloppe de la voix 3 ; **doit être « gâté » pour produire une sortie**. Ajouté à la coupure du filtre → enveloppes harmoniques, wah-wah ; ajouté à une fréquence d'oscillateur → sons « phaser ». [SID p.7]

1.8 Broches essentielles

[SID p.7–8]

- **CAP1A/1B (1, 2), CAP2A/2B (3, 4)** : condensateurs du filtre (§1.6).
- **RES (5)** : reset actif bas ; maintenu bas **au moins dix cycles $\phi 2$** → tous les registres à zéro, sortie audio coupée.
- **$\phi 2$ (6)** : horloge maître (nominal 1,0 MHz) ; toutes les fréquences et taux d'enveloppe y sont référencés ; les transferts de données n'ont lieu que $\phi 2$ haut (agit comme chip select actif haut).
- **R/W (7), CS (8)** : lecture si CS bas + $\phi 2$ haut + R/W haut ; écriture si CS bas + $\phi 2$ haut + R/W bas.
- **A0–A4 (broches 9–13 — ligne OCR mutilée)** : sélection d'un des 29 registres ; les 3 adresses restantes : écriture ignorée, lecture invalide.
- **GND (14)** : masse séparée des autres circuits numériques pour minimiser le bruit.
- **D0–D7 (15–22)** : bus bidirectionnel TTL, 2 charges TTL en sortie ; haute impédance hors lecture.
- **POTY (23), POTX (24)** : conversion A/N par constante de temps $R \cdot C = 4,7 \cdot 10^{-4}$ (R = résistance max du pot, C condensateur du pin vers la masse ; pot vers +5 V). Recommandé : **470 k Ω et 1000 pF** ; C plus gros = moins de gigue. Un pot + un condensateur par broche. (La p.6 imprime « POTY (pin 29) » — OCR douteux, la p.7 donne bien 24/23.)
- **Vcc (25)** : +5 V (ligne dédiée + condensateur de découplage).

- **EXT IN (26)** : entrée audio analogique, impédance $\sim 100\text{ k}\Omega$; le signal doit reposer sur 6 V DC et ne pas dépasser 3 V crête-à-crête ; couplage AC par électrolytique 1–10 μF . Chemin direct (FILTEX = 0) à gain unité → chaîne de plusieurs SID ; le volume maître affecte aussi les entrées externes.
- **AUDIO OUT (27)** : tampon « open-source » ; max $\sim 3\text{ V}$ crête-à-crête sur niveau DC 6 V ; **résistance de source vers la masse obligatoire** (1 $\text{k}\Omega$ recommandé) ; couplage AC 1–10 μF vers l'ampli.
- **Vdd (28)** : +12 V (ligne dédiée + découplage).

Valeurs limites absolues [SID p.8] : Vdd $-0,3\dots+17\text{ V}$; Vcc $-0,3\dots+7\text{ V}$; entrée analogique $-0,3\dots+17\text{ V}$; entrée numérique $-0,3\dots+7\text{ V}$; fonctionnement $0\dots+70\text{ }^\circ\text{C}$; stockage $-55\dots+150\text{ }^\circ\text{C}$. Les colonnes chiffrées des caractéristiques électriques (p.8) et des chronogrammes lecture/écriture (p.9) **n'ont pas survécu à l'OCR** — seuls les intitulés (Tacc, Twh, etc.) sont lisibles ; ne pas citer de valeurs.

1.9 Gamme tempérée complète — annexe A du datasheet SID ($\phi 2 = 1,0\text{ MHz}$, La4 = 440 Hz)

Fn = valeur 16 bits des registres de fréquence. † = colonnes décimal/hex incohérentes dans l'OCR. $\triangle$ Pour ces lignes, la colonne hex n'est **pas fiable telle quelle** (certaines cellules reproduisent l'impression OCR, d'autres une reconstruction) : se fier au **décimal**, consulter le « détail des † » en fin de table et, au moindre doute, recalculer depuis la fréquence (Fn = Hz $\div$ 0,0596). [SID p.10]

#	Note	Hz	Fn déc	Fn hex	#	Note	Hz	Fn déc	Fn hex
0	C0	16,35	274	\$0112	48	C4	261,63	4389	\$1125
1	C0#	17,32	291	\$0123	49	C4#	277,18	4650	\$122A
2	D0	18,35	308	\$0134	50	D4	293,66	4927	\$133F
3	D0#	19,44	326	\$0146	51	D4#	311,13	5220	\$146A
4	E0	20,60	346	\$015A	52	E4	329,63	5530	\$159A
5	F0	21,83	366	\$016E	53	F4	349,23	5859	\$16E3
6	F0#	23,12	388	\$0184	54	F4#	370,00	6207	\$183F
7	G0	24,50	414†	\$018B†	55	G4	392,00	6577	\$19B1†
8	G0#	25,96	435	\$01B3	56	G4#	415,30	6968	\$1B38
9	A0	27,50	461	\$01CD	57	A4	440,00	7382	\$1CD6
10	A0#	29,14	489	\$01E9	58	A4#	466,16	7821	\$1E8D†
11	B0	30,87	518	\$0206	59	B4	493,88	8286	\$205E
12	C1	32,70	549	\$0225	60	C5	523,25	8779	\$224B
13	C1#	34,65	581	\$0245	61	C5#	554,37	9301	\$2455
14	D1	36,71	616	\$0268	62	D5	587,33	9854	\$267E
15	D1#	38,89	652	\$028C	63	D5#	622,25	10440	\$28C8
16	E1	41,20	691	\$02B3	64	E5	659,25	11060	\$2B34
17	F1	43,65	732	\$02DC	65	F5	698,46	11718	\$2DC6
18	F1#	46,25	776	\$0308	66	F5#	740,00	12415	\$307F
19	G1	49,00	822	\$0336	67	G5	783,99	13153	\$3361
20	G1#	51,91	871	\$0367	68	G5#	830,61	13935	\$366F
21	A1	55,00	923	\$039B	69	A5	880,00	14764	\$39AC
22	A1#	58,27	978	\$03D2	70	A5#	932,33	15642	\$3D1A
23	B1	61,74	1036	\$040C	71	B5	987,77	16572	\$40BC
24	C2	65,41	1097	\$0449	72	C6	1046,50	17557	\$4495
25	C2#	69,30	1163	\$048B	73	C6#	1108,73	18601	\$48A9
26	D2	73,42	1232	\$04D0	74	D6	1174,66	19709	\$4CFC†
27	D2#	77,18†	1305	\$0519	75	D6#	1244,51	20897	\$518F†
28	E2	82,41	1383	\$0567	76	E6	1318,51	22121	\$5669
29	F2	87,31	1465	\$05B9	77	F6	1396,91	23436	\$5B8C
30	F2#	92,50	1552	\$0610	78	F6#	1479,98	24830	\$60FE
31	G2	98,00	1644	\$066C	79	G6	1567,98	26306	\$66C2†
32	G2#	103,83†	1742	\$06CE	80	G6#	1661,22	27871	\$6CDF
33	A2	110,00	1845	\$0735	81	A6	1760,00	29528	\$7358
34	A2#	116,54	1955	\$07A3	82	A6#	1864,65	31234†	\$7A34†
35	B2	123,47	2071	\$0817	83	B6	1975,53	33144	\$8178
36	C3	130,81	2195	\$0893	84	C7	2093,00	35115	\$892B
37	C3#	138,59	2325	\$0915	85	C7#	2217,46	37203	\$9153
38	D3	146,83	2463	\$099F	86	D7	2349,32	39415	\$99F7
39	D3#	155,56	2610	\$0A32	87	D7#	2489,01	41759	\$A31F
40	E3	164,81	2765	\$0ACD	88	E7	2637,02	44242	\$ACD2
41	F3	174,61	2930	\$0B72	89	F7	2793,83	46873	\$B719
42	F3#	185,00	3104	\$0C20	90	F7#	2959,95	49660	\$C1FC
43	G3	196,00	3288	\$0CD8†	91	G7	3135,96	52613	\$CD85†
44	G3#	207,65	3484	\$0D9C	92	G7#	3322,44	55741	\$D9BD†
45	A3	220,00	3691	\$0E6B	93	A7	3520,00	59056	\$E6B0
46	A3#	233,08	3910	\$0F46	94	A7#	3729,31	62567	\$F467
47	B3	246,94	4143	\$102F	95	B7	3951,06	*66288	*\$102F0†

Détail des † : #7 déc/hex incohérents tels qu'imprimés (414 $\neq$ \$018B) ; #27 « 77,18 » probablement 77,78 (D2#) ; #32 OCR « 403,83 » ; #43/55/58/74/79/91 hex OCR (« 0C08 », « 1981 », « 1E80 », « 4CFC », « 6602 », « CO85 ») incohérent avec le décimal ; #75 20897 = \$51A1 $\neq$ OCR \$518F ; #82 déc. probable 31284 (= \$7A34 = 2×15642) ; #92 hex illisible (« 0980 ») ; #95 hex OCR « 1F2FO ». B7 (astérisque) **dépasse 16 bits** : hors plage des oscillateurs, mais à garder dans la table pour le calcul (le MSB exige un cas logiciel spécial, p. ex. génération dans la retenue avant décalage). Astuce mémoire [SID p.10-11] : ne stocker que les 12 notes de l'octave 8 (24 octets au lieu de 192) et diviser par 2 (décalage à droite) par octave d'écart ; encodage 1 octet = quartet bas (semitone 0–11) + quartet haut (nombre de décalages).

1.10 Notes d'utilisation

Usage type d'une voix : régler fréquence, effets (SYNC, RING MOD), taux d'enveloppe, puis positionner GATE quand le son est voulu ; maintien libre, fin par GATE = 0. Les voix peuvent jouer à l'unisson (léger désaccord = son riche et animé). [SID p.5]

2. CIA 6526 — le modèle logiciel de Wolfgang Lorenz

2.1 Portée du modèle — à savoir avant de citer

Le document décrit le fonctionnement **interne** du 6526 du point de vue d'un développeur d'émulateur, validé par des programmes de la C64 Emulator Test Suite. **TOD (horloge temps réel) et SDR (registre série) n'y sont PAS implémentés** (« haven't been implemented yet ») ; les ports d'E/S généraux A/B et leurs DDR ne sont pas traités non plus — seule la sortie timer sur PB6/PB7 est modélisée. Ce chapitre ne peut donc rien affirmer de sourcé sur TOD, SDR, PRA/PRB/DDRA/DDRDB au-delà de ce qui suit. Seule mention du SDR : la sortie du flip-flop toggle « peut aussi servir au SDR — pas encore étudié ». [Lorenz §Intro, §Fig.4]

2.2 Sources d'horloge des timers

- **Timer A** : compte soit les horloges système $\varnothing 2$, soit les fronts montants de la ligne CNT ; sélection par **bit 5 du Control Register A (CRA, offset \$0E)** — masque $0x20 : 0x00 = \varnothing 2$, $0x20 = \text{CNT}$. En mode $\varnothing 2$, l'entrée vaut toujours 1 → décrémentation à chaque cycle. En mode CNT, le front est **synchronisé sur $\varnothing 2$ par un pipeline** : le front met à 1 le bit CountA0 de `dwDelay` ; à chaque cycle, `dwDelay` est décalé à gauche (rempli depuis `dwFeed`) ; le 1 traverse jusqu'à CountA3, qui décrémente alors le compteur. [Lorenz §Fig.1]
- **Timer B** : idem, plus le mode **cascadé** (compte les débordements du timer A) ; sélection par **bits 6–5 du CRB (offset \$0F)** — masque $0x60 : 0x00 = \varnothing 2$, $0x20 = \text{CNT}$, $0x40 = \text{débordements TA}$, $0x60 = \text{TA ET CNT}$. [Lorenz §Fig.2]
- **CNT est haut par défaut** (testé en cascade avec CRB = \$61). [Lorenz §CNTDEF]

2.3 Compteur, latch, démarrage, rechargement (subtilités de timing)

Chaque timer = compteur 16 bits + latch 16 bits. [Lorenz §Fig.3]

- **Démarrage** : le timer commence à compter **deux horloges après** l'écriture de 1 dans le bit 0 (start, $0x01$) du CRA.
- **Débordement** : arrivé à zéro, le compteur est rechargé depuis le latch **dès qu'une horloge attend dans le pipeline** — toujours vrai en mode $\varnothing 2$. D'où : **on ne lit jamais 0 en mode $\varnothing 2$** (séquence lue : 2-1-2), mais on lit des 0 en mode cascadé (2-2-2-1-1-1-0-0-2).
- **Rechargement** (par débordement ou par force load, bit 4 $0x10$ du CRA) : **l'horloge suivante est retirée du pipeline** → en mode $\varnothing 2$ on lit deux fois la valeur rechargée (2-1-2-2-1-2).
- **Écriture du latch** [Lorenz §LOADTH] : écrire l'octet **haut** charge le compteur **seulement si le timer est arrêté** ; timer en marche → pas de chargement ; écrire l'octet **bas** ne charge jamais (arrêté ou non).
- **Commutation $\varnothing 2 \leftrightarrow \text{CNT}$** : la bascule de source n'est reconnue qu'après un **délai de deux horloges**. [Lorenz §CNT02]

2.4 Mode one-shot

Bit 3 (0x08) du CRx. Si, au débordement, le bit one-shot est à 1 **ou l'était à l'horloge précédente**, le bit start (b0) est effacé ; comme CountA2 a déjà été vidé par le débordement, le timer s'arrête immédiatement. [Lorenz I.60–63] Cas limites mesurés (débordement à t) :

- **Basculer one-shot** [Lorenz §FLIPOS] : la mise à 1 prend effet dès t−1, la mise à 0 dès t−2 :

instant d'écriture	set →	clear →
t−2	stop	count
t−1	stop	stop
t	count	stop

- **Lecture du CRA autour du débordement** [Lorenz §ONESHOT] : à t−1 on lit \$09 (start encore 1), à t on lit \$08 (start déjà effacé).

2.5 Sortie timer sur PB6 (A) / PB7 (B)

- Activation par **bit 1** (0x02) du CRx (timer vs port). Sortie normale = **impulsion haute d'une horloge 02** au débordement. Mode **toggle** : PBx bascule à chaque débordement — le texte de Lorenz imprime « bit 3 », mais sa propre figure 4 et ses programmes de test utilisent le **bit 2** (0x04) (le bit 3 étant one-shot) : contradiction interne du document, la figure et les tests font foi. [Lorenz §Fig.4, §CIA1PB6-CIA2PB7]
- **Flip-flop toggle** : mis à 1 par un **front montant du bit start** (b0) du CRx, remis à zéro par le reset système. Résultats des tests [Lorenz §CIA1PB6-CIA2PB7] : le bit PBx résultant vaut 0 si le nouveau PBxToggle = 0, 1 si = 1, indéterminé si PBxOut = 0 ; **les anciennes valeurs, Start et Force Load n'influencent pas ce résultat**. PBx passe à 0 au premier débordement ; ni écrire CRx (hors bit 0) ni écrire Timer Hi/Lo ne le remet à 1 — **la seule source capable de remettre la ligne toggle à 1 est un front montant du bit start**. La ligne toggle est indépendante des bits PBxOut/PBxToggle (les changer n'empêche pas la bascule au débordement).

2.6 Interruptions : ICR/IMR — la lecture qui acquitte

[Lorenz §Fig.5, §ICR01, §IMR]

- Un débordement met à 1 le bit correspondant de l'**ICR** (0x01 = TA, 0x02 = TB ; 0x80 = drapeau IR).
- Si le bit correspondant de l'**IMR** (masque) est aussi à 1, le CIA lève l'interruption **avec un retard d'une horloge 02**.
- **L'interruption peut être empêchée en lisant l'ICR au moment exact du débordement**. Mesure [§ICR01] (débordement à t, CIA 2 → NMI) :

lecture à	valeur ICR lue	NMI exécutée ?
t−1	\$00	oui
t	\$01	non
t+1	\$81	oui

- Une fois le flip-flop d'interruption armé, **modifier l'IMR est sans effet** : si une condition est vraie dans l'ICR, mettre à 1 le bit d'IMR correspondant déclenche aussi l'interruption ; effacer le bit d'IMR **ne l'efface pas**. **Seule la lecture de l'ICR l'efface** (et efface l'ICR, y compris le bit \$80).

En clair — lire l'ICR, c'est le **vider** : l'information ne survit pas à sa consultation. Une lecture « de curiosité » de \$DC0D/\$DD0D peut avaler une interruption attendue ailleurs. On lit une fois, on dispatche ensuite depuis la copie.

2.7 Cas limites chiffrés (programmes de test)

- **Écriture de CRB, cycles 1–3 après le STA \$DD0F** [Lorenz §CIA1TB123/CIA2TB123] (mesuré en exécutant le STA en \$DD03 pour lire \$DD06 au cycle suivant ; #1 puis #2 = valeurs écrites dans \$DD0F) :

#1	#2	cycle 1	cycle 2	cycle 3	(4)
00	01	keep	keep	count	count
00	10	keep	load	keep	keep
00	11	keep	load	keep	count
01	11	count	load	keep	count
01	10	count	load	keep	keep
01	00	count	count	keep	keep

- **Mode cascadié complet** [Lorenz §CIA1TAB] : latches A et B à 2, TA sur ø2, TB sur débordements de TA, PB6 = impulsion TA, PB7 = toggle TB, IMR = \$02. Séquence cycle par cycle attendue (toute divergence = échec du test) :

```

TA  01 02 02 01 02 02 01 02 02 01 02 02
TB  02 02 02 01 01 01 00 00 02 02 02 02
PB  80 C0 80 80 C0 80 80 C0 00 00 40 00
ICR 00 01 01 01 01 01 01 01 03 83 83 83

```

Notes d'utilisation

En pratique sonore : GATE/ADSR et le volume \$D418 pilotent des effets courts ; OSC3 noise (\$D41B) est le générateur aléatoire matériel gratuit ; côté CIA, retenir surtout que **lire l'ICR acquitte tout** (un LDA \$DC0D mal placé peut avaler une IRQ, et une lecture au cycle du débordement la supprime), que le start d'un timer a 2 cycles de latence, et que l'octet haut du latch ne recharge le compteur que timer arrêté. Rappel : sur un C64 Ultimate au turbo, toute valeur de timing dérivée du 1 MHz nominal est à re-mesurer sur le vrai matériel.

Chapitre 5 — Le REU : DMA \$DF00

La RAM d'expansion et son contrôleur DMA : des déplacements d'octets à la vitesse du bus — jusqu'à un octet par cycle d'horloge (écran et sprites coupés), CPU gelé pendant le transfert. Table des registres verbatim, quatre commandes, huit pièges recensés.

Sources : texte de Richard Hable (zimmers.net programming.reu) → ancre [REU-prog] ; zimmers.net reu.registers → ancre [REU-reg] ; memory map du firmware Ultimate (doc officielle) → ancre [memmap].

5.1 Vue d'ensemble

Les REU Commodore (1700 = 128 Ko, 1764 = 256 Ko, 1750 = 512 Ko, extensibles à plusieurs Mo) fournissent de la RAM externe **non adressable directement** par le 6510 : le contrôleur intégré REC (*RAM Expansion Controller*) transfère les blocs entre la RAM C64 et la RAM d'expansion par DMA. Ses registres apparaissent mappés en I/O entre \$DF00 et \$DF0A quand une REU est branchée sur le port d'expansion, et se lisent/écrivent comme des registres VIC ou SID. [REU-prog]

5.2 Table des registres \$DF00–\$DF0A (verbatim, [REU-reg])

Address	Bits	Function
0		Status register - read only
	7	Interrupt Pending (1=interrupt waiting to be serviced)
	6	End of Block (1=transfer complete)
	5	Fault (1=block verify error)
	4	Size (1=256 kB)
	3-0	Version number
Note: Bits 7-5 are cleared when this register is read.		

Other registers are R/W:

1		Command Register
	7	Execute (1=initiate transfer per current config)
	6	reserved (left 1 in an example program in the manual)
	5	Load (1=enable AUToload option)
	4	FF00 (1=disable FF00 decode)
	3-2	reserved (left 1)
	1-0	Transfer type: 00=C64->REU 01=REU->C64 10=swap 11=verify
2	7-0	C64 start address (LSB)
3	7-0	C64 start address (MSB)
(address overflow is not detected; it will continue from \$0000)		
4	7-0	REU start address (LSB)
5	7-0	REU start address (More SB)
6	2-0	REU start address (most significant bits)
7	7-0	Transfer length (LSB) (\$0000=64 kB)
8	7-0	Transfer length (MSB)
9		Interrupt Mask Register
	7	Interrupt enable (1=interrupts enabled)
	6	End of Block mask (1=interrupt on end of block)
	5	Verify error (1=interrupt on verify error)
	4-0	unused (left 0 in an example program)
A	7-6	Address Control Register
		00=increment both addresses 01=fix expansion address 10=fix C64 address 11=fix both addresses

Précisions croisées [REU-prog] : dans \$DF00, le bit 5 FAULT est posé si une différence C64/REU est trouvée pendant un *compare* ; le bit 4 SIZE est posé sur 1764/1750, effacé sur 1700 ; bits 3..0 = version (0 sur la REU de l'auteur). \$DF04..\$DF06 = adresse REU sur trois octets (lo, hi, banque) ; normalement seuls les **bits 2..0 de la banque** sont valides (max 512 Ko), les autres bits lisent toujours 1. Labels d'assembleur suggérés : status=\$DF00, command=\$DF01, c64base=\$DF02, reubase=\$DF04, translen=\$DF07, irqmask=\$DF09, control=\$DF0A.

5.3 Les quatre commandes (bits 1..0 de \$DF01)

- **%00 stash (C64 → REU) et %01 fetch (REU → C64)** : poser les adresses de base, la longueur, puis écrire le registre de commande. Exemple canonique [REU-prog] : `lda #10010000 / sta command = C64→REU` avec exécution immédiate ; `#10010001` pour le retour REU→C64.
- **%10 swap (C64 ↔ REU)** : échange les deux zones ; même programmation que le transfert simple. **Deux fois plus lent** qu'un stash/fetch (deux accès mémoire C64 par octet, lecture + écriture). Sert à simuler du bank switching (ex. RAMDOS : 256 octets résidents, driver de 6 Ko swappé à la demande). [REU-prog]
- **%11 verify (compare)** : rien n'est transféré ; les octets sont comparés et le bit FAULT (\$DF00 bit 5) est posé à la première différence. **Lire \$DF00 avant de comparer** pour effacer un FAULT résiduel et obtenir une info valide. La comparaison s'arrête à la première différence trouvée ; le REC stoppe le DMA et les registres d'adresses + banque pointent alors **une position au-dessus** de l'adresse fautive — sauf si AUTOLOAD est activé, auquel cas cette adresse d'erreur est **perdue**. [REU-prog][REU-reg]

Exécution immédiate vs \$FF00 (\$DF01 bit 4) : bit 4 à 1 = la commande part **dès l'écriture** dans \$DF01 ; bit 4 à 0 = l'exécution est différée jusqu'à une **écriture quelconque à \$FF00**. L'intérêt : la RAM C64 est vue par le REC dans la configuration mémoire **au moment de l'écriture du registre de commande** — or il faut l'I/O active pour écrire \$DF01. Pour transférer la RAM sous \$D000–\$DFFF ou la ROM caractères, on arme la commande avec FF00=0, on bascule \$01 sur RAM 64 Ko, puis `lda $FF00 / sta $FF00` (relire d'abord pour ne pas altérer le contenu de \$FF00) déclenche le DMA, avant de restaurer \$01. [REU-prog] L'option FF00 est **auto-effacée à chaque usage**. [REU-reg]

En clair — le DMA voit la carte mémoire au moment où il **s'exécute**. Le déclencheur \$FF00 sert exactement à découpler les deux : on arme pendant que l'I/O est visible (il le faut, pour écrire \$DF01), on bascule \$01, et on tire pendant que la RAM l'est — c'est ainsi qu'on copie la RAM cachée sous les registres.

Autoload (\$DF01 bit 5) : à 1, les registres d'adresse C64, d'adresse REU (banque incluse) et le compteur d'octets sont **rechargés automatiquement** en fin de transfert — utile pour répéter l'opération sur le même bloc. À 0 (fonctionnement normal), les deux pointeurs finissent sur **la première position hors de la plage transférée** (dernier octet + 1, banque comprise, sauf pointeur fixé), et le compteur d'octets **décrompte jusqu'à 1** (pas 0) : attention en testant la « fin de transfert » par le compteur, car une longueur de **0 = 64 Ko pleins**. [REU-prog][REU-reg]

Adresses : wrap et fixation. Longueur \$0000 = 64 Ko. Si C64 base + longueur dépasse 64 Ko, l'adresse C64 **wrappe et repart de \$0000** (overflow non détecté) ; si REU base + longueur dépasse 512 Ko, l'adresse REU wrappe à 0. Le wrap se produit **dans tous les modes**. La « banque » \$DF06 est en réalité la partie haute de l'adresse : elle **s'incrémente** au franchissement d'une frontière de 64 Ko ($2^{19} = 0,5$ Mo max sur silicium d'origine ; les bits hauts inutilisés permettraient $2^{24} = 16$ Mo à un clone — c'est exactement le cas de la REU 16 Mo de l'U64). \$DF0A fige au choix l'adresse C64 (bit 7) et/ou REU (bit 6) : une adresse fixée fait porter tout le DMA sur **le même octet**. Applications citées : REU fixée = remplissage ultra-rapide d'une zone C64 avec un octet (ou détection de fin de zone d'octets égaux pour la compression) ; C64 fixée = sortie rapide vers un port I/O, « magic byte » VIC pour du bitmap dans les bordures, échantillonnage ~1 MHz. [REU-prog][REU-reg]

IRQ (\$DF09) : bit 7 valide les interruptions, bit 6 = IRQ en fin de bloc, bit 5 = IRQ sur erreur de verify. Les deux docs concordent : c'est **inutile en pratique**, car le CPU est gelé pendant le DMA — le transfert est toujours terminé dès l'instruction suivant le `sta command` (ou le `store à $FF00`) ; l'IRQ tomberait juste après. Pas besoin non plus de scruter END OF BLOCK. [REU-prog][REU-reg]

Vitesse : CPU halté, 1 octet par cycle d'horloge (985 248/s en PAL) écran et sprites coupés ; un peu moins écran allumé (cycles volés par le VIC). Compare = même vitesse (arrêt à la première différence) ; swap = moitié. Copie C64→C64 via la REU (aller-retour, méthode GEOS) ≈ 5× plus rapide qu'une copie en langage

machine. Le code en RAM externe ne s'exécute pas sur place : toujours le recopier (ou le swapper) en RAM C64. [REU-prog]

5.4 Pièges de programmation recensés dans les docs

1. **Lire \$DF00 efface les bits 7-5** (Interrupt Pending, End of Block, Fault) — lecture destructive. Corollaire : lire le statut **avant** un verify, et, si IRQ utilisées, lire \$DF00 au moins une fois **entre deux transferts** successifs. [REU-reg]
2. **Longueur 0 = 64 Ko**, pas « rien ». [REU-prog][REU-reg]
3. **Wrap silencieux** des adresses C64 (à 64 Ko) et REU (à 512 Ko), non détecté, dans tous les modes. [REU-prog][REU-reg]
4. **Compteur final = 1** (pas 0) en fin de transfert normal — prudence si on teste le compteur. [REU-reg]
5. **AUTOLOAD + verify = adresse d'erreur perdue** (les pointeurs sont rechargés au lieu de pointer après l'octet fautif). [REU-reg]
6. **Configuration mémoire figée à l'armement** : le DMA voit la carte mémoire du moment de l'écriture de \$DF01 ; passer par le déclenchement \$FF00 (avec `lda $FF00` avant `sta $FF00` pour préserver l'octet, et sous `sei`) pour atteindre la RAM sous I/O/ROM. [REU-prog]
7. **Bits réservés incohérents entre sources** : \$DF01 bits 6 et 3-2 « normalement 0 » selon [REU-prog], « laissés à 1 dans un exemple du manuel » selon [REU-reg] ; \$DF09 bits 4-0 « normalement tous à 1 » [REU-prog] vs « laissés à 0 » [REU-reg]. Les bits hauts de \$DF06 lisent toujours 1 sur une vraie REU ≤ 512 Ko.
8. **Détection de la REU** : écrire 1,2,3... dans \$DF02..\$DF08 et relire (si ça ne reste pas → pas de REU, mais un autre module peut répondre à ces adresses) ; le bit SIZE ne dit que « ≥ 2 ou ≥ 4 banques », la taille réelle se sonde banque par banque en écriture+verify. [REU-prog]

5.5 Memmap firmware Ultimate-II/U64 en bref [memmap]

Côté firmware (adresses **internes** au SoC, pas côté C64) : 32 Mo de RAM à 00000000-01FFFFFF (U2 ; le double sur U2+/U64 avec 02000000-03FFFFFF), I/O à 04000000 (bloc commun `ultimate_logic_32`) et A0000000 (spécifique U2+/U64). Points utiles en pratique :

- **01000000-01FFFFFF = 16 Mo « REU / GeoRAM space »** : la REU émulée est un bloc dédié de 16 Mo dans la RAM du firmware — cohérent avec la remarque de [REU-reg] sur les clones adressant 2^{24} octets, et avec une REU configurée à 16 Mo.
- **02000000-02FFFFFF = 16 Mo « RAM Disk (/temp) »** (U2+/U64) : le RAM-disk /Temp où s'accumulent les fichiers temporaires d'upload est un bloc fixe de 16 Mo — d'où sa saturation possible.
- Autour : ~14,6 Mo d'espace application (code/tas/pile du firmware), espaces Kernal de remplacement, échantillons son et CPU des drives A/B, 00EF0000 = 64 Ko RAM cartouche, 00F00000 = 1 Mo ROM cartouche, 03000000 réservé au chargement de l'updater ; sur U64 seul, 08000000-0BFFFFFF = 64 Mo réservés non implémentés. Sur U2 (sans le bloc dédié), le RAM Disk est pris sur le tas de l'application et limité à 3 Mo.

Chapitre 6 — L'Ultimate 64 : services (REST, streams, turbo, UCI)

La machine moderne : ce que l'Ultimate 64 construit autour du C64 — API REST, flux vidéo et audio temps réel, turbo réglable, et l'interface UCI qui ouvre le système de fichiers au 6510. Documentation officielle condensée ; sur un firmware dérivé, tout point critique mérite re-vérification sur cible (l'annexe A en donne un exemple complet).

Ce chapitre synthétise la documentation officielle Ultimate. Ces textes décrivent le firmware Ultimate « amont » (Gideon) ; sur une machine à firmware dérivé (p. ex. Commodore 64 Ultimate, fork réduit), chaque service doit être vérifié sur le matériel avant usage.

Sources (documentation officielle Ultimate, dépôt GideonZ/1541u-documentation — noms de fichiers amont) : `api/api_calls.rst`, `data_streams.rst`, `config/turbo_mode.rst`, `ultimate64.rst`, `uci/core_uci_architecture.rst`, `uci/ultimate_dos_target.rst`, `uci/control_target.rst`.

1. Organisation de la documentation [ultimate64.rst]

`ultimate64.rst` est la page d'accueil (table des matières) de la doc Ultimate-64 : Getting Started, Power Button, User Interface, Configuration, Hardware, **Data Streams**, SID player, How-to. L'application qui gère l'Ultimate-64 est en essence la même que celle de la cartouche Ultimate-II+ (mêmes fonctions).

2. API REST [api_calls.rst]

Disponible à partir du firmware **3.11**. Format d'URL :

`/v1/<route>/<path>:<command>?<arguments>`

Verbes : **GET** = lecture sans changement d'état ; **PUT** = action avec informations dans l'URL (ou fichier référencé par l'URL) ; **POST** = action avec données attachées à la requête. La plupart des réponses sont en JSON (Content-Type: `application/json`) et contiennent toujours au moins le tableau `errors` (liste des erreurs, vide si succès).

Mot de passe réseau (firmware ≥ 3.12) : si un « Network Password » est configuré, toute requête doit inclure l'en-tête HTTP `X-Password: <mot-de-passe>` ; sinon 403 Forbidden. L'en-tête est ignoré si aucun mot de passe n'est défini.

2.1 About

Route	Params	Action / Réponse
GET /v1/version	—	Version de l'API REST : <code>{"version": "0.1", "errors": []}</code>
GET /v1/info	—	Infos machine : <code>product</code> , <code>firmware_version</code> , <code>fpga_version</code> , <code>core_version</code> (U64 uniquement), <code>hostname</code> , <code>unique_id</code> (sauf si désactivé dans Network Settings)

2.2 Runners

Route	Params	Action
PUT /v1/runners:sidplay	<code>file</code> , [<code>songnr</code>]	Joue un fichier SID présent sur le FS de l'Ultimate (chanson par défaut sauf <code>songnr</code>) ; cherche les durées dans le sous-dossier <code>SONGLNGTHS</code>
POST /v1/runners:sidplay	[<code>songnr</code>]	Joue le SID attaché à la requête ; 2 ^e pièce jointe optionnelle = durées
PUT /v1/runners:modplay	<code>file</code>	Joue un MOD Amiga présent sur le FS

Route	Params	Action
POST /v1/runners:modplay	—	Joue le MOD attaché
PUT /v1/runners:load_prg	file	Reset machine + charge le programme en mémoire par DMA. Ne lance pas le programme
POST /v1/runners:load_prg	—	Idem avec le programme attaché
PUT /v1/runners:run_prg	file	Reset + chargement DMA + lance le programme
POST /v1/runners:run_prg	—	Idem avec le programme attaché
PUT /v1/runners:run_crt	file	Reset avec la cartouche indiquée active ; ne modifie pas la configuration
POST /v1/runners:run_crt	—	Idem avec le fichier .crt attaché

2.3 Configuration

Route	Params	Action
GET /v1/configs	—	Liste toutes les catégories de configuration (Audio Mixer, SID Sockets Configuration, U64 Specific Settings, Clock Settings, Network settings, Data Streams, Drive A/B Settings...)
GET /v1/configs/<category>	—	Tous les items d'une catégorie ; jokers autorisés ; profondeur 1. Ex. GET /v1/configs/drive%20a*
GET /v1/configs/<category>/<item>	—	Détail d'item(s) : current, min, max, format, default. Jokers autorisés ; profondeur 2
PUT /v1/configs/<category>/<item>	value	Écrit une valeur. Ex. PUT /v1/configs/drive%20a*/bus*?value=9
POST /v1/configs	corps JSON	Change plusieurs réglages d'un coup ; JSON à 2 niveaux : catégorie → item → valeur
PUT /v1/configs:load_from_flash	—	Recharge la configuration sauvee en mémoire non volatile
PUT /v1/configs:save_to_flash	—	Écrit la configuration courante en flash (chargée au boot) —  sur le firmware Commodore 1.0, une configuration réseau/stream sauvee en flash peut réveiller un bug de démarrage (écran noir persistant) ; récupération : maintenir RESTORE à l'allumage (<i>comportement observé sur un exemplaire, juillet 2026 — non documenté officiellement ; détail en annexe A</i>)
PUT /v1/configs:reset_to_default	—	Réglages courants → défauts usine (ne touche pas la flash)

2.4 Machine

Route	Params	Action
PUT /v1/machine:reset	—	Reset de la machine, configuration inchangée
PUT /v1/machine:reboot	—	Redémarre : ré-initialise la configuration cartouche + reset
PUT /v1/machine:pause	—	Pause : ligne DMA tirée basse à un moment sûr (CPU stoppé, timers non stoppés)
PUT /v1/machine:resume	—	Relâche la ligne DMA, le CPU reprend
PUT /v1/machine:poweroff	—	U64 uniquement : éteint la machine (réponse probablement non reçue)
PUT /v1/machine:menu_button	—	Équivaut au bouton Menu (cartouche) / appui bref Multi Button (U64) : entre/sort du menu Ultimate
PUT /v1/machine:writemem	address, data	Écrit en mémoire C64 par DMA (map mémoire courante ; registres I/O du 6510 inaccessibles). address en hexa, data = chaîne d'octets hexa, max 128 octets . Ex. ?address=D020&data=0504 → \$05 dans \$D020, \$04 dans \$D021
POST /v1/machine:writemem	address	Idem avec données en pièce jointe binaire ; ne doit pas déborder \$FFFF
GET /v1/machine:readmem	address, [length]	Lecture DMA sur le bus cartouche, résultat en binaire ; 256 octets par défaut
GET /v1/machine:debugreg	—	Lit le registre debug \$D7FF , renvoie value en hexa (U64 uniquement)
PUT /v1/machine:debugreg	value	Écrit value (hexa) dans \$D7FF puis le relit et renvoie value (U64 uniquement)

2.5 Lecteurs de disquette

Route	Params	Action
GET /v1/drives	—	État de tous les lecteurs IEC internes : enabled, bus_id, type, rom, image_file, image_path ; entrée softiec avec last_error et partitions
PUT /v1/drives/<drive>:mount	image, [type], [mode]	Monte une image du FS de l'Ultimate. type : d64 , g64 , d71 , g71 , d81 (défaut = extension). mode : readwrite , readonly , unlinked (écriture autorisée mais non répercutée dans l'image)
POST /v1/drives/<drive>:mount	[type], [mode]	Idem avec l'image en pièce jointe (nom via Content-Deposition (<i>sic</i>))
PUT /v1/drives/<drive>:reset	—	Reset du lecteur
PUT /v1/drives/<drive>:remove	—	Retire la disquette montée. <i>NB : la source liste : remove deux fois ; la seconde entrée décrit la rupture du lien image↔lecteur (écritures non répercutées ensuite)</i>
PUT /v1/drives/<drive>:on	—	Allume le lecteur (reset s'il était déjà allumé)
PUT /v1/drives/<drive>:off	—	Éteint le lecteur (disparaît du bus série)
PUT /v1/drives/<drive>:load_rom	file	Charge une ROM lecteur (16 K ou 32 K selon type) depuis le FS ; temporaire (perdue au changement de type ou reboot)
POST /v1/drives/<drive>:load_rom	—	Idem, ROM en pièce jointe
PUT /v1/drives/<drive>:set_mode	mode	Change le mode du lecteur : 1541 , 1571 , 1581 ; recharge la ROM par défaut

2.6 Data Streams (U64 uniquement)

Route	Params	Action
PUT /v1/streams/<stream name>:start	ip	Démarre un flux. Noms valides : video , audio , debug . Ports par défaut : 11000 (video), 11001 (audio), 11002 (debug). Port personnalisé après deux-points : 192.168.178.224:6789.
PUT /v1/streams/<stream name>:stop	—	Démarrer la vidéo coupe automatiquement le flux debug Arrête le flux (video , audio , debug)

2.7 Manipulation de fichiers (état V3.11 alpha, inachevé)

Route	Params	Action
GET /v1/files/<path>:info	—	fstat du fichier (taille, extension). Jokers supportés. <i>Unfinished</i>
PUT /v1/files/<path>:create_d64	[tracks], [diskname]	Crée un .d64 (chemin complet depuis la racine). 35 pistes par défaut, 40 possible ; diskname = nom d'en-tête (sinon nom de fichier)
PUT /v1/files/<path>:create_d71	[diskname]	Crée un .d71 (70 pistes fixes)
PUT /v1/files/<path>:create_d81	[diskname]	Crée un .d81 (160 pistes, 80 par face)
PUT /v1/files/<path>:create_dnp	tracks, [diskname]	Crée un .dnp ; tracks obligatoire, 256 secteurs/piste, max 255 pistes (≈16 Mo)

3. Flux de données réseau [data_streams.rst]

L'Ultimate-64 streamer des données temps réel sur son port Ethernet, en **UDP** (sans connexion, pas de garantie de livraison). Trois modes d'adressage : **unicast** (une IP), **multicast** (224.0.0.0–239.255.255.255, le récepteur « joint » le groupe via IGMP ; nécessite un switch avec IGMP snooping, sinon comportement broadcast), **broadcast**. En multicast le tri se fait par adresse IP, pas par port UDP : utiliser des IP multicast différentes pour plusieurs flux.

3.1 Démarrage / arrêt

Trois moyens :

1. **Menu** : « action menu » (touche **F5**) → options start/stop ; l'IP par défaut vient de l'écran de config « Data Streams » (qui ne démarre rien par lui-même). Destination = IP ou nom DNS, port optionnel après : (ex. 192.168.0.119:11000, 239.0.2.77:64738).

2. **API REST** : PUT /v1/streams/<nom>:start?ip=... / :stop (cf. §2.6).

3. **Interface de commande TCP** (socket TCP « 64 ») :

- Activer un flux : mot de commande **FF2n** ; désactiver : **FF3n** (n = numéro de flux).
- Structure générique : <mot de commande, little-endian> <longueur des paramètres, little-endian> <paramètres>.
- Paramètres d'activation (optionnels) : (1) durée en ticks système de **5 ms** (0 = infini) ; (2) destination (chaîne).
- Exemples verbatim :
 - Activer le flux 0 pendant 1 s (200 ticks) vers la destination pré-configurée : 20 FF 02 00 00 C8
 - Activer le flux 0 indéfiniment vers 192.168.0.119 : 20 FF 0F 00 00 00 192.168.0.119 (fin en ASCII ; longueur de commande = longueur de chaîne + 2)
 - Désactiver le flux 0 : 30 FF 00 00 (longueur de paramètres 0)

3.2 Flux vidéo VIC (ID 0)

Partie active de la sortie VIC. Chaque datagramme UDP = **en-tête 12 octets** + pixels :

Champ	Taille	Valeur
Numéro de séquence	16 bits LE	incrémental
Numéro de trame	16 bits LE	
Numéro de ligne	16 bits LE	bit 15 mis = dernier paquet de la trame
Pixels par ligne	16 bits LE	toujours 384
Lignes par paquet	8 bits	toujours 4
Bits par pixel	8 bits	toujours 4
Type d'encodage	16 bits	toujours 0 (« 1 » réservé pour un futur RLE)

Exemple de paquet (trame 3, lignes 100–103, début bleu foncé) :

```
A1 00 02 00 64 00 80 01 04 04 00 00 66 66 66 66 66 66 ...
seq #|frm #|line#|width|lp|bp|-enc-|data.....
```

Pixels : couleurs VIC 4 bits, **little-endian**, **nibble 3..0 en premier**. 4 lignes × 384 pixels = 768 octets ; datagramme total = **780 octets**. Pour capturer une trame : attendre un paquet avec bit 15 du numéro de ligne, puis capturer jusqu'au suivant → **68 paquets = 272 lignes**. Vidéo rognée à **384 × 272** (alignée sur les images de référence VICE ; sortie écran réelle 400 × 288 en PAL). En NTSC : flux **384 × 240** (écran 400 × 240).

3.3 Flux audio (ID 1)

Pris à la sortie du mixeur audio (identique au HDMI / codec analogique). En-tête = **2 octets** seulement : numéro de séquence. Suivent **192 échantillons stéréo** en **16 bits signé little-endian**, gauche/droite entrelacés, **gauche d'abord**. Paquet UDP total = **770 octets** (2 + 192 × 4) :

```
00 00 fe ff 02 00 ff ff 03 00 fd ff 00 00 ...
-seq- left--right-left--right-left--right--
```

Fréquence d'échantillonnage ≈ 48 kHz, dépend du mode vidéo :

- **PAL** : $(F_c \times 16/9 \times 15 / 77 / 32) = 47\,983\text{ Hz}$ ($F_c = 4\,433\,618,75\text{ Hz}$; −356 ppm vs 48 kHz)
- **NTSC** : $(F_c \times 16/7 \times 15 / 80 / 32) = 47\,940\text{ Hz}$ ($F_c = 3\,579\,545,45\text{ Hz}$; −1243 ppm)

3.4 Flux debug (ID 2)

Fonction avancée (firmware ≥ 3.7 / V1.28) : trace **temps réel, au cycle près**, des accès bus du 6510, du VIC ou du CPU 1541. Modes : **6510 Only**, **VIC Only**, **6510 & VIC**, **1541 Only**, **6510 & 1541**. Chaque flux ≈ **32 Mbps** : incompatible avec le flux vidéo (port 100 Mbps), et les trois sources ne peuvent pas être combinées.

Paquet : premier mot 32 bits = numéro de séquence 16 bits + 16 bits réservés, puis **360 entrées de 32 bits** → charge utile **1444 octets** :

```
05 16 00 00 xx xx xx xx yy yy yy yy ...
-seq- resvd |- entry 0 -|- entry 1 -|
```

Format d'une entrée 32 bits :

bit	31	30	29	28	27	26	25	24	23..16	15..0
6510 / VIC	PHI2	GAME#	EXROM#	BA	IRQ#	ROM#	NMI#	R/W#	Data	Address
1541	'0'	ATN	DATA	CLOCK	SYNC	BYTE_READY	IRQ#	R/W#	Data	Address

Les formats 6510 et VIC sont identiques ; la distinction CPU/VIC se fait sur AEC et PHI2 (le bit 31 reflète littéralement PHI2 pendant l'accès). Le flux 1541 inclut ATN/CLOCK/DATA du bus IEC. Outils : `grab_debug.py` (dépôt GideonZ/1541ultimate), `dump_bus_trace.c` → fichier VCD (GtkWave). Visionneuses vidéo : TSB U64 Streamer (Windows), u64view (Linux/Mac), scripts `grab.py` / `grab_audio.py`.

4. Mode turbo [`turbo_mode.rst`]

4.1 Réglages menu

- **Turbo control** : 0ff (pas de turbo) ; Manual (réglages U64, **non** modifiables par registres) ; U64 Turbo Registers (modifiables par registres) ; TurboEnable Bit (modifiables par registres).
- **CPU Speed** : défaut **1 MHz (1×)** ; max **48 MHz (48×)** sur U64, **64 MHz (64×)** sur U64 Elite-II.
- **Badline Timing** : activer/désactiver le timing badline (désactiver = plus de cycles CPU) ; contrôlable par registre.
- **SuperCPU Detect (\$D0BC)** : registre de détection compatible SuperCPU.

4.2 Registres de contrôle turbo

Adresse	Registre	Détail
53297 / \$D031	U64 Turbo Control	Disponible seulement si Turbo Mode = « U64 Turbo Registers » ou « Turbo Enable Bit », sinon lit \$FF. bits 0–3 : index de vitesse CPU ; bit 7 : timing badline, 0 = activé, 1 = désactivé
53296 / \$D030	Turbo Enable Bit	Disponible seulement en mode « Turbo Enable Bit », sinon lit \$FF. bit 0 en écriture : 0 = 1 MHz + badlines, 1 = réglages du menu. bit 0 en lecture : 0 = turbo off, 1 = turbo on
53370 / \$D07A	SuperCPU compatible	Écriture seule ; Software Speed Select – Normal . Disponible dans les deux modes registre
53371 / \$D07B	SuperCPU compatible	Écriture seule ; Software Speed Select – Turbo (20 MHz) (\$079 (<i>sic</i>)). Disponible dans les deux modes registre
53436 / \$D0BC	SuperCPU Detect	Lecture seule ; activé séparément dans la config

Table des index de vitesse (\$D031 bits 0–3) :

Index	U64	U64E2	Index	U64	U64E2
0	1 MHz	1 MHz	8	12 MHz	14 MHz
1	2 MHz	2 MHz	9	14 MHz	16 MHz
2	3 MHz	3 MHz	10	16 MHz	20 MHz
3	4 MHz	4 MHz	11	20 MHz	24 MHz
4	5 MHz	6 MHz	12	24 MHz	32 MHz
5	6 MHz	8 MHz	13	32 MHz	40 MHz
6	8 MHz	10 MHz	14	40 MHz	48 MHz
7	10 MHz	12 MHz	15	48 MHz	64 MHz

CPU et mémoire tournent en permanence à 48 MHz (U64) / 64 MHz (U64E2) ; l'index détermine combien de créneaux temporels sont attribués au CPU. Le VIC vole toujours des cycles quand il est actif ; le réglage badline n'affecte **pas** le signal BA du port cartouche (accès externes — dont les sockets SID — restent soumis aux limitations badline).

4.3 Badlines en turbo (notes)

Le VIC a toujours priorité, quel que soit le réglage badline (contrairement au C128 en 2 MHz, le VIC affiche correctement en turbo). Sur C64 classique : 2 créneaux par MHz (PHI2=0 pour le VIC, PHI2=1 pour le CPU), et le DMA VIC (sprites + lecture des 40 octets caractère toutes les 8 lignes) bloque le CPU ~43 µs — d'où « bad lines ». Quand le VIC réclame un cycle CPU, **BA** passe bas. Sur U64 : 48 créneaux par MHz (64 sur C64U). Tout accès **interne** peut se faire dans n'importe quel créneau ; un accès au bus externe passe par le pont cartouche, toujours à 1 MHz, et seulement quand BA=1. Le réglage badline fait une seule chose : badlines **activées** = CPU gelé pendant BA=0 (timing compatible C64) ; **désactivées** = le CPU continue pendant BA=0 si l'accès est interne. (S'applique à Ultimate 64 et Elite-II ; certains réglages requièrent firmware ≥ 1.33.)

5. UCI — Ultimate Command Interface [core_uci_architecture.rst]

Interface de commande programmable depuis le C64 vers l'application de gestion Ultimate, via les registres I/O du port cartouche. Les « cibles » (targets) incluent Ultimate DOS, Network, Module Control, SoftIEC Bypass, client HTTP (firmware 3.15).

5.1 Registres (\$DF1B–\$DF1F, masquent les 5 derniers registres du REU — formulation de la source qui suppose que l'émulation REU du U64 décode des MIROIRS au-delà de \$DF0A [ch.5] ; à vérifier sur la machine cible — ; mapping optionnel, à activer dans le menu « Command Interface »)

Adresse	Accès	Rôle	Défaut
\$DF1B	R	SoftwareIEC Bus ID	
\$DF1C	W	Registre de contrôle	
\$DF1C	R	Registre d'état	\$00
\$DF1D	W	Registre de données de commande	
\$DF1D	R	Registre d'identification	\$C9
\$DF1E	R	Registre de données de réponse	
\$DF1F	R	Registre de données d'état (status)	

5.2 Registre de contrôle (\$DF1C en écriture)

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
DMA	TRIGGER	IRQ	réservé	CLR_ERR	ABORT	DATA_ACC	PUSH_CMD

- **PUSH_CMD** : pousse la commande écrite dans \$DF1D vers le logiciel Ultimate.
- **DATA_ACC** : signale que toutes les données ont été acceptées ; vide et réinitialise les files de réponse/status. Ignoré hors des états de données.
- **ABORT** : lève le drapeau abort (pollé par l'Ultimate, qui ramène la machine d'états à idle).
- **CLR_ERR** : efface le drapeau d'erreur d'état (commande poussée hors idle).
- **IRQ** (≥ V3.15) : active l'interruption de fin de commande ; auto-effacé à la lecture des files ou au DATA_ACC. IRQ actif → l'identification lit \$49 au lieu de \$C9 (bit 7 effacé).
- **TRIGGER** : si mis lors du push, l'Ultimate passe en mode DMA dès qu'une écriture sur \$FF00 survient ; relâché à la fin de la commande.
- **DMA** : si mis lors du push, mode DMA immédiat ; relâché à la fin de la commande.

5.3 Registre d'état (\$DF1C en lecture)

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
DATA_AV	STAT_AV	STATE	STATE	ERROR	ABORT_P	DATA_ACC	CMD_BUSY

États (bits 5–4) : **00** = Idle ; **01** = Command Busy ; **10** = Data Last (dernier bloc) ; **11** = Data More (données restantes). DATA_AV = données disponibles dans \$DF1E ; STAT_AV = status disponible dans \$DF1F ; ERROR = commande envoyée hors idle ; ABORT_P = abort en attente de traitement ; CMD_BUSY = commande en attente.

5.4 Poignée de main (handshake), octet par octet

1. **État Idle** (bits STATE = 00 dans \$DF1C) : écrire la commande **octet par octet** dans \$DF1D (1^{er} octet = numéro de cible).
2. Écrire '1' sur **PUSH_CMD** (\$DF1C) → transition vers **Command Busy**.
3. L'Ultimate prépare réponse et status, puis passe en **Data Last** ou **Data More**.
4. Lire les données dans **\$DF1E** et le status dans **\$DF1F** tant que DATA_AV / STAT_AV valent '1' (ils passent à '0' quand tout est lu).
5. Écrire '1' sur **DATA_ACC** : si l'état était Data Last → retour **Idle** ; si Data More → retour **Command Busy** (bloc suivant), reprendre à l'étape 3.

En clair — ce que la liste ci-dessus révèle : \$DF1C est le **plan de contrôle** (états et drapeaux — tout s'y lit et s'y commande), les octets transitent par le **plan de données** \$DF1D–\$DF1F. Et la machine doit être au repos (Idle) avant tout dépôt, sinon drapeau ERROR.

Tailles de files (= taille max de transfert par commande) : **commande 896 octets (\$380)**, **données de réponse 896 octets (\$380)**, **status 256 octets (\$100)**. Le premier octet de la commande désigne la cible (dispatcher léger) ; en firmware 2.6, seule cible : « Ultimate-DOS », en **deux instances aux cibles 1 et 2** (deux répertoires/fichiers ouverts simultanément possibles, un par cible).

6. Cible DOS (targets \$01 / \$02) [ultimate_dos_target.rst]

Accès programmatique au système de fichiers de l'Ultimate. Premier octet de commande = **\$01** ou **\$02** (deux instances indépendantes, chacune son état). Exemples ci-dessous sur \$01. Le status est une chaîne type CBM DOS (00, 0K, etc.).

Opcode	Commande	Format	Réponse (données)	Status possibles
0x01	IDENTIFY	\$01 \$01	Chaîne d'identification, ex. « ULTIMATE-II DOS V1.0 »	00, 0K (ne peut échouer)
0x02	OPEN_ FILE	\$01 \$02 <attrib> <filename>	aucune	00, 0K ou erreur FS
0x03	CLOSE_ FILE	\$01 \$03	aucune	00, 0K ; 84, NO FILE TO CLOSE
0x04	READ_ DATA	\$01 \$04 [len_lo] [len_hi]	données du fichier, par blocs de 512 octets max (accepter chaque bloc via DATA_ACC), total max 65 535 octets	silence si OK ; 85, NO FILE OPEN (+ paquet vide)
0x05	WRITE_ DATA	\$01 \$05 [dummy] [dummy] [data ...]	aucune (2 octets dummy = alignement long-word ; 512 octets/commande conseillé)	85, NO FILE OPEN ; ACCESS DENIED si pas ouvert en écriture
0x06	FILE_ SEEK	\$01 \$06 [posL] [posML] [posMH] [posH] (32 bits LSB first)	aucune	00, 0K ; 85, NO FILE OPEN ; erreur FS
0x07	FILE_ INFO	\$01 \$07	struct: uint32 size ; uint16 date ; uint16 time ; char extension[3] ; uint8 attrib ; char filename[]	00, 0K ; 85, NO FILE OPEN ; 88, NO INFORMATION AVAILABLE
0x08	FILE_ STAT	\$01 \$08 <filename>	même struct que 0x07	00, 0K ; 88, FILE NOT FOUND
0x09	DELETE_ FILE	\$01 \$09 <filename>	aucune	00, 0K ou erreur FS (v1.1+)
0x0A	RENAME_ FILE	\$01 \$0a <filename> \$00 <newname>	aucune	00, 0K ou erreur FS (v1.1+)
0x0B	COPY_ FILE	\$01 \$0b <source> \$00 <destination>	aucune	00, 0K ou erreur FS (v1.1+)
0x11	CHANGE_ DIR	\$01 \$11 <directory name> — . et .. valides ; peut « entrer » dans un .D64 (sous-système de fichiers)	aucune	00, 0K ; 83, NO SUCH DIRECTORY

Opcode	Commande	Format	Réponse (données)	Status possibles
0x12	GET_PATH	\$01 \$12	chemin courant depuis la racine	00,OK (ne peut échouer)
0x13	OPEN_DIR	\$01 \$13	aucune	00,OK ; 01,DIRECTORY EMPTY ; 86,CAN'T READ DIRECTORY (voir 0x13 pour l'ouverture)
0x14	READ_DIR	\$01 \$14	un paquet de données par entrée : 1 octet d'attribut + nom de fichier (déprécié depuis firmware 3.0)	99,FUNCTION NOT IMPLEMENTED
0x15	COPY_UI_PATH	\$01 \$15		
0x16	CREATE_DIR	\$01 \$16 <dirname>	aucune	00,OK ou erreur FS (v1.1+)
0x17	COPY_HOME_PATH	\$01 \$17	enchaine sur GET_PATH (renvoie le chemin courant)	erreur FS si le home n'existe pas (v1.1+)
0x21	LOAD_REU	\$01 \$21 [addrL] [addrML] [addrMH] [addrH] [lenL] [lenML] [lenMH] [lenH]	chaîne détaillée, ex. « \$003000 BYTES LOADED TO REU \$126800 »	00,OK ; 02,REQUEST TRUNCATED ; erreur FS
0x22	SAVE_REU	\$01 \$22 [addrL] [addrML] [addrMH] [addrH] [lenL] [lenML] [lenMH] [lenH]	chaîne, ex. « \$008000 BYTES SAVED FROM REU \$852000 »	00,OK ; 02,REQUEST TRUNCATED ; erreur FS
0x23	MOUNT_DISK	\$01 \$23 <id> <filename>	aucune	00,OK ; 89,NOT A DISK IMAGE ; 90,DRIVE NOT PRESENT (v1.1+)
0x24	UMOUNT_DISK	\$01 \$24 <id>	aucune	00,OK (même sans disque monté) ; 90,DRIVE NOT PRESENT (v1.1+)
0x25	SWAP_DISK	\$01 \$25 <id> (la source écrit « \$0x25 », coquille) — équivaut au menu-button > 1 s	aucune	00,OK ; 90,DRIVE NOT PRESENT (v1.1+)
0x26	GET_TIME	\$01 \$26 [fmt] — fmt 0/absent : yyyy/mm/dd hh:mm:ss ; fmt 1 : www yyyy/mm/dd hh:mm:ss	date/heure	00,OK (v1.2+)
0x27	SET_TIME	\$01 \$27 <Y> <M> <D> <h> <m> <s> (source : « \$0x27 », coquille) — année — 1900	aucune	00,OK ; 98,FUNCTION PROHIBITED si désactivé (v1.2+)
0xF0	ECHO	\$01 \$F0	écho de la commande	00,OK (ne peut échouer)

Attributs OPEN_FILE (0x02) : FA_READ = \$01 ; FA_WRITE = \$02 ; FA_CREATE_NEW = \$04 (tronque à 0 octet) ; FA_CREATE_ALWAYS = \$08 (peut écraser). Lecture seule = \$01 ; écriture sur fichier existant sans le vider = \$02 ; écriture avec écrasement systématique = **\$0E**. Le nom de fichier n'est **pas** null-terminé (la longueur de commande fait foi).

Octet d'attribut READ_DIR (0x14) (repris du format FAT, réutilisé pour les FS non-FAT) :

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
—	—	ARCHIVE	DIR	VOLUME	SYSTEM	HIDDEN	READONLY

LOAD_REU / SAVE_REU (0x21/0x22) : opèrent sur le fichier **actuellement ouvert** ; deux paramètres 32 bits LSB first (adresse REU, longueur). Pas de wrap-around : transfert **tronqué** si adresse + longueur dépasse la fin du REU. Les octets hauts d'adresse et de longueur sont masqués (octets dummy en pratique). **Suppose une configuration REU de 16 Mo.**

7. Cible Control (target \$04), en bref [control_target.rst]

Interface bas niveau de gestion matérielle (exécution C64, alimentation des lecteurs émulés, utilitaires REU/disque). Premier octet de commande = **\$04**.

Opcode	Commande	Format / Essentiel
0x01	IDENTIFY	\$04 \$01 → chaîne « CONTROL TARGET V1.1 » ; 00,OK
0x03	FINISH_CAPTURE	\$04 \$03 — clôt une capture cassette en cours et ferme le fichier
0x05	FREEZE	\$04 \$05 — gel matériel (équivalent bouton) ; l'acquiescement après le freeze reste problématique (à corriger, dixit la doc)
0x06	REBOOT	\$04 \$06 — reboot complet du C64 + ré-init cartouche ; aucune réponse (l'UCI est resetée avec le C64)

Opcode	Commande	Format / Essentiel
0x08 / 0x09	LOAD_REU / SAVE_REU	\$04 \$08 <FILENAME> / \$04 \$09 <FILENAME> — charge/sauve une image REU par nom de fichier (contrairement à la version DOS 0x21/0x22 qui opère sur le fichier ouvert). Réponse : 4 octets LE = octets transférés (négatif = erreur). Status : 00,0K ; 81,INVALID PARAMS ; 84,REU NOT ENABLED ; 85,REU FILE CANNOT BE OPENED.
0x0F	U64_SAVEMEM	\$04 \$0F <FILENAME> — sauve toute la RAM (U64/Elite/Elite-II/C64U seulement) ; défaut /temp/c64_memory.bin ; 00,0K ou 87,DISK ERR: <...>
0x11	DECODE_TRACK	\$04 \$11 <TRK> <MAX_SEC> <GCR_ADDR> <BIN_ADDR> <GCR_LEN> — conversion GCR→binaire rapide via REU (adresses/longueur LSB first). Réponse : 1 octet (secteurs réels) + 2 octets de drapeaux d'erreur par secteur ; 00,0K ou 82,ERRORS ON TRACK.
0x20	EASYFLASH_ERASE	\$04 \$20 \$00 <BANK> <BASEADDR> — émule l'effacement secteur EasyFlash : 64 KiB (8 banques de 8 KiB) mis à \$FF ; BANK bits 3–5 = secteur ; BASEADDR = octet haut d'adresse C64 (Low \$8000–\$9FFF / High \$A000–\$BFFF) ; 00,0K ou 81,INVALID PARAMS
0x28	GET_HWINFO (déprécié)	\$04 \$28 <SUB_CMD> — \$00 : nom du modèle (ex. « ULTIMATE 64 » ; défaut sans sous-commande depuis 3.15) ; \$01 : config SID (1 octet = nb de trames, puis 5 octets/trame : base primaire 2 octets LSB first, base secondaire 2 octets MSB first, indicateur de type au sens flou)
0x29	GET_DRVINFO	\$04 \$29 <EFFECTIVE_ADDR_FLAG> — octet 0 = nombre de lecteurs, puis groupes de 3 octets [Type] [Adresse bus IEC] [État alim.]. Types : 1541=\$00, 1571=\$01, 1581=\$02, Undecided=\$03, Software IEC=\$0F, Printer=\$50 ; alim \$00/\$01
0x30–0x33	ENABLE/DISABLE_DRIVE_A/B	\$04 \$30 (A on), \$04 \$31 (A off), \$04 \$32 (B on), \$04 \$33 (B off) ; 00,0K
0x34 / 0x35	GET_DRIVE_A/B_POWER	\$04 \$34 / \$04 \$35 → réponse on ou off ; 00,0K
0x40	GET_MP3_RAMDISKINFO	\$04 \$40 — spécifique GEOS MegaPatch 3 : bloc de 8 octets (2/lecteur) ; 1 ^{er} octet = type (\$41, \$71, \$81 ou \$DD = Native) ; 2 ^e = taille en blocs de 64 KiB pour les partitions natives, sinon 0

Annexe A — Le C64 Ultimate mesuré (firmware Commodore 1.0, juillet 2026)

Ici commence l'apport original du volume : ce que la machine réelle a répondu quand on lui a posé les questions des chapitres 1 à 6. Chaque fait porte son étiquette — mesuré, documenté ou hypothèse.

Cette annexe consigne des comportements observés sur un **Commodore C64 Ultimate** — la réédition FPGA du Commodore 64 vendue sous la marque Commodore, dont le firmware (« Commodore 1.0 ») est un dérivé réduit du firmware Ultimate 64 de Gideon Zweijtzter. Toutes les observations datent de juin-juillet 2026 et proviennent d'un **exemplaire unique** ; elles ne valent pas automatiquement pour toute la série ni pour les firmwares ultérieurs — re-vérifier sur votre machine.

Convention d'étiquetage, conservée fait par fait :

- **[mesuré]** : reproduit sur le matériel décrit ;
 - **[documenté]** : affirmé par une documentation de référence (section « Sources et crédits ») sans remesure indépendante ;
 - **[hypothèse]** : explication plausible, non prouvée.
-

A.1 Le turbo 64 MHz

- **[mesuré]** Le registre \$D031 pilote la vitesse CPU (bits 0-3 = index de vitesse, bit 7 = timing badline) quand l'option de configuration « Turbo Control » vaut « **C64U Turbo Registers** » — c'est le nom affiché par ce firmware ; la documentation amont (Gideon) appelle la même option « U64 Turbo Registers ». \$D031 = \$0F donne 64 MHz constants. Écrire \$D031 est inoffensif sur un C64 d'origine (registre VIC inutilisé) : un même binaire peut donc s'auto-configurer sans détection préalable.
- **[mesuré]** **Re-basculer la vitesse en cours d'exécution est interdit** : changer \$D031 pendant que des écritures RAM sont en vol **réinitialise le firmware de la machine** (écran noir). **[hypothèse]** Hazard de traversée de domaines d'horloge dans le FPGA. Règle pratique : choisir la vitesse au démarrage et ne plus jamais y toucher.
- **[mesuré — chiffre concordant avec la documentation turbo officielle, pas une mesure indépendante du chiffre]** **La badline gèle le CPU même à 64 MHz** : environ 43 µs par badline. Le cœur FPGA fige le CPU rapide pendant que le VIC tient le bus (réglage « badlines activées » = timing compatible C64). Toute technique qui force des badlines (FLI notamment, annexe B) paie donc ce prix plein, quelle que soit la vitesse CPU.
- **[mesuré]** Conversion utile pour tout timing logiciel au turbo : **1 cycle VIC volé ≈ 13 itérations d'une boucle DEY/BNE** (5 cycles CPU par itération à 64 MHz ; $64 \div 5 = 12,8$). C'est l'unité de base de toute compensation calculée.

A.2 Accès mémoire DMA via l'API REST

Le firmware expose une API REST ; machine:readmem et machine:writemem lisent et écrivent la RAM du C64 par DMA, machine en marche.

- **[mesuré]** readmem est **fiable à tout moment**, y compris pendant l'exécution au turbo — l'outil d'observation de choix (compteurs, sondes, instantanés d'écran). **[documenté]** Chaque lecture vole un peu de bus : micro-perturbation possible d'un affichage au timing critique pendant la lecture.
- **[mesuré]** writemem **pendant l'exécution au turbo n'est pas fiable** : octets perdus ou corrompus. Un poke isolé survit s'il est relu puis réécrit jusqu'à confirmation ; un flux d'écritures, non. Corollaire pratique : les paramètres d'un test passent par la compilation, jamais par des écritures DMA à chaud.
- **[mesuré]** Un writemem effectué **avant** un lancement de programme (runners:run_prg) vers la zone \$0200-\$03FF est **perdu** : le reset qui précède le chargement exécute la routine KERNAL RAMTAS, qui efface cette zone (visible dans le désassemblage de la ROM). Corollaire : le tampon cassette (\$033C-

\$03FB) n'étant pas couvert par un fichier .prg, son contenu au démarrage est **indéterminé** — toute variable qu'on y loge doit être initialisée explicitement par le programme.

- [mesuré] Deux appels de configuration sont **dangereux** : configs:reset_to_default remet immédiatement **tous** les réglages utilisateur aux défauts usine (perte sèche de la configuration) ; configs:save_to_flash, combiné à certaines configurations réseau/flux, réveille au boot suivant un bug du firmware — **écran noir persistant**, machine qui répond pourtant au ping. Récupération : **maintenir RESTORE enfoncée à l'allumage** (efface la configuration en flash). Explorer l'arbre configs en GET uniquement ; ne sauver en flash que ce qu'on a vérifié.
- [mesuré] **Fuite de fichiers temporaires** : chaque envoi de programme par l'API crée un fichier /Temp/tempNNNN jamais supprimé. Vers ~330 envois, le disque RAM interne est plein et l'API répond « Cannot open file » (404) — cohérent avec la memory map du firmware, où le disque RAM est un bloc fixe de 16 Mo (ch.5 §5.5). Purge **par FTP** (commandes DELE en lot), **sans redémarrer** la machine.

A.3 Le fork Commodore face à la documentation Gideon

- [mesuré] Sur ce firmware : l'UCI (Ultimate Command Interface) cœur fonctionne, le DMA REU fonctionne, l'API REST de configuration fonctionne ; en revanche les **sockets réseau UCI bloquent** (l'appel ne rend jamais la main) et l'accès au registre audio **\$DF20** documenté côté Gideon **réinitialise la machine**. S'ajoute la divergence de nom d'option turbo (« C64U Turbo Registers » vs « U64 Turbo Registers », §A.1).
- Règle générale qui en découle : la documentation Gideon décrit le firmware **amont** ; sur le fork Commodore, chaque service documenté est une **base à vérifier** sur la machine réelle, jamais un acquis.

A.4 Flux vidéo UDP

Le firmware peut diffuser la sortie VIC en temps réel sur Ethernet (UDP).

- [mesuré] Avant streams/video:start vers une destination donnée, il faut **amorcer la résolution ARP** (quelques pings entre la machine et la destination), sinon l'API répond « Resolve Error ».
- [mesuré] Un lancement de programme (run_prg) **réinitialise l'état du flux** : refaire stop, re-définir la destination, puis start à chaque session de capture.
- [documenté + mesuré] Le transport est de l'UDP sans garantie : des paquets se perdent réellement. Pour toute analyse d'image, **ne retenir que les trames complètes** (compter les paquets entre deux marqueurs de fin de trame et jeter les trames lacunaires).

A.5 REU : DMA fiable, mise en place coûteuse

- [mesuré] Le DMA REU ponctuel est fiable (commande \$90/\$91 écrite dans \$DF01, exécution immédiate) — parfait pour ranger puis rappeler des blocs de données aux changements d'écran. Mais la **latence de mise en place est d'environ 60 µs par transfert** (*ordre de grandeur, mesuré indirectement par différence de temps de trame entre variantes avec/sans transferts*) : inadapté aux micro-transferts répétés dans une boucle de rendu, où des dizaines de petits DMA par trame coûtent plus qu'ils ne rapportent. Peu de gros transferts : oui ; beaucoup de petits : non.

A.6 Manette au turbo

- [mesuré] À 64 MHz, la lecture de la manette (\$DC00, port 2) est polluée dans ~43 % des cas (*ordre de grandeur, mesuré par échantillonnage répété du port sur bancs*) si l'on laisse faire le scan clavier et la direction du port. Correctif mesuré : **couper la source d'interruption du scan clavier** (écrire \$7F dans \$DC0D, puis lire \$DC0D — sur le CIA, la lecture du registre d'interruption acquitte) et **figer DDRA en entrées**. Après quoi la lecture est stable.

A.7 PETSCII et le piège des simulateurs

(Leçon de chaîne d'outils plutôt que de silicium — elle figure ici parce qu'elle frappe précisément qui programme cette machine avec les outils croisés modernes.)

- **[mesuré]** La chaîne d'outils cc65 ciblant le C64 traduit les littéraux de caractères en **PETSCII** ('A' = \$C1, pas \$41). Le même code assemblé pour la cible simulateur générique (sim6502) garde l'**ASCII** : un test qui passe au simulateur **ne valide donc pas** un mapping de caractères destiné à la machine réelle. Deux parades : écrire les correspondances caractère→index en C (le compilateur traduit pour chaque cible), et valider tout affichage de texte **sur le matériel**, jamais seulement au simulateur.
-

Annexe B — Étude de cas : servir du FLI 16 couleurs à 64 MHz par polling

Et voici la confrontation en action : un cas complet, du principe à la calibration, avec ses pièges mesurés. Les constantes appartiennent à un exemplaire ; la méthode est le livrable.

Cette étude de cas montre comment afficher du **FLI** (Flexible Line Interpretation : une badline forcée et une matrice écran différente à **chaque ligne raster**, pour lever la limite des attributs couleur par blocs de 4×8 pixels) sur le C64 Ultimate de l'annexe A, **au turbo 64 MHz constant** — puisque re-basculer la vitesse est interdit (annexe A, §A.1). Le FLI classique se sert à 1 MHz, sous interruption raster, avec du code compté au cycle. Ici, ni l'un ni l'autre : tout ce qui suit a été relevé sur l'exemplaire décrit en annexe A — chaque item porte son étiquette [mesuré]/[hypothèse]. Les constantes numériques (délais, offsets) sont **propres à cette machine** ; c'est la **méthode de calibration** qui se transporte. Le cas d'usage mesuré : un moteur de rendu **temps réel plein écran** (matrices intégralement réécrites à chaque trame, calcul logé dans les bordures) — seule la charge importe, pas la nature de l'application.

Étiquettes : [mesuré] = reproduit sur le matériel ; [hypothèse] = explication plausible non prouvée. Les ancres [Bauer §...] et [pal.timing] renvoient aux sources de la section « Sources et crédits ».

B.1 Le principe : pas d'IRQ, le gel badline comme synchro

- Cadre logiciel : **SEI permanent**, KERNAL et BASIC débranchés (\$01 = \$35), aucun vecteur d'interruption utilisé. Le service d'affichage est une boucle qui, pour chaque ligne : **polle \$D012** jusqu'au changement de ligne, brûle un **délai logiciel calibré**, écrit \$D018 (choix de matrice) puis \$D011 (badline forcée), et recommence. [mesuré] Ce schéma tient 200 lignes par trame, indéfiniment, sans dérive.
- Le point contre-intuitif : le **gel badline** (~43 µs même à 64 MHz, annexe A §A.1) fige le CPU à un point déterministe de chaque ligne — il **ré-aligne** le logiciel sur le raster à chaque itération. Le défaut devient l'horloge : nul besoin de code compté au cycle, la machine resynchronise d'elle-même. [mesuré]
- Le prix : ~200 badlines forcées × ~43 µs ≈ **8,6 ms gelées par trame de 20 ms**. Le débit CPU utile est à peu près **divisé par deux** par rapport au même turbo sans FLI. Tout le calcul lourd doit loger dans les bordures haut/bas de l'écran. [mesuré]

B.2 La fenêtre d'écriture de \$D011

- Théorie [Bauer §3.7.2] : le compteur de rangée RC et le compteur vidéo VC obéissent à des règles fixées aux cycles 14-15 de chaque ligne. Forcer la condition badline **trop tôt** dans la ligne remet RC à zéro : l'image s'effondre (symptôme observé : un aplatissement uniforme, la même rangée répétée). La règle pratique : **l'écriture de \$D011 doit tomber après le cycle 14 de la ligne**. [mesuré, conforme à Bauer]
- Au turbo, le positionnement se fait par une boucle de délai DEY/BNE calibrée (~13 itérations par cycle VIC, annexe A §A.1). Sur l'exemplaire mesuré, 240 itérations placent l'écriture vers le **cycle ~19-21** (fourchette nominale du délai : ~16-22). [mesuré]
- **Le piège de validation** [mesuré] : une plage large de délais (180 à 320 itérations sur cette machine) donne une « scène complète » — VC/RC corrects, image entière. Ce critère **ne suffit pas**. La contrainte réellement serrée est celle des **c-access** : une badline déclenchée au cycle T ne re-lit que les colonnes T-14 à 39 ; les colonnes plus à gauche affichent la rangée **précédente**, restée dans le tampon de ligne interne du VIC — aucune donnée en RAM ne peut corriger ce qui n'a pas été re-lu. Résultat : bord gauche corrompu bien avant que la scène ne casse. Marge réelle mesurée entre « bord propre » et « trop tard » : **~2 cycles**. Et ne jamais raccourcir le délai au point de passer sous le cycle 14 (reset RC, cf. ci-dessus). Calibrer sur le critère le plus exigeant, pas sur le plus visible.

B.3 La rotation des 8 matrices et son offset machine

- Le FLI utilise 8 matrices écran, sélectionnées cycliquement par \$D018, une par ligne de chaque rangée de caractères. **[mesuré]** La correspondance entre l'index écrit à la ligne N et la matrice réellement **consommée** par le VIC présente un **décalage constant qui doit être calibré sur la machine cible** : sur l'exemplaire mesuré, un seul offset de rotation (7) donnait un décalage vertical constant — compensable une fois pour toutes dans les données ; un offset voisin (1) produisait un décalage **alternant** +8/+16 pixels, soit des tirets sur toutes les arêtes de l'image. La valeur est **dépendante de la plateforme** : à re-calibrer sur tout autre exemplaire, révision de silicium ou cœur FPGA.
- **[hypothèse]** Le pipeline du cœur vidéo introduit une latence fixe entre l'écriture de \$D018 et le c-access qui la consomme ; l'offset de rotation la compense.

B.4 Le FLI-bug : 24 pixels irréductibles, masquables

- [Bauer §3.14.3] Sur une badline forcée par ce procédé, les trois premiers c-access lisent \$FF : les **24 pixels de gauche** sont inutilisables — « there is no way around that » (Bauer). **[mesuré]** Confirmé sur cette machine.
- Parades : \$D016 avec CSEL=0 masque 7 pixels [Bauer §3.4] ; le reste se couvre avec des **sprites** posés sur la colonne de gauche (ici deux sprites étendus en Y et multiplexés verticalement, environ 56 pixels couverts marge comprise). La « bordure gauche élargie » devient un élément de design assumé. **[mesuré]**

B.5 Le décalage +4,5 cellules : le piège du nibble

- **[mesuré]** Sur cette implémentation, l'affichage FLI consomme la matrice avec un décalage de **+4,5 cellules** : la cellule écran C affiche le **nibble bas de la cellule C-5** et le **nibble haut de la cellule C-4**. Tout code qui écrit directement dans les matrices doit donc émettre un flux de nibbles **pré-décalé** d'autant — la parade est purement dans les données, et elle est fiable.
- Méthode de sonde **[mesuré]** : écrire des motifs de test dont les deux nibbles **diffèrent** ($hi \neq lo$). Un motif $hi=lo$ rend le demi-décalage invisible et masque entièrement le phénomène — on croit l'affichage sain alors qu'il ne l'est pas.
- **[hypothèse]** Classe d'explication : interaction entre l'écriture de \$D018, le compteur VMLI et le registre à décalage du VIC [Bauer §3.7.2]. Ce type de comportement est réputé instable sur silicium (il varie selon la révision et la température) ; sur l'implémentation FPGA il est **déterministe**, ce qui rend la compensation par les données sûre.

B.6 Sprites actifs : les vols DMA qui font glisser l'écriture

- Théorie [pal.timing] : les s-access d'un sprite actif tombent à des **cycles fixes déterminés par le numéro de sprite** (le « slot »), à cheval sur la fin de la ligne précédente et le début de la ligne courante — jamais par sa position X.
- **[mesuré ; mécanisme causal = hypothèse forte, cohérente avec les chronogrammes de Mäkelä ([pal.timing])]** Conséquence sur un displayer par polling : les slots dont les vols tombent en **fin de ligne** (slots 0-2, cycles ~58-63) sont absorbés par la phase de **poll** — le programme attendait de toute façon le changement de ligne : aucun effet. Les slots dont les vols tombent en **début de ligne** (slots 3-7, cycles ~1-10) frappent **pendant la boucle de délai** : l'écriture de \$D011 glisse d'autant, et les colonnes de gauche sont fausses **uniquement sur les lignes affichées du sprite**. Effet visuel : un « peigne ».
- Compensation **[mesuré]** : une **table de délais par ligne** (délai nominal moins ~13 itérations par cycle volé, appliquée exactement aux lignes affichées du sprite, Y+1 à Y+H) annule le peigne d'un sprite isolé — table **calibrée empiriquement** ligne à ligne (la variante *calculée a priori* depuis les chronogrammes n'a pas été testée). Deux pièges, tous deux mesurés :
 1. **Compenser une ligne saine la casse** : l'écriture passe sous le cycle 14 → reset RC, dégât pleine largeur. Aucune « marge de sécurité » possible — la plage compensée doit être exacte.

2. **Deux sprites volant sur la même ligne** (un slot fin de ligne + un slot début de ligne) ne s'annulent pas avec un seul bouton de délai : là, **masquer ou réorganiser les slots**, pas compenser.
- Corollaire de design [**mesuré**] : placer les objets mobiles sur des slots à compensation connue (3-7) ou nulle (0-2), et raisonner **par slot**, jamais par sprite individuel.

B.7 La leçon de calibration : un banc statique ment

- [**mesuré**] Un compteur de dépassements (le service d'affichage part-il en retard sur la trame ?) logé en RAM et lu de l'extérieur par DMA affichait **zéro** sur un banc à scène figée — charge de calcul réduite. En **charge réelle** (déplacement, objets animés, +18 % de cellules de matrice écrites par trame), le même réglage débordait par intermittence : artefacts clignotants à position variable, la signature d'une **course** et non d'un timing fixe. Le banc disait vrai... sous une charge que l'usage réel dépassait.
- Règles qui en découlent [**mesuré**] :
 - calibrer **au pire cas de charge**, jamais sur une scène figée ;
 - instrumenter à **deux niveaux** — retard de début de trame **et** retard par ligne : un compteur global n'attrape pas les glissements ligne à ligne ;
 - **étaier** tout gros transfert vers les matrices sur plusieurs trames (de l'ordre de 8 rangées de caractères par trame) plutôt qu'un blit massif dans le temps de bordure.

B.8 Ce qui se transporte, ce qui ne se transporte pas

Se transportent : le schéma polling + gel badline comme synchro, la règle « \$D011 après le cycle 14 », la sonde $hi \neq lo$, le raisonnement par slot de sprite, la discipline de calibration en charge. Ne se transportent **pas** : les constantes (≈ 13 itérations par cycle, 240 itérations de délai, offset de rotation 7, +4,5 cellules) — chacune est le produit d'une calibration sur **une** machine, à refaire sur la vôtre par la même méthode : déployer, capturer la sortie vidéo réelle, compter les colonnes fausses.

Sources et crédits

Références VIC-II et timing

- **Christian Bauer**, *The MOS 6567/6569 video controller (VIC-II) and its application in the Commodore 64*, révision du 2024-09-29. Texte de référence : <https://www.cebix.net/VIC-Article.txt> — des copies (parfois de révisions antérieures) circulent sur <http://www.zimmers.net/>. Les ancres [Bauer §x.y] des chapitres et annexes renvoient à sa numérotation de sections.
- **Marko Mäkelä**, *The memory accesses of the MOS 6569 VIC-II and MOS 8566 VIC-IIe Video Interface Controller* (dit « pal.timing »), 3 juin 1994 (Bauer cite une révision du 15 juillet 1994). <http://www.zimmers.net/anonftp/pub/cbm/documents/chipdata/> (fichier pal.timing). Chronogrammes mesurés sur matériel réel ; corrige explicitement Ojala.
- **John West, Marko Mäkelä**, *Documentation for the NMOS 65xx/85xx Instruction Set* (dit « 64doc »), 3 juin 1994 (patchlevel 2). <http://www.zimmers.net/anonftp/pub/cbm/documents/chipdata/> (fichier 64doc).
- **Pasi Ojala**, « Missing Cycles », *C=Hacking* n° 3, 1992. <http://www.zimmers.net/anonftp/pub/cbm/magazines/c=hacking/> (numéro 3). Article fondateur sur le vol de cycles du VIC-II ; approximations connues, corrigées par Mäkelä.
- **Linus Åkesson**, « Massively Interleaved Sprite Crunch », 2016. <https://linusakesson.net/scene/lunatic/misc.php> Exploitation extrême du DMA sprite, démontrée par une production fonctionnelle.

SID, CIA, REU

- **MOS Technology**, datasheet du **6581 SID** (Sound Interface Device), 12 pages. Scans : <http://www.zimmers.net/anonftp/pub/cbm/documents/chipdata/> (archive 6581.zip). Réserve : la transcription OCR (deux colonnes) entrelace des valeurs par endroits ; les valeurs douteuses sont signalées comme telles au chapitre 4. Ancre : [SID p.N].
- **Wolfgang Lorenz**, *A Software Model of the CIA6526*, version 2.15, mai 1997. Diffusé avec l'Emulator Developers Kit (accompagné de CIA6526.cpp/.h et de la suite de tests TESTSUIT.ZIP, reprise depuis par la plupart des émulateurs C64). Ancre : [Lorenz].
- **Richard Hable**, *Programming the Commodore REU* (fichier programming.reu) ; et la table *REU registers* (fichier reu.registers). <http://www.zimmers.net/anonftp/pub/cbm/documents/chipdata/> — Ancres : [REU-prog], [REU-reg].

Ultimate 64 / 1541 Ultimate-II

- **Gideon Zweijtzer (GideonZ)**, documentation officielle Ultimate-64 / 1541 Ultimate-II : <https://github.com/GideonZ/1541u-documentation> — rendu lisible sur <https://1541u-documentation.readthedocs.io>. Sections utilisées : API REST, data streams, turbo mode, UCI (architecture, cibles DOS et Control). Fichiers amont cités : api/api_calls.rst, data_streams.rst, config/turbo_mode.rst, ultimate64.rst, uci/*.rst.
- **Memory map du firmware Ultimate** (ancre [memmap]) : fichier doc/memmap.txt du dépôt firmware <https://github.com/GideonZ/1541ultimate> (sans auteur/date déclarés dans le fichier — provenance : le dépôt officiel). Attention : ces documents décrivent le firmware **amont** ; le firmware Commodore des annexes A et B en est un fork réduit qui en diverge sur plusieurs points mesurés (annexe A, §A.3).

Brevets (cités par Bauer)

- **Albert J. Charpentier et al.**, « Digital sine-cosine generator », brevet US 4,551,682, 5 novembre 1985.
- **James W. Redfield, Albert J. Charpentier**, « Display logic circuit for multiple object priority », brevet US 4,561,659, 31 décembre 1985.

Note sur les mesures

Les mesures des annexes A et B ont été réalisées en juin-juillet 2026 sur un Commodore C64 Ultimate (firmware Commodore 1.0). Document d'étude à diffusion privée. Chaque affirmation des chapitres 1 à 6 est vérifiable contre les sources ci-dessus ; les annexes A et B rapportent des **mesures originales** effectuées sur cet exemplaire (protocoles décrits dans le texte) — elles ne figurent dans aucune des sources et n'engagent que l'exemplaire mesuré.